

分类号 _____
U D C _____

学 号 DX120210090
密 级 _____



博士学位论文

(学术学位)

多智能体系统中的分布式任务规划关键技术研究

学 院： 信息工程学院（人工智能学院）
专 业： 软件工程
研 究 生： 夏宇
指 导 教 师： 朱留存 教授
答辩委员会主席： 黄海平 教授
答 辩 日 期： 2025 年 5 月 20 日

Research on Key Technologies of Distributed Task Planning in Multi-Agent Systems

A dissertation submitted to
Yangzhou University
in partial fulfillment of the requirements
for the degree of
Doctor of Engineering

By

Yu Xia

Supervisor: Prof. Liucun Zhu

Software Engineering

May 2025

多智能体系统中的分布式任务规划关键技术研究

本论文研究得到国家高端外国专家引进计划项目（编号：No.G2022033007L）、江苏省研究生科研创新计划（编号：KYCX22_3504）、广西八桂学者专项经费（编号：No.2019A08）资助。

目 录

第一章 绪论.....	1
1.1 研究背景.....	1
1.2 研究意义和目的.....	5
1.2.1 研究意义.....	5
1.2.2 研究目的.....	7
1.3 国内外研究现状与分析.....	8
1.3.1 集中式分配方法的研究现状.....	10
1.3.2 基于拍卖方法的研究现状.....	10
1.3.3 基于博弈论方法的研究现状.....	12
1.3.4 多智能体机器学习方法的研究现状.....	12
1.3.5 优化算法研究现状.....	14
1.3.6 研究现状分析.....	14
1.4 主要研究内容与主要贡献	17
1.4.1 研究内容.....	17
1.4.2 研究贡献.....	18
1.5 论文组织结构.....	20
1.6 本章小结.....	21
第二章 相关概念与研究基础	23
2.1 多智能体系统的基本概念与理论基础	23
2.2 分布式任务规划理论与建模	25
2.3 博弈论理论基础.....	28
2.4 基于博弈的多智能体分布式任务分配理论	29
2.5 本章小结.....	32
第三章 多智能体任务分解中的动态角色发现与分配	33
3.1 引言.....	33
3.2 基于角色的任务分解模型构建	36
3.3 动态角色发现与分配的方法	37
3.3.1 基于行动表征的角色发现.....	38
3.3.2 基于表征的角色选择.....	40
3.3.3 与表征相关的角色策略选择.....	42
3.4 实验结果与分析.....	44
3.4.1 实验设计.....	44
3.4.2 性能表现.....	45
3.4.3 消融实验.....	49

3.5 本章小节.....	54
第四章 多智能体系统中的动态任务最大化重分配	55
4.1 引言.....	55
4.2 基于动态任务的任务分配模型构建	57
4.3 最大化任务分配与动态任务分配方法	59
4.3.1 最大化任务分配的动态任务分布式框架	59
4.3.2 基于原始任务分配解的最大化分配方案	62
4.3.3 基于子团队的动态任务分配算法.....	63
4.3.4 动态任务驱动的任务最大化再分配算法	66
4.4 实验结果与分析.....	67
4.4.1 场景和仿真设置.....	67
4.4.2 实验结果对比分析.....	68
4.5 本章小结.....	74
第五章 基于均衡性的多智能体分布式任务分配方法	75
5.1 引言.....	75
5.2 分布式任务分配系统模型	77
5.2.1 系统模型.....	77
5.2.2 预期响应时间模型.....	78
5.2.3 智能体均衡性变化评价模型.....	79
5.3 均衡性博弈的纳什均衡.....	80
5.3.1 均衡性问题优化.....	80
5.3.2 最佳响应计算算法.....	81
5.3.3 基于纳什均衡的分布式负载均衡算法	84
5.4 实验结果与分析.....	85
5.4.1 实验设计.....	85
5.4.2 权重组合比较.....	86
5.4.3 性能比较.....	87
5.4.4 系统利用率的影响.....	89
5.4.5 系统异构性的影响.....	92
5.4.6 系统规模的影响.....	94
5.5 本章小结.....	97
第六章 多智能体系统分布式任务规划的实际应用	99
6.1 引言.....	99
6.2 多传感器融合与多机器人协同映射技术	100
6.2.1 多传感器融合定位方法.....	100
6.2.2 多机器人协同建图方法.....	101
6.3 实验设计与实现.....	103
6.3.1 硬件平台与实验软件架构.....	103

6.3.2 实验场景与任务设置.....	105
6.3.3 数据采集处理与实验评估指标.....	106
6.4 实验结果与分析.....	106
6.4.1 定位精度评估.....	106
6.4.2 地图构建质量分析.....	108
6.4.3 任务执行效率与负载均衡性评估.....	110
6.4.4 综合对比与讨论.....	110
6.5 本章小结.....	112
第七章 总结与展望	113
7.1 总结.....	113
7.2 展望.....	114
参考文献.....	115

摘要

多智能体系统近些年来是人工智能、机器人技术以及分布式计算领域的研究焦点，被广泛应用于机器人集群、无人机协同、智能交通、分布式能源管理等诸多领域。它具有强大的协作能力以及动态任务适应性，已成为解决复杂任务分配问题的关键手段。现有的任务分配和规划方法在动态环境、不确定性任务需求以及负载均衡优化这些方面仍然面临着不少挑战，如系统适应性不够、资源利用率低以及任务分配不均衡等问题。本文着重关注多智能体系统里的分布式任务规划关键技术研究，系统地探寻怎样借助创新的理论建模与优化方法，去提高多智能体任务分配的效率、负载均衡以及系统适应性。本文的研究工作围绕下面这些研究内容开展：

针对现有多智能体角色分配方法大多采用预定义角色或者静态映射，以至于在任务类型、执行状态或者环境条件发生变化时，没办法及时自适应地进行角色切换的问题，设计一种动态角色发现与分配机制，它可支持依据角色特性、任务需求以及历史贡献对智能体角色进行动态调整，提升任务分解与分配效率。引入基于动作特征的编码器，运用聚类算法对动作贡献进行量化，以此构建动态角色模型。在角色选择中引入奖励地平线机制，优化角色切换频率，平衡角色间的学习共享与多样性，构建一个基于部分可观测马尔可夫决策过程的角色分解框架，让角色的选择与策略学习在限定的动作空间中得以完成。

针对现有任务分配方法大多以静态环境作为前提，忽略了在实际动态环境中智能体对新任务的感知和适配能力，任务的增加、修改或者删除对初始分配方案造成巨大挑战的问题，设计一种基于子团队的任务重分配机制用来解决动态任务插入、删除和状态变更问题，最大限度地增加任务分配数量。提出新的任务成本计算机制在任务分配过程中实现灵活的成本调整。设计任务驱动的重分配算法，凭借子团队结构和任务释放机制动态响应任务列表的变化，在分布式框架中利用子团队缩小任务响应范围，以此降低通信与计算开销，依靠数值模拟与实验测试，评估该机制在任务完成率、计算效率以及动态响应能力方面的改进效果。

多智能体系统里资源利用不均衡的情况时常出现，这会致使部分智能体出现过载现象，而另一些智能体则存在资源浪费问题，最终对系统的长期稳定性以及可靠性产

生影响。本文给出一种多智能体分布式任务分配方法，此方法基于均衡性，能优化系统资源利用率，还可避免个体负载过重。构建多目标优化模型，将均衡性指标引入其中，对任务分配做出动态调整，把博弈论和分布式算法相结合，以此解决智能体之间的任务分配冲突，在动态环境里借助实时反馈来调整任务分配方案，达成智能体资源利用最大化以及负载均衡。在异构智能体场景中，借助实验对负载均衡以及效率指标开展量化评估，验证该机制有优越性。

最后在具身智能的基础上，借助多机器人系统实际平台，对本论文所提理论方法加以验证。实验运用多传感器融合技术达成高精度定位与环境感知，同时采用多机器人协同映射技术，构建统一全局环境地图，验证了动态角色发现与分配、动态任务最大化重分配以及负载均衡方法在真实复杂环境中的适应性和有效性。实验结果说明，这些方法提高了机器人系统在动态任务环境下的定位精度、地图构建质量以及任务执行效率，还实现了机器人间的负载均衡，证实理论方法的实际应用价值。

关键词：多智能体系统；分布式系统；任务规划；任务分配；动态任务

Abstract

In recent years, multi-agent systems have become increasingly popular in the fields of artificial intelligence, robotics, and distributed computing. Such systems are now often used in robot swarms, drone work together, intelligent transportation systems, and managing distributed energy. Because they enable multiple agents to work well together and adjust tasks according to the situation, they are particularly useful in dealing with complex task allocation problems. However, current methods still encounter many problems in the actual situation, such as the environment often changes, the task needs are not very certain, and how to do the problem of equitable distribution. New theoretical models and optimization methods are used to improve the efficiency of task distribution, make distribution more equitable, and allow the system to adapt to different situations better. Several important innovations and research results are presented in the dissertation:

Many multi-agent system role allocation methods are assigned according to the pre-set role, or fixed mapping relationships, so that when the task type, execution state, or environmental conditions suddenly change greatly, it is difficult to adjust the role in time. This dissertation has come up with a new approach, that is, dynamic role discovery and distribution mechanism, which will intelligently adjust the role allocation according to the characteristics of the role, the specific needs of the task, and past performance, so that the task decomposition and distribution become more efficient. This ensures both knowledge sharing and character diversity. The dissertation also proposes a role-decomposition framework based on a partially observable Markov decision-making process that allows for better choice of roles and learning strategies in the context of limited space for movement.

Many task assignments now assume that the environment is constant, without taking into account the ability of the agent to cope with new tasks in reality. When tasks suddenly increase, decrease, or change in content, the original plan encounters great problems. This dissertation designs a task reallocation method based on sub-teams, specifically to handle these dynamic task situations, so that the system can complete as many tasks as possible. To verify the effect,

we did digital simulation and practical testing to see if the task was completed, the calculation speed, and the ability to cope with changes had improved.

This often happens in multi-agent systems, where some agents are too busy to use enough resources, while others are idle, and the problem of uneven resource allocation for a long time will make the whole system unstable and unreliable. This article has come up with a way to solve this problem, that is, to design a distributed method that can make the task allocation more uniform, so that it can make better use of resources, without making a good job of the task of a certain agent. Specifically, we have built an optimization model that considers multiple goals, and adds a balance indicator to adjust the task distribution at any time. In the actual situation, the system environment is always changing, so we also added a real-time feedback mechanism to allow the task allocation strategy to continuously improve, so that the resource utilization rate can be improved as much as possible, and the burden of each agent can be balanced. In order to verify the actual effect of this method, we have experimented on different types of agents, comparing the indicators of fairness and efficiency with specific data, and it turns out that the mechanism we proposed is indeed better than other methods.

Finally, based on Embodied Intelligence, the theoretical methods proposed in this paper are verified using a multi-robot system platform. In the course of the experiment, we mainly use multi-sensor fusion technology, which can help the robot achieve more accurate positioning and environmental identification. At the same time, we also use a multi-robot collaborative mapping method to build a complete environment map. These experiments are a good proof of the practicality of dynamic role allocation, task optimization adjustment, and load balancing in complex environments. All of this shows that the theory proposed in the dissertation is really useful in practical applications.

Keywords: Multi-Agent Systems; Distributed Systems; Task Planning; Task Allocation; Dynamic Tasks

第一章 绪论

1.1 研究背景

在当今迅速发展的信息时代，智能化与自动化技术正不断深度融入人类社会的生产与生活。伴随着传感、计算、通信和控制技术的不断进步，各类智能体已在工业生产、交通运输、环境监测、能源调度、智能制造、国防安全及服务业等众多领域得到广泛应用。传统的单智能体系统在执行简单且环境相对稳定的任务时表现良好，但在面对日益复杂的应用场景和不断提升的决策需求时，智能体的性能上限成为限制进一步发展的桎梏，往往难以独立完成目标复杂、空间分布广泛、任务异质性强且动态变化的工作。为了解决现阶段单一智能体性能上限难以提升的问题，智能体技术正从单机智能^[1-3]向多机智能转变，朝着集群化、智能化的方向发展。

多智能体系统（Multi-Agent Systems, MAS）作为一种典型的分布式智能系统，是对智能体技术的扩展，在面对复杂多变的环境时通过松散联结的自治智能体协同行动以达成共同目标，解决更为复杂的问题。多智能体系统由多个相对自治、具备一定感知、决策、执行与协作能力的智能体通过一定的交互和协调机制组成，在复杂、不确定和动态的环境中，智能体通过一定的交互与协调机制协同工作，从而共同完成单个智能体难以独立实现的复杂任务。

与单智能体或集中式控制系统相比，多智能体系统在理论和实践上均展现出诸多潜在优势，凭借局部信息共享和动态协商机制，提升了动态环境下的适应能力。首先，多智能体系统具有更高的鲁棒性和灵活性，当某个智能体出现故障或性能下降时，其余智能体能够迅速补位或分担任务，降低对系统整体性能的影响。其次，多智能体系统易于扩展和适应新需求，通过增加或移除智能体，系统能够实现动态调配资源，从而更好地应对任务规模和复杂度的变化。此外，多智能体系统的分布式感知与控制能力使其能够实现并行的广域监测与高效决策，这在实际应用场景中尤为关键。智能体之间的协同、博弈和信息共享为实现更高层次的群体智能和全局优化提供了基础条件。

在多智能体系统的研究中，多智能体任务分配^[4-6]问题是一个至关重要的核心议题，即如何将一组任务合理地分配给多个智能体以提升系统整体性能。任务分配旨在

将待完成的任务集合分配给多个智能体，达成任务完成总数最大化、完成时间最短化、系统资源利用最大化或者能源消耗最小化等目标。与传统集中式控制体系不同，多智能体系统具有分布性（系统中不存在全局控制者或全局信息集中点）、自治性（智能体独立决策）、异构性（不同智能体在能力、资源、偏好、位置 and 成本等方面存在差异）以及动态性（任务、环境及智能体状态随时间变化）等特点，这使得传统静态和一次性任务分配策略难以满足实际应用需求。

如图 1-1 所示，多智能体任务分配问题广泛存在于各类复杂应用场景中。例如，在无人机集群中，面对灾后搜救、环境监测、物资投放等任务时，需要将不同类型的和优先级的任务合理分配，以确保整个飞行编队在有限时间内完成尽可能多、最紧急或价值最高的作业目标。在仓储分拣与配送流程中，多移动机器人需要对进入仓库的订单和物品进行高效分派，以提高出库速度、减少拥堵并降低能耗。这些应用在任务种类、复杂度和执行标准上差异很大，但本质问题都是在动态、不确定且受多重约束条件影响的情形下，将一组任务合理分配给智能体集群。

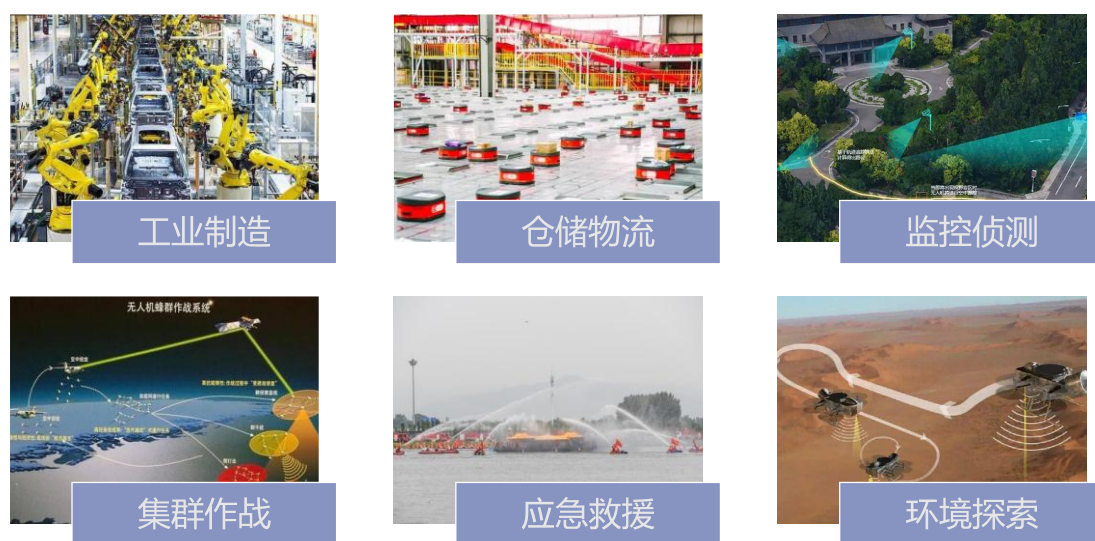


图 1-1 多智能体系统应用场景示意图

多智能体任务分配问题的优化目标通常具有多元性且需要相互权衡。一方面希望最大化任务完成率、系统收益或执行效率；另一方面又需要考虑时间约束、能耗限制、路径冲突、环境条件变化、资源稀缺及维护成本等制约因素。由于各智能体在感知能力、执行效率、机动性能、能源储备、通信带宽与计算资源方面可能存在差异，任务分配过程不仅要求实现全局最优或近似全局最优的分配，还需在局部层面充分利用各智能体的独特优势。这使得找到既兼顾全局与局部、又平衡长远与即时利益的分配策略变得更加困难，增加了问题的复杂度。

与如图 1-2(a)所示的集中式系统不同，多智能体系统强调分布式的决策与控制结

构。分布式方案在通信开销、可扩展性和鲁棒性方面具有一定优势，但也存在额外的协调成本与不确定性。为了适应分布式结构下的决策需求，研究者们提出了多种机制与框架，试图在信息与控制分散的条件下，协调多个自治智能体通过局部信息交换与独立决策，趋向某种全局最优或均衡状态。

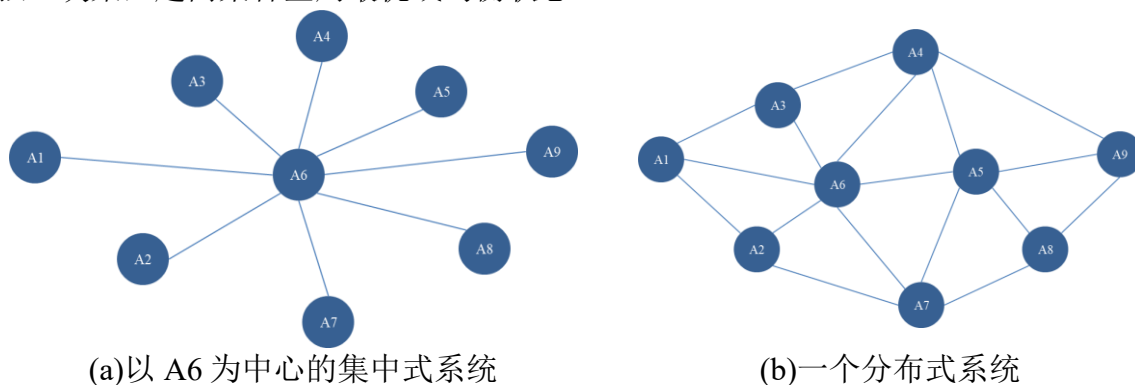


图 1-2 集中式系统及分布式系统示意图

多智能体任务分配问题不仅在理论研究里有着重要意义，在技术落地以及产业升级方面也呈现出广阔应用前景。近些年来多智能体系统渐渐从抽象的智能体^[7-9]概念朝着有真实物理载体的机器人系统发展，这样的转变让具身智能^[10-13]概念逐渐进入研究视线。具身智能强调智能体有抽象的计算能力，还拥有真实的物理实体，可借助与实际环境的交互、感知、决策以及行动，持续提升自身的适应性与鲁棒性。和传统的抽象智能体不同，具身智能更关注智能体与环境的协同演化进程，依靠实际物理交互来达成更高效、可靠的任务执行。这种研究视角的改变，给多智能体系统的任务分配方法提供了新的理论支持和实践想法。从理论方面来说，解决这个问题需要跨学科整合运筹优化、博弈论、控制理论、机器学习和人工智能等方法。在实际应用方面，需要于现实场景当中对所提出的算法以及模型加以验证与调整，以此保证最终策略可在动态且复杂的环境里发挥实际功效。

面对多智能体系统在实际应用中所面临的多重挑战，过去的几十年内研究者们提出了多种分布式任务规划方法，然而随着应用场景变得日益复杂，动态变化程度加剧，现有方法在以下几个关键领域存在不足，有待进一步探索与突破：

（一）面向动态环境的角色分工与任务分解问题

当下众多现有研究在任务分解以及角色分配阶段依然有着颇为突出的局限性。在多智能体协同工作的进程中，任务一般呈现出复合性以及层次化的特性，需要先把复杂任务分解成多个子任务，接着依据子任务的类型、复杂程度、时限以及资源需求将它们分配给恰当的智能体。然而现有研究在任务分解和角色分配阶段还是存在较大的

局限，传统框架中角色大多时候是由设计者预先定义的，缺乏根据环境变化进行动态调整的机制。当外部条件像任务数量、任务类型、资源可用性以及环境约束等发生改变时，预先定义的角色和任务适配方案大多时候会变得效率低下甚至不再适用。在一些应用场景中，多智能体系统需要应对不断出现的新任务、不断变化的任务拓扑结构，以及各个智能体在能力和资源分配方面的动态调整需求。这就需要找到一种在执行过程中可动态找出适合当前环境与任务特性的角色，并凭借自适应策略对智能体进行角色分配的办法。只有达成动态角色分工，系统才可以在复杂多变的环境里维持较高的协同效率和灵活性。

尽管已经有部分工作尝试运用机器学习、强化学习或者博弈论方法对多智能体角色进行策略性分配，但是目前仍然缺少系统化、可扩展的角色发现与分配机制。在多层次任务结构、动态拓扑以及非平稳环境中，怎样在分布式条件下高效识别角色类型，动态调整角色职责，并且让智能体可灵活切换角色以适应任务需求的变化，依旧是迫切需要解决的难题。

（二）动态环境下的任务重分配与全局目标最大化问题

在现实世界里多智能体系统所处于的运行环境大多时候呈现出动态的特性，新的任务会在系统运行进程中不断出现，已有的任务在优先级、需求量以及时间限制等方面也会发生改变，部分智能体可能会出现故障状况、资源短缺问题或者突发意外情况，外部环境如天气、地形、交通状况等变化同样会对任务的执行造成重大的影响。这就说明了即便在初始状态的时候拥有相对比较合理的任务分配方案，但是随着时间的不断推移，原先的策略说不定会渐渐失去其最优性。这使得任务重分配成为实际部署环节当中必不可少的一种机制，借助于在任务执行的过程中对分配方案适时地作出调整，系统可在持续变化的环境之中维持较高的任务完成效率以及目标达成程度。

不过任务重分配自身面临着诸多方面的挑战，首先是计算复杂度方面的问题，实时去求解最优的重分配方案或许会导致非常高的计算成本以及计算复杂度。其次是通信与协调方面的问题，多个智能体之间需要高效的信息交换以及协调机制，以此来保证重分配决策可在合理的时间范围内达成一致。另外不同任务以及智能体之间所存在的时空约束与资源冲突问题，也让任务重分配的设计变得更为复杂。

当前的研究大多是在简化条件的情形下探索任务重分配策略，而在大规模、多场景、异构性较强并且高度动态的复杂应用场景中，现有的方法在鲁棒性以及实用性方面仍然有待提升。而且多数已有的工作在进行任务重分配的时候仅仅考虑局部最优，或者只是采用贪心策略来做出调整，缺少全局的视角以及整体的优化思路，最终的分

配结果与全局最优之间依然存在着较大的差距。设计一种分布式的、可在线更新并且可以在多个时间步范围之内最大化全局目标比如任务完成总量或者系统收益的任务重分配框架是当前研究的一个关键突破点之一。

（三）多智能体任务分配中的负载均衡挑战与多目标优化问题

多智能体任务分配研究在追求整体性能最优的同时，往往忽略了智能体之间的负载均衡。这就致使部分智能体出现过载现象，而另外一些智能体的资源却处于闲置状态，如此一来便会对系统的稳定性以及长期效能产生影响。这种资源分配不均衡的状况，会对智能体的运行寿命以及维护成本造成影响，还存在降低系统整体鲁棒性的可能性，系统难以应对动态环境里的突发变化。在智能体所构成的博弈环境当中，任务分配的不均衡特性，可能会削弱智能体相互之间的信任以及合作意愿，对系统的整体协同能力产生影响。在分布式任务分配机制里，怎样在提升任务完成效率的达成率和任务在智能体之间的合理均衡，是当前研究迫切需要解决的关键问题。

任务分配的优化目标大多时候涉及效率与负载均衡之间的权衡，二者存在相互制约的关系。传统的优化方法一般倾向于将响应时间最小化或者把任务完成量最大化，却忽略了个体智能体的资源约束条件，最终导致高性能智能体出现超载情况，低性能智能体的资源利用不充分，使得系统整体性能有所下降。多智能体系统一般处于动态环境之中，任务需求、智能体状态以及通信约束都有可能随着时间发生变化，然而现有的方法大多是基于静态优化，缺少自适应动态调整机制，很难契合实时任务调度的需求。随着系统规模不断扩大，任务分配的计算复杂性急剧升高，如何在保证解的质量的同时提高求解效率，依旧是当前研究面临的挑战。由此可见怎样在保证全局效用的前提下，借助设计多目标优化模型或者激励机制，实现任务在各个智能体之间的均衡分配，保证系统可持续运行以及保持长期稳定性，已然成为当前研究的关键课题。

面对任务复杂性增加、效率和性能要求提高、分布式决策挑战以及实际应用需求，多智能体系统中的分布式任务规划有关键的理论研究意义以及实际应用价值。凭借对多智能体系统的任务规划算法以及协同策略展开研究，可有效提升系统整体效能以及执行能力，推动多智能体系统在各个领域的应用与发展。

1.2 研究意义和目的

1.2.1 研究意义

（一）对多智能体协同理论的深化与拓展

多智能体系统近年已成为交叉学科里的一个关键研究范畴，它涉及人工智能、运筹学、控制工程、博弈论以及分布式计算等多个领域。随着多智能体系统应用范围持续扩展，其理论基础和方法论急需丰富完善。多智能体任务分配是多智能体协同与规划其中的一个核心问题，可为系统在动态、异构以及不确定环境下的协作机制提供统一的理论框架和分析工具。

首先，补充分布式优化与协同控制方面的理论空白。传统集中式优化方法依靠全局信息和中央控制来达成任务最优分配，然而多智能体系统一般呈现分布式信息结构，注重智能体在局部感知和部分信息约束下达成整体最优或近似最优，对分布式任务分配方法展开系统化研究可加深对分布式优化理论与算法的理解，为构建鲁棒且可扩展的协同控制体系奠定坚实数学和算法基础。

其次，可以推动博弈论和协商机制的创新。多智能体间的角色分工和任务分配本质上有博弈特性，不同智能体有各自利益或成本函数，实现全局最优需设计合理博弈机制或协调协议。深入研究此过程，能拓宽博弈论在多智能体协同决策中的应用场景，又能为协商算法和激励机制创新提供丰富研究素材。

此外，可以丰富分布式强化学习与元启发式优化方法的应用。多智能体系统的决策问题大多时候有高维度、强耦合以及实时变化的特点。除了传统运筹优化和博弈方法外，近年来深度强化学习、多智能体强化学习和进化算法等智能优化方法逐渐进入这个领域。利用这些方法可为复杂任务分配问题提供可学习、自适应且可扩展的解决方案，丰富和深化多智能体决策理论体系。

（二）对实际应用与产业发展的支撑价值

多智能体分布式任务分配的应用潜力广泛存在于工业、农业、交通、能源、国防、医疗等诸多领域，依靠对多智能体系统的任务规划与分配加以优化，可在实际当中带来一定的经济与社会效益。

在灾害救援以及环境监测领域，多智能体无人机编队可在较大的区域范围内同时开展灾后搜救工作、进行数据采集以及实施物资运输。灾区的情况大多时候处于动态变化之中，传统的静态分配方法很难充分利用无人机资源，而先进的分布式协同规划可让无人机灵活地应对多种复杂状况，有效减少救援时间并且提升救援覆盖率，

仓储物流以及制造业普遍采用搬运机器人和协作机器人来提升生产以及运输效率。在动态订单和异构机器人协作的场景下，实时且协同的任务分配可降低拣货时间，还可以提高仓储空间利用率，任务分配算法可为智能制造以及自动化物流提供有力的技

术支持。

在交通系统以及智能网联车队领域，对于机器人车队、无人驾驶出租车队以及公共交通调度，多智能体分布式决策能让车辆在局部感知以及 V2X 通信条件下实现全局最优的路径规划以及交通流量分配，减少交通拥堵，提高通行效率。

在社会公共领域以及应急管理方面，公共资源的分配与运用一直是社会治理的关键课题。多智能体系统为这些领域提供了可行的自动化或半自动化协同机制。例如多个警用或医疗无人机协同执行巡逻和服务调度，需要依据突发事件位置、紧急程度、飞行器状态等多维信息进行动态任务分配，这对公共安全和城市管理有意义。

（三）对构建可持续智能系统的辅助意义

在当下人工智能技术快速发展的情形下，怎样让智能系统于动态且复杂的环境里达成长期、可持续的运行，已然成为备受瞩目的课题，多智能体任务分配所带来的协同与优化机制，给这一目标给予了有力的支持。分布式架构中某个智能体出现故障或者偏离预期时，系统可以迅速调整分配策略，实现故障智能体任务的重新分配，维持整体系统的高效稳定，降低单点故障概率。多智能体系统综合运用局部信息和全局信息平衡资源与任务分配，智能体相互支持合作提升群体智能水平。合理的任务分配策略能在智能体之间平衡资源利用和负载，让智能体工作时不会过度消耗，保持长时间健康运行，同时为系统规模扩展创造条件。高效的任务分配是发挥群体智慧以及实现智能协同的前提，这些协同与优化机制可提升系统的鲁棒性和容错能力，还可以提高自主协作、促进资源均衡，为构建可持续智能系统奠定坚实基础。

1.2.2 研究目的

本研究的主要目的是在多智能体系统理论框架下，针对复杂多变、资源异构且约束多元的应用需求，系统探讨分布式任务规划的关键技术，并力求在理论与实践层面做出一定的创新与提升。具体而言，本研究的目的从以下几个角度展开：

（一）构建适应动态环境的多智能体任务分配模型

面对多智能体系统在任务分配中如何在不确定的动态环境中进行感知、决策与协同的复杂挑战。本研究旨在深入剖析多智能体任务分配问题在不同场景下的特点，提炼出具备通用性与可扩展性的模型假设与数学描述，从而构建一套能够统一处理异构任务、智能体能力和多维资源约束的分布式决策模型，为后续算法设计提供坚实的理论指导。

（二）探究可扩展的分布式优化方法与协同协议

传统集中式算法在多智能体系统中难以适应分布式信息结构，无法兼顾大规模系统的实时性与鲁棒性。为克服这些局限，设计并实现若干可扩展的分布式算法与协同协议，使各智能体在有限通信条件下，能经局部协商快速达成局部最优，又能在全局层面接近最优分配。引入收益分配和激励机制，激励智能体在自治决策中趋向协作，结合自学习与自适应机制，依据环境和任务的在线反馈，不断调优各智能体的决策策略，在算法收敛性能、计算效率与分配质量间找到合理平衡。

（三）实现对异构智能体和多约束下的全面优化

多智能体系统由功能与资源各异的智能体构成，在资源层面需关注时间、能量、空间位置、网络带宽、数据处理能力等。在实际应用中，如无人机编队或工业物流场景，不同智能体在负载能力、飞行半径、电池续航、机械结构等方面存在差异。在任务分配策略设计中要充分考量各维度资源与性能指标，凭借多目标优化方法实现全局效用与局部资源利用的平衡，保证系统在动态变化环境中高效执行任务和均衡分配资源，保证系统的长期稳定性和可持续运行。

（四）大规模多智能体系统的效率与鲁棒性验证

实际部署时大多时候会涉及大量智能体，数量从数十到数百、数千不等，系统需要应对智能体故障、通信延迟、环境干扰以及任务突变等复杂状况。借助理论分析和实验仿真全面验证所提出的算法和框架在大规模系统中的可扩展性，期望在保证近似最优分配质量的把算法的时间复杂度控制在可行范围之内。

（五）为工业与社会应用提供参考与示范

本研究把所提出的方法与真实场景结合并开展实例化验证，借助仿真平台或者小规模实体实验，在实际约束和噪声条件下考察系统表现，努力为无人机搜救、智能调度、物流配送以及智慧城市管理等领域给予可行的解决办法跟技术思路。这些成果可为下一阶段技术落地与工程实现提供参考依据，也可为智能社会以及智慧城市建设提供坚实的技术支撑。

1.3 国内外研究现状与分析

多智能体系统是一种分布式智能系统，由多个自治智能体依据一定交互、协作或者博弈机制，共同去完成复杂任务。随着人工智能、运筹优化、控制工程以及通信技

术快速发展，多智能体系统在国防军工、机器人协同、无人机编队、智能交通、智慧物流与仓储、能源互联网等诸多领域的应用前景变得日益广阔。分布式任务规划与分配作为多智能体系统的一个关键研究议题，能为多个智能体在动态、不确定且异构环境下达成合理任务决策给予理论和技术方面的支撑，受到国内外学术界和工业界广泛关注。

当下，任务规划与分配的核心难题主要表现在怎样在分布式自治智能体框架下达成高效协同，让系统整体目标实现最大化或者近似最大化，怎样应对智能体间信息不对称、资源异构、通信受限，以及环境和任务动态变化所带来的复杂性，以及怎样在效率、能耗、效用和成本等多个目标甚至相互对立的目标之间达成有效折衷与优化。近些年来，各类研究方法和算法工具，像分布式优化、博弈论、市场机制、强化学习以及元启发式算法等，已被引入多智能体任务分配领域，针对搜索救援^[14-16]、军事行动^[17-20]、物理灾害管理^[21-24]、云计算^[25-28]及制造业资源分配^[29-32]等实际应用场景，也持续推动着理论和实践的发展。

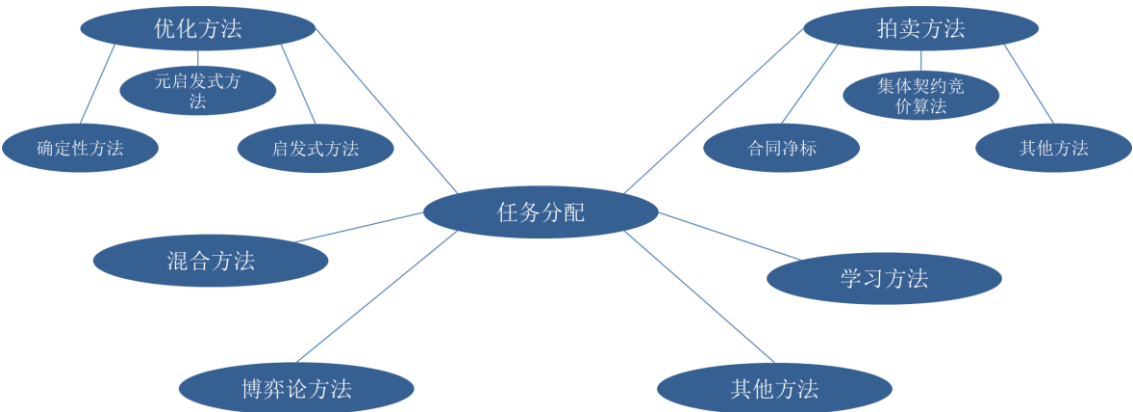


图 1-4 任务分配技术种类

用于解决多智能体系统中任务分配问题^[33,34]的技术如图 1-4 所示，主要包括基于拍卖（或市场）机制的方法、基于博弈论的方法、基于优化（启发式和元启发式算法）的方法和机器学习技术的方法。不同技术在问题可扩展性、复杂性和适应性上存在不同程度的差异。表 1-1 列出了部分代表性方法的对比，为研究者提供了技术选择的参考。

表 1-1 一些代表性任务分配方法的对比

方法类别	代表性方法或算法	优点	缺点	适用场景
集中式优化方法	LP ^[35] 、MILP ^[36]	在小规模场景下获得全局最优解	依赖全局信息；计算复杂度高，难以扩展	小规模、静态任务场景
分布式拍卖/市场机制	CNP ^[37] 、CBBA ^[38]	分布式自治、灵活扩展，适应动态任务	竞价策略设计困难，容易陷入局部	中等规模、动态环境下任务分配

		务变化	最优；通信负担较大	
基于博弈论的方法	纳什均衡 ^[39] 、Shapley 值 ^[40]	强调自治决策，适合多目标优化	高维状态空间下求解困难；协作性较弱	动态、异构系统中的任务分配
基于机器学习的方法	Q-learning ^[41] 、QMIX ^[42]	能处理环境不确定性，实现在线学习与自适应决策	训练时间长，收敛性、扩展性和安全性问题需改善	复杂动态任务分配，多目标场景
进化算法与元启发式方法	遗传算法 ^[43] 、粒子群优化 ^[44] 、蚁群优化 ^[45]	全局搜索能力强，易于并行处理，适应性较好	迭代次数多，收敛速度较慢，通常只能得到近似最优解	大规模、非线性和复杂任务分配

1.3.1 集中式分配方法的研究现状

早期的研究中集中式分配方法主要采用线性规划（Linear Programming, LP）^[35]、混合整数线性规划（Mixed Integer Linear Programming, MILP）^[36]等方法来求解多智能体的任务分配问题。这些方法利用运筹学模型和数学规划工具，依赖集中式信息和全局控制，在小规模场景中能够获得全局最优解，当问题规模扩大或环境持续动态变化时，求解复杂度和通信需求呈指数级增长，在分布式多智能体场景中的应用^[46]受到很大限制。

对于任务分配层次化结构的需求，在面对复杂或者异质任务分解情况时，部分研究在集中式框架里提出了分层调度以及任务规划办法^[47,48]，比如有的研究先于高层运用基于整数规划或者规划树的方式确定任务顺序，然后在底层细化到各个智能体的具体执行次序或者路径规划。另外有研究采用多智能体系统的 Slave 结构来进行分阶段求解。此类分层调度方法在通信资源比较充足、系统规模处于适中状态以及对最优性有着较高要求的场景中有一定的应用价值。然而当多智能体系统规模变得较大，或者需要在高度动态的环境里频繁更新任务分配方案时，层次化的集中式调度方法在实时性以及鲁棒性方面存在较大的不足。

集中式研究方法在理论层面拥有较高的解析度，可为问题提供最优或者近似最优的解决方案。但是在大规模分布式应用当中，存在通信负担较重、单点故障风险较高、可扩展性较差等缺点。随着多智能体系统规模和应用复杂度的不断攀升，研究方向逐渐转向分布式优化与自主决策，以期在分布式环境下实现高效、鲁棒的任务分配。

1.3.2 基于拍卖方法的研究现状

基于拍卖的任务分配算法^[49-51]是多智能体系统任务分配领域中的重要分支，其理论基础源于经济学，智能体根据各自对环境的局部感知，通过谈判协议在拍卖过程中根据他们计算的效用或成本为任务投标，以求实现任务完成效用最大化或成本最小化。代理的目标是为分配的任务完成最高的效用或最低的成本。拍卖者既可以是一个中央代理，也可以采用分布式方式进行拍卖，拍卖过程可能分为多轮，并可以针对一个或多个任务进行处理。

在分布式场景中，具代表性的方法是合同网协议（Contract Net Protocol, CNP）^[37,52,53]，发包者与承包者之间的招投标过程实现任务分配，构成了许多任务分配算法的基础。CNP 作为一种标准化的任务分配平台，高度依赖智能体之间的信息通信，能够将任务分配给最合适的智能体或在必要时进行重分配，信息传递的计算成本较高，整体通信效率和系统性能^[54-56]较低。

随后分布式拍卖类算法逐渐涌现，如基于共识的捆绑算法（Consensus-Based Bundle Algorithm, CBBA）^[38,57,58]以及多轮拍卖方法^[59]等。这些方法广泛应用于无人机协同和多机器人任务分配问题^[60]中。CBBA 作为一种分散算法，能够为多目标优化问题提供解决方案，基本流程如图 1-5 所示。在部分应用场景中，其成本函数反映的是各智能体对捆绑任务的局部效用，难以充分体现全局情境。PI-MaxAss^[61]对性能影响（Performance Impact, PI）^[62]算法进行改进，在一定程度上解决了部分问题。此后，CCBBA^[63]、G-CBBA^[64,65]等方法在 CBBA 的基础上进行了进一步改进。FMCTA^[66]通过本地竞价与局部最优选择逐步逼近全局可行解，通信开销较低且实现简单。分布式拍卖方法需要设计合理的竞价策略和拍卖轮次，否则容易陷入局部次优状态，在大规模系统或高频动态环境中，其收敛效率和稳定性^[67,68]仍有待提升。

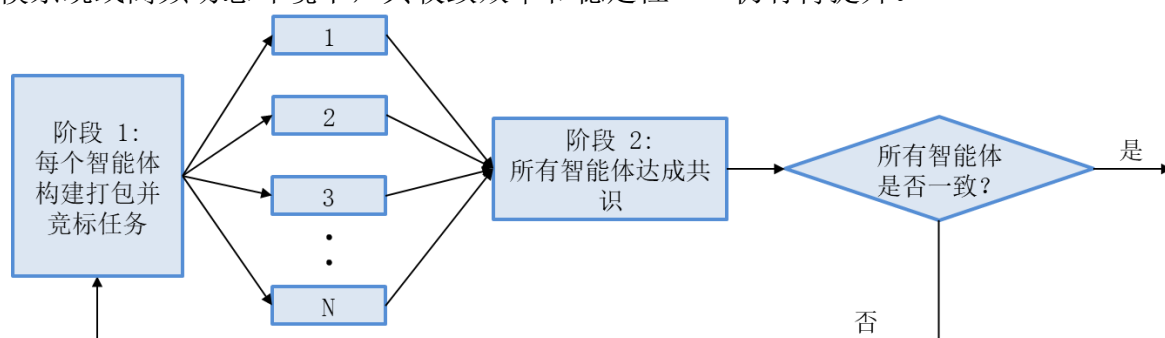


图 1-5 基于共识的捆绑算法

除拍卖方法外，市场机制^[69-71]也是常用的分布式任务分配方法之一。市场机制借鉴经济学中价格调节与供需平衡的思想，通过虚拟货币或虚拟价格引导智能体在资源分配或任务选择方面趋向社会福利最优。市场与博弈方法的典型优势在于分布式自治

和灵活可扩展，因为不是完全集中的算法，它们对动态任务分配很好，可以从拍卖程序中增加或删除新任务，在实际应用中需要精心设计激励机制、通信协议和收敛条件。

1.3.3 基于博弈论方法的研究现状

博弈论为多智能体协作和博弈建模提供了基础理论工具，其基本思想是各智能体在获取外部信息和自身信息后做出使自身利益最大化的决策。研究者将博弈论应用于资源分配^[72]和分布式任务规划中，通过纳什均衡^[73,74]、Shapley 值^[75]等概念分析智能体之间的收益分配与协作稳定性。实际系统往往存在信息不完全、智能体间利益差异和多目标冲突等复杂性问题，在高维连续状态空间下构建博弈论模型具有较大挑战。而在博弈论架构下智能体通常只为自身利益进行决策，未能充分体现协作性。因此如何在博弈论基础上设计协作策略，平衡智能体的个体利益与系统总效用，是解决基于博弈论的任务规划问题^[76-81]的关键。解决基于博弈论架构的任务规划问题的核心在于建立恰当的效用函数及采用适当的协商策略。考虑到集群系统普遍存在着通信带宽和其他通信条件限制，在设计效用函数时需要尽可能地保证智能体间传递的信息量较小。

拍卖与博弈论的方法用于分布式任务规划时各有优势，这些方法有分布式自治以及灵活可扩展特性，也可在一定程度上减轻系统通信负担。不过要在实际应用中实现，还要精心设计激励机制、通信协议以及收敛条件。现有的效用函数有范围约束效用函数^[72]、均等共享效用函数^[82]、最优效用函数^[83]等。协商策略包含基于行为的虚构对策、基于效用的虚构对策、后悔匹配、空间自适应对策。这些策略虽能保证收敛到一个纯策略纳什均衡^[84]，但求解结果大多时候与最优相差较大，所以需要协商策略进行进一步研究。面对长期动态任务、连续到来的新任务以及随机出现的紧急任务，现有方法往往着重于静态或单阶段问题求解，缺少在线更新和自适应调整机制。空间自适应对策虽可使问题收敛到近似最优解，但其运算周期比较长。因此，构建一个能在多时间步范围内最大化全局目标函数的分布式、在线更新任务重分配框架，成为当前研究急需突破的关键方向。

1.3.4 多智能体机器学习方法的研究现状

智能体必须面对难以预测的未来干扰，尤其在明确数学模型描述环境行为的情况下，这种干扰具有动态性。一个解决方案是让智能体通过学习适应干扰，利用过去的行动经验和其他智能体的行为信息提高系统效率^[85-87]。

强化学习（Reinforcement Learning, RL）^[88]是典型的机器学习技术，基本框架是基于“状态-动作-回报”的模型。智能体通过与环境的交互，利用获得的经验不断学习如何在不同状态下采取最佳行动，实现最优或近似最优的决策策略。环境大多时候被构建为马尔科夫决策过程，智能体借助对成本或者奖励函数的优化展开学习。在单智能体问题方面，像 Q-learning^[89]、SARSA 以及 Actor-Critic 等方法已经得到了广泛的验证，其中 Q-learning 作为一种不依赖模型的 RL 方法，可帮智能体在 MDP 里寻找到最优的解决方案。把这些方法延伸至多智能体场景，也就是多智能体强化学习（Multi-Agent Reinforcement Learning, MARL）^[90-92]，可以在一定程度上达成自治以及自适应的任务分配。不过 MARL 在分布式任务分配中的运用面临着由于智能体交互而产生的高维状态和动作空间方面的问题，以及可能出现的非平稳性问题^[93,94]。

强化学习有处理环境不确定性、在训练充分的网络状况下可实时实施以及处理不同任务的优点。在大规模复杂系统当中，大多数 RL 算法都需要较高的计算能力^[95]。近些年来，随着深度学习的发展，深度 MARL^[96]在对抗和协作场景里取得了一定的进展。这类方法借助深度神经网络对复杂的高维状态进行特征提取，降低了对精确环境模型的依赖，为异构系统以及动态场景提供了潜在的可行性。比如在多机器人分拣以及无人机巡逻等任务分配问题上，已经有基于多智能体的 DDPG^[97]、MADDPG^[98]、QMIX^[99]等深度强化学习架构的相关研究。然而这些深度 MARL 算法在收敛性、训练开销、可扩展性以及安全性方面仍然面临着挑战。在大规模多智能体系统中，其网络训练成本以及通信复杂度呈现出指数增长的态势。为此分层强化学习^[100]和分布式网络结构^[101]作为改进的方向被提出，但是要广泛应用还需要更多的理论和工程方面的验证。

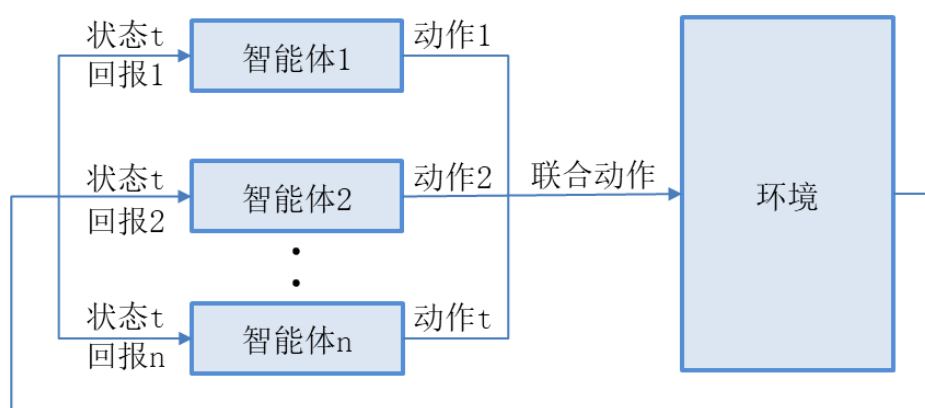


图 1-6 多智能体强化学习系统

为提升多智能体强化学习在动态任务分配中的应用效果，部分学者引入了迁移学习^[102]和元学习^[103]的思想，希望通过在相似任务或场景中累积经验加速新环境下的学

习过程。这种方法对频繁出现新任务类型或环境剧烈变化的分布式系统具有很大吸引力，目前相关成果多为概念性验证或小规模实验，距离真正的大规模应用仍有差距。

1.3.5 优化算法研究现状

任务规划问题实际上就是一个组合优化问题，而优化算法是解决这类问题的自然手段，优化技术可分成确定性方法和随机方法^[104]，确定性方法在求解过程中不考虑随机因素，也就是从相同起点出发，得到的解的路径始终保持一致，常见的确定性方法包含图形方法、顺序编程、线性编程以及混合整数线性编程等。其中匈牙利算法^[105-108]作为一种基础优化方法^[109]，被广泛应用于开发新型任务分配算法。

元启发式随机优化方法是指在计算过程里包含随机性的方法，在处理多目标的时候，需要设计特定的适应度函数和帕累托优化策略，研究难度相对较大。元启发式算法^[110]包括进化算法^[111,112]、群体智能^[113-115]、模拟退火^[116,117]等，希望能够在较短时间内找到快速且高质量的近似解决方案，针对那些确定性方法因计算成本过高而难以承受的优化问题提供了一种可行的替代方案。进化算法如遗传算法^[43,118,119]借助群体搜索以并行方式获取候选解，天然适合映射到多智能体系统的分布式结构上。例如把智能体当作一个演化群体中的个体，各自进行局部搜索与适应度评估，再依靠适当的合作、竞争或进化操作逼近最优解^[120,121]。这类方法大多时候需要大量迭代才能收敛，在动态环境中如何快速更新和达到收敛仍是一个大挑战。蚁群优化^[45,122,123]和蜂群智能^[124-126]作为受生物行为启发的方法，已广泛用于多智能体系统的任务分配，其灵感来源于昆虫群、鱼群和鸟群等有高度社会行为的动物群体。这些动物借助分工协作实现极高的整体效率^[127,128]，即便个体智能体较为简单，整体协作仍能产生强大、高效且低成本的解决方案^[129,130]。不过这些算法有时可能会给智能体分配不必要的任务，引发冲突，并且对环境变化的整体响应比较缓慢。

基于确定性和随机性方法的优化技术各有长短，确定性方法在理论上可提供全局最优或近似全局最优解，但其计算成本在大规模问题中会急剧上升，而随机方法和元启发式算法则提供了高效、近似的解决方案，适用于动态、多目标场景，但在收敛性和快速更新方面仍存在挑战。以上方法为多智能体任务分配问题提供了多样的技术路径，也为构建分布式、自适应任务规划系统奠定了理论基础。

1.3.6 研究现状分析

多智能体系统在无人机编队协作、仓储机器人调度、智能车联网以及自动化生产线等诸多场景中都有广泛应用，分布式任务规划与分配作为该系统的核心研究领域，对于系统在复杂且不确定环境下的协同效率以及整体性能起着决定性作用。当下的研究主要聚焦于怎样把待执行任务合理地分配给多个异质或者同质的智能体，并且已经形成了不少有代表性的理论和方法。然而从实际应用需求以及大规模部署的角度而言，在动态角色分配、在线任务重分配，以及负载均衡与效率目标优化这三个关键方向上，依然存在较为十分突出的不足与空白，急需深入剖析并加以完善。

（一）多智能体任务分解与动态角色发现的不足

在动态环境里，多智能体协同工作时面临的首要问题便是任务分解以及角色分工。传统方法一般依赖于预先定义好的角色类型，像“指挥代理”“执行代理”“侦察代理”等，这些角色依据功能和资源差异事先固定下来，一旦设定，就很难依据环境和任务的变化进行动态调整。一些文献中虽然考虑了在博弈论或者拍卖机制下对角色进行短期切换，但更多的是一种有监督的分配过程，并没有实现角色的自适应发现和演化。当环境在时空维度发生剧烈变化，比如新任务类型频繁出现、子任务耦合度提高、资源约束失衡等情况时，系统大多时候无法有效地更新角色结构，角色分配模式也随之失效。同时，多智能体任务一般有层次化、耦合性高的特点，可能需要先把大任务拆解成若干子任务，然后按照子任务类型将它们映射到合适的角色上。怎样在任务执行过程中动态发现适合当前环境与任务特性的角色，并依靠自适应策略对智能体进行合理分配，成为提升系统协同效率和灵活性的关键课题。国内外有学者尝试将多智能体强化学习与角色发现结合起来，期望在执行过程中依据收益与贡献度在线学习新的角色类型。不过在复杂环境下，这种方法的训练成本与收敛速度依旧是妨碍算法应用的主要瓶颈，而且缺乏足够的算法稳健性分析。许多关于角色发现和分配的实验验证大多停留在中小规模场景，或者假定智能体之间功能相对同质，对于实际生产中可能出现的大规模、多属性差异化智能体体系，动态角色机制的可扩展性、计算负担以及通信开销缺乏深入的分析与验证。在真实或仿真平台上，如果角色变动频率过高，系统能否快速收敛到稳定状态也尚无定论。动态角色发现与分配是目前多智能体协同理论中的薄弱环节，难以契合高动态、强耦合环境对角色灵活性与适应度的要求。

（二）多智能体系统中的动态任务重分配的不足

在有高度动态性的场景当中，仅仅依靠初始阶段的一次性分配，很难始终维持在最优状态，需要在执行的过程里开展在线任务重分配。有一些研究团队提出了分布式拍卖或者滚动时域优化的方式来达成频繁更新，然而在大规模以及多约束的场合下会

面临计算和通信开销过大的难题。当任务变化速度过快或者智能体数量众多时,怎样在可接受的时间范围内求解出高质量的重分配方案并且保证系统整体的协调性,依旧是工程领域需要解决的问题。许多分布式任务重分配算法过度依赖特定的场景假设或者针对性策略,缺少可迁移到其他异构系统或者更大规模场景下的通用框架。与此同时,尚未形成对于在线重分配方法在不同负载水平、通信网络延迟或者故障率条件下的整体稳定性评估,国内外的研究大多集中在单一维度,还缺少对复杂耦合资源情况下在线重分配体系的系统化分析。在现实应用中,任务数量与类型会动态变化,智能体的状态也在持续演变。如何在多不确定性叠加的情形下实现鲁棒的重分配策略,目前仍然缺乏成熟的理论与实践基础。常见的做法是做出较强的简化假设,或者仅仅利用启发式方法快速给出近似解。一旦环境变化超出预设范围,算法性能就会急剧下降,这对多智能体系统在持续运行时的适应能力造成限制。

(三) 基于均衡性的任务分配研究不足

多智能体任务分配的研究大多以提高整体任务完成率或者收益为主要目标,很少关注智能体之间的负载差异、资源消耗以及均衡性问题。在长期运行或者需要智能体保持持续稳定输出的应用里,如果个别智能体负载过重,会影响这些智能体的执行寿命以及维护成本,还可能在后期因为过载致使系统整体性能下降。怎样在全局效率与局部平衡之间找到合理的折衷办法,是现有研究还没有充分解决的问题。均衡性大多时候凭借多目标优化或者博弈论概念引入,不过具体怎样量化和测度还存在争议。部分文献使用 Gini 系数或者均衡度量进行简单衡量,然而实现真正的均衡分配需要更细化的机制设计、激励或者惩罚机制。现有研究中关于均衡性的理论探索局限于小规模试验系统,缺少对大规模、多类型负载指标的验证。在动态任务不断到来、异构智能体共存的复杂环境下,怎样兼顾均衡性并且保持高效分配依旧是较大的技术难点。面向多智能体系统的持续部署场景,仅仅依靠瞬时完成率无法衡量真实的运行效果,还需要评估智能体资源消耗的可持续性、协作意愿以及长期稳定性。现有的诸多方法主要是针对短期性能来开展指标评估工作,然而却欠缺对持续多阶段运行进程里负载均衡方案的演变情况以及智能体健康程度的剖析。如此一来算法很难保证在长期视角下始终维持合作与均衡的最优状态,同时也没办法充分评估策略在“大场景、长周期”情形下的适用性。

上述提及的三个关键方向存在的不足之处,明显反映出当下多智能体分布式任务规划在应对复杂动态环境、异质性以及多目标约束等诸多共同挑战时所面临的状况。随着无人机集群、机器人调度、智能交通与能源网络等应用的不断深入发展,仅仅依

靠传统的集中式或者一次性分配框架，已然无法契合实际需求。国内外研究在多智能体任务规划的上述方向已经展开了大量有价值的探索，不过在算法规模化、通用方法构建以及针对不确定性和多目标约束的系统化分析方面依旧存在欠缺之处。这些不足为后续工作留出了重要创新空间，也为更深层次的动态协同优化理论研究与实际场景落地应用提供了明确的努力方向。

1.4 主要研究内容与主要贡献

多智能体系统属于当前人工智能与分布式计算研究里的关键领域，在近些年的理论研究跟工业应用当中均呈现出蓬勃的发展态势，于协作任务分解、动态任务重分配以及任务均衡分配方面有着关键应用。然而随着应用需求持续扩大，现有的方法在动态环境适应性、任务分配负载均衡以及任务完成效率等方面，依旧存在着十分突出的不足之处。本文重点围绕多智能体分布式任务规划中的一些关键问题展开系统研究，针对这些问题提出三项主要的创新点，并且围绕这些创新点展开深入剖析，构建了多智能体分布式任务分配的综合框架。

1.4.1 研究内容

结合前文对研究背景与意义的阐述以及国内外研究现状分析，本文的研究将聚焦于多智能体任务分解中的动态角色发现与分配、多智能体系统中的动态任务最大化重分配、基于均衡性的多智能体分布式任务分配方法这三个方面的问题。基于这三个核心方向，论文力图构建起一个面向复杂动态环境、能兼顾效率与均衡，并具备通用扩展性的分布式任务规划技术体系。具体研究内容可概括为如下几个层次：

（一）多智能体角色分解与动态角色适配机制

针对现有多智能体角色分配方法多采用预定义角色或静态映射，导致在任务类型、执行状态或环境条件发生显著变化时，无法及时自适应地进行角色切换的问题。设计一种动态角色发现与分配机制，支持依据角色特性、任务需求以及历史贡献来进行智能体角色动态调整，提升任务分解与分配效率。引入基于动作特征的编码器，借助聚类算法对动作贡献进行量化，构建动态角色模型，在角色选择中引入奖励地平线机制，优化角色切换频率，平衡角色间的学习共享与多样性，构建一个基于部分可观测马尔可夫决策过程的角色分解框架，让角色的选择与策略学习在限制的动作空间中完成。

（二）面向动态任务最大化的任务重分配机制

针对现有任务分配方法大多以静态环境为前提，忽视了在实际动态环境中智能体对新任务的感知和适配能力，任务的增加、修改或删除对初始分配方案构成巨大挑战的问题。开发一种基于子团队的任务重分配机制，解决动态任务插入、删除和状态变更问题，最大化任务分配数量。提出新的任务成本计算机制，在任务分配过程中实现灵活的成本调整。设计任务驱动的重分配算法，借助子团队结构和任务释放机制动态响应任务列表的变化，在分布式框架中利用子团队缩小任务响应范围，降低通信与计算开销，凭借数值模拟与实验测试，评估该机制在任务完成率、计算效率及动态响应能力方面的改进效果。

（三）基于均衡性的多智能体分布式任务分配方法

针对多智能体系统中任务分配的不均衡常导致部分智能体过载，另一些智能体资源浪费，最终影响系统的长期稳定性与可靠性的问题。构建了一个异构多智能体系统的任务分配模型，综合考虑智能体的性能差异、任务复杂度、能量消耗以及任务执行顺序等多方面因素。在此基础上，提出了一种均衡性变化指数，以此来量化智能体之间的负载不均衡程度，并且结合响应时间优化构建了双目标优化模型。之后将任务分配问题转化为非合作博弈形式，提出了一种基于纳什均衡的分布式负载均衡任务分配方法。借助这种方法，智能体在进行自利性策略选择时，可实现系统整体任务分配的均衡。为了提升算法的适应性以及计算效率，还设计了一种动态调整机制，依靠该机制，任务分配可依据智能体状态的变化实现实时优化，提高系统的稳定性与鲁棒性。经过实验验证，该方法在均衡性、响应时间以及负载分配稳定性等方面都比现有方法表现更优。

1.4.2 研究贡献

在基于上述研究内容的基础上，本文旨在为多智能体分布式任务规划提供更具通用性和实用价值的理论与方法体系。论文的主要贡献可概括为以下几个方面：

（一）、提出了面向动态环境的角色发现与分配框架

过去在多智能体任务规划里，针对角色要素大多时候采用静态预定义或者固定划分的模式，这种方式很难适应任务以及环境出现的剧烈波动情况。本研究设计了一种依靠动作编码器与奖励机制的角色动态发现办法，达成了角色特征的自适应建模以及实时分配。此方法可动态改变智能体在不同任务中的角色身份，全面发挥智能体异构

资源的协作优势。借助对多智能体和任务特征的动态学习以及特征聚合，形成可随时间演化更新的角色集，并且利用分布式策略达成灵活切换，引入部分可观测马尔可夫决策过程，提高了角色分配的稳定性以及任务匹配效率。借助动作编码器与奖励机制，实现了动态角色的自适应发现以及高效分配。在多个模拟场景中，论文提出的角色自适应算法在任务完成率以及资源利用率方面都比静态角色分配方法更具优势。该方法在动态环境里提升了智能体的任务分解能力以及协作效率，为复杂任务场景中的多智能体角色管理提供了新的思路。

（二）、设计了可扩展的动态任务最大化重分配机制

在持续有新任务流入、环境以及智能体状态高度变化的情形下，本文设计了一种高效动态任务重分配机制。此机制解决了动态任务插入、任务删除以及智能体状态变化给原有分配方案带来的影响，依靠把分布式拍卖算法和强化学习相结合，提出了增量式任务重分配策略，能在少量增加通信与计算开销的情况下完成实时优化。还提出了一种基于子团队划分的任务分配机制，巧妙地把任务分解成可局部协同的子任务并分配给智能体子团队。该方法借助局部信息共享和小范围优化，减少了全局分配里的通信延迟与复杂度，在大规模任务协作中呈现出不错的适用性。实验显示，该框架提高了分布式系统的响应速度，同时降低了整体计算负担，呈现出优异的任务完成率和分配效率，在任务频繁变化和资源受限的环境中表现突出。

（三）、构建了融合均衡性的分布式任务分配算法

多智能体系统存在资源利用率与负载分配不均的情况，本文提出一种基于均衡性的分布式任务分配办法，构建均衡性变化指数来量化智能体间的负载不均衡程度，结合响应时间优化建立双目标优化模型，实现效率与负载均衡的平衡。采用非合作博弈对任务分配问题进行建模，并基于纳什均衡设计分布式优化算法，让智能体在自主决策时维持整体任务负载均衡。提出的动态调整机制能适应智能体状态以及任务需求的动态变化，提高系统的适应性以及鲁棒性。实验验证说明该方法在均衡性、全局响应时间以及负载分配稳定性这些方面均优于现有方法，有效提升了多智能体系统的整体性能。

本论文在多智能体分布式任务规划领域内，围绕动态角色发现与分配、动态任务重分配以及负载均衡与效率这三个创新点展开系统的理论研究以及算法设计，结合仿真平台的多场景测试评估所提方法的可行性和优越性。研究结果表明，本文工作为多智能体系统在复杂且不确定的实际应用环境中实现高效、灵活以及长期稳定的协作提供了新的思路和实践参考。

1.5 论文组织结构

本文聚焦于多智能体分布式任务分配的动态角色发现、任务重分配以及均衡性优化展开深入剖析，全文总共包含七章内容，其具体的组织结构呈现于图 1-5 之中，论文各主要章节的安排如下：

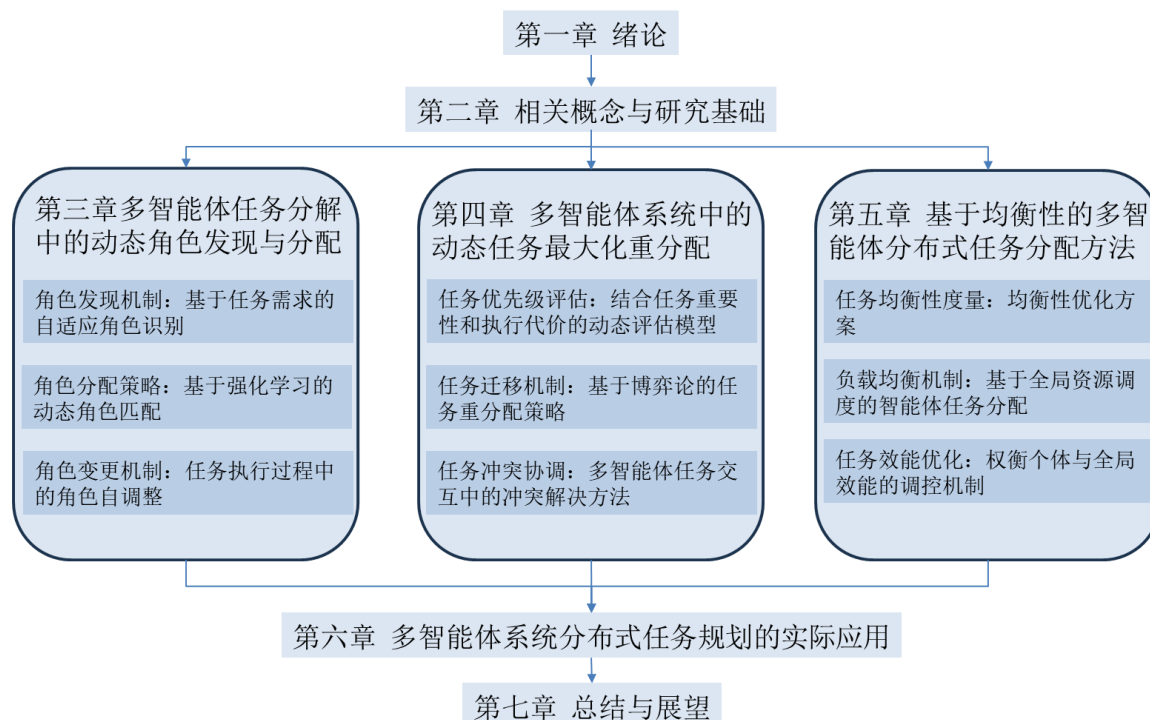


图 1-5 论文组织结构

第一章，着重对多智能体分布式任务分配的研究背景与意义给予概述，对当前国内外的研究现状以及存在的主要问题展开分析，明确本文的研究目标、创新点以及主要的研究内容与贡献。介绍了论文的整体组织结构，为后续章节构建起逻辑框架。

第二章，系统地梳理了多智能体系统任务分配的相关理论和方法，覆盖分布式优化、博弈论、多智能体强化学习以及多目标优化等核心技术，结合当下任务分配在动态角色管理、任务重分配以及负载均衡优化方面的研究现状，剖析了各类方法的适用场景及其局限性，为后续研究提供理论支持。

第三章，鉴于传统角色分配方法存在不足，故而提出一种借助动作编码器以及奖励机制的动态角色发现与分配办法，搭建角色特征建模框架，以此达成任务需求和智能体能力的高效映射，设计部分可观测马尔可夫决策过程来优化角色分配策略，提升角色适配灵活性以及系统任务完成率。借助仿真验证动态角色分配机制在异构任务环境里的适用性与有效性。

第四章，对动态环境下任务分配方案的实时调整问题展开研究，提出一种把增量式分布式拍卖和强化学习相结合的任务重分配机制，包含任务成本动态调整方法，构建兼顾收益最大化以及资源约束的分配模型，以及子团队划分策略，用于降低动态任务插入对系统全局通信与计算产生的影响。在多种动态环境场景中验证该方法的任务完成率与响应效率，分析其优越性与鲁棒性。

第五章，针对多智能体系统里资源分配不均以及智能体过载问题，提出一种基于多目标优化的负载均衡与效率平衡任务分配机制，构建包含均衡性指标的多目标分配模型，并且采用博弈论解决冲突任务分配问题。动态调整机制依靠激励与约束方案来平衡个体智能体的任务负载，在异构智能体场景里，对该机制在效率以及均衡性优化方面的综合表现加以验证。

第六章，验证了本文所提出的分布式任务规划方法在实际应用中的有效性，借助多传感器紧耦合融合以及多机器人协同映射，达成高精度定位和鲁棒地图构建，为动态角色发现、任务重分配以及负载均衡提供实时数据支持。

第七章，总结了本文的主要研究内容和研究贡献，探讨了研究过程中存在的局限性，并且对多智能体任务分配未来的研究方向给出展望。

1.6 本章小结

本章作为绪论，首先阐述了研究背景与意义，解释了多智能体系统中任务规划的重要性。接着，对国内外在此领域的研究现状进行了详尽的分析，比较了不同研究的进展与差异。然后，介绍了研究的主要研究内容与主要贡献。最后，介绍了论文的组织架构。

第二章 相关概念与研究基础

2.1 多智能体系统的基本概念与理论基础

多智能体系统的研究起始于 20 世纪 70 年代，一开始是以单一智能体当作研究对象，想要赋予系统有感知、决策以及执行复杂任务的能力。不过随着任务复杂度以及动态化场景的增多，单一智能体的局限性慢慢显现出来。在分布式计算、动态任务分配以及实时协作等领域，单一智能体很难有效处理复杂任务、动态环境以及资源受限这些问题。于是研究者开始转向去探索由多个智能体构成的系统，也就是多智能体系统。多智能体系统的发展和分布式计算以及并行处理紧密相连。20 世纪 80 年代，随着计算机网络的迅速发展，人们意识到单点计算难以契合复杂任务的需求，而多智能体系统借助分布式架构把任务分解成多个子任务，由不同智能体分别进行处理，然后依靠协作达成全局目标。这种分布式设计奠定了多智能体系统的理论基础，并推动其逐步在航空航天、机器人集群和智能交通等领域得到广泛应用。

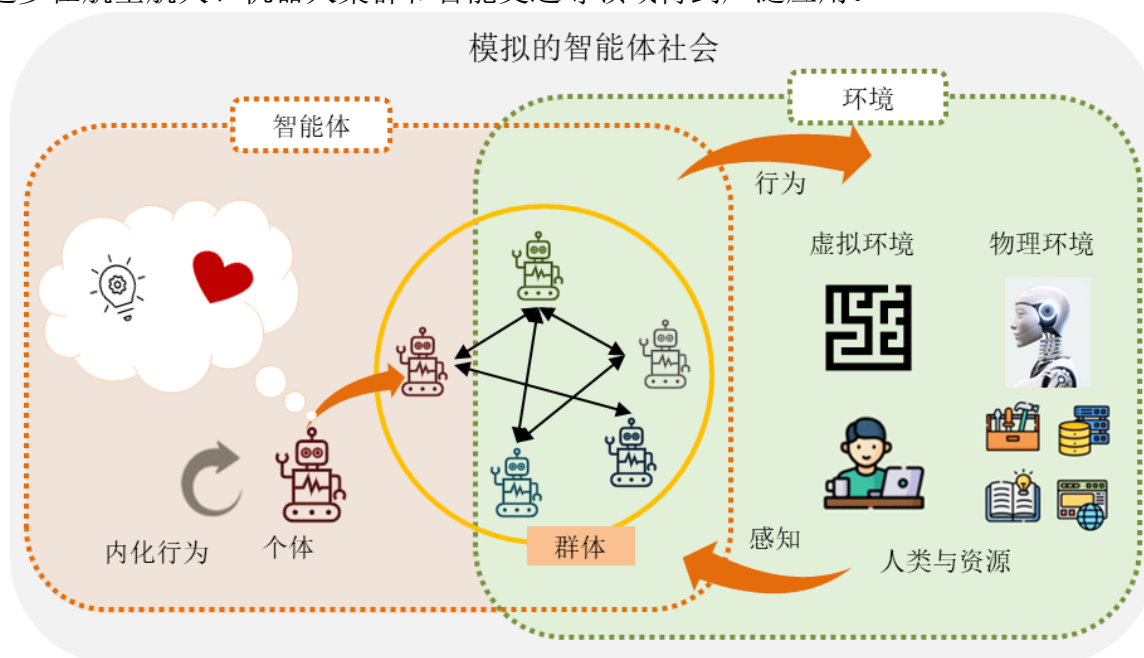


图 2-1 智能体社会的概念框架

多智能体系统属于分布式人工智能的一个关键分支，它是由多个有一定自主性与智能性的个体所构成的系统，这些智能体借助感知环境、相互通信以及协作或者竞争等方式，来达成系统级的目标或者任务，每一个智能体都可被视作是一个拥有感知、

决策以及执行能力的独立单元。相较于单一智能体系统，多智能体系统在应对复杂任务、动态环境以及异构资源的时候，呈现出了独特的优势，受到了广泛的关注。多智能体系统的提出是希望依靠多个智能体的协作、竞争以及分布式决策，去解决单智能体系统难以处理的问题。它的分布式架构以及协作行为可适应复杂动态环境，并且不需要依赖集中式控制。这种分布式特性让多智能体系统在动态任务分配、资源协调以及信息共享方面有优势，不过也带来了系统稳定性以及全局最优性难以保证的挑战。

在多智能体系统中每个智能体的行为受到内部目标以及外部环境这两方面的驱动，该系统含有一个智能体集合 $\mathcal{A} = \{a_1, a_2, \dots, a_N\}$ ，其中每个智能体 a_i 都有自主运行的能力。多智能体系统的运行环境呈现出动态的特征，其状态会随着时间持续发生变化。智能体之间借助通信网络或者共享环境来开展信息交换，形成协作或者竞争的关系。多智能体系统的特点可以概括为以下几个方面。其一为自主性，意思是智能体可依据局部感知信息独立做出决策并执行任务。其二是协同性，智能体借助直接或者间接通信，依据全局目标相互配合来完成复杂任务。其三是分布性，多智能体系统中的任务以及资源是分散的，并非依赖于单一的中心节点。其四是异构性，多智能体系统中的智能体可能拥有不同的功能、能力以及资源，需要依据任务需求进行合理的分工。其五是动态性，多智能体系统一般运行在动态环境中，系统需要可实时适应环境的改变。这些特性决定了多智能体系统有灵活性和适应性，使其成为解决复杂动态问题的关键工具，不过也给予了如任务分解、通信开销以及协作优化等挑战。

表 2-1 三种控制模型的比较

控制模型	控制器	节点功能	系统规模	性能	鲁棒性
集中式控制	由一个中央控制器管理	所有节点在任务分配中是被动的	小	全局最优	差
分布式控制	无控制器	所有节点在任务分配中是自主的	大	局部最优	好
混合控制	部分节点可以充当控制器	节点可在被动和自主角色之间切换	中等	中等	中等

多智能体系统的控制架构对其任务分配以及协作机制的实现途径有着直接的影响，一般情况下可分为集中式控制、分布式控制和混合式控制这三种形式。集中式控制是依靠单一中央节点来开展任务分配以及资源调度的一种控制方式，中央节点借助全局信息实施规划，随后把结果分配给各个智能体去执行。这种架构的好处是在系统规模较小的时候可达成全局最优，不过它的扩展性欠佳，并且对中心节点的依赖性较强，一旦中心节点出现失效的情况，就会致使系统崩溃，存在单点故障的风险。在集中式控制架构里，任务分配是由一个中心节点来完成的，所有智能体只要依照分配的指令

执行任务就行。集中式控制比较适用于小规模、静态的场景，与之相对的是，分布式控制着重于智能体之间的自主性，每个智能体依据局部信息独立做出决策，并且借助邻域通信来进行协作。这种架构拥有良好的扩展性和鲁棒性，特别适合大规模动态系统，但因为缺少全局信息，分布式控制或许无法实现全局最优。分布式控制被广泛运用在无人机编队、车联网调度等需要实时响应的场景中。混合式控制融合了集中式和分布式控制的优点，中央节点负责全局规划，而智能体在局部完成自主优化，适用于大规模、复杂环境下的任务分配与协作。这种架构保留了集中式的全局优化能力，又有分布式的灵活性和鲁棒性。混合式控制广泛应用于机器人协作系统，像仓储物流中多机器人分拣任务的规划与执行。

多智能体系统的理论发展得益于多个学科领域的支持，这些支持为多智能体系统的设计与优化打造出了坚实的理论根基。人工智能为智能体的能力设计奠定了基础，像感知、规划以及学习等功能，让智能体可凭借自身进行决策，适应复杂多变的环境。控制论为多智能体系统提供了动态行为建模以及稳定性分析方面的理论支持，借助反馈控制理论来设计智能体的自适应行为，以此保障系统在动态环境里的稳定性，还可以实现任务分配和资源调度的动态调整。博弈论构建起了智能体之间交互以及收益分配的分析框架。其中合作博弈研究的是智能体怎样凭借协作让系统效益达到最大化，而非合作博弈主要关注竞争环境下的策略优化，很好地刻画了多智能体系统中的协作与竞争关系。分布式计算为多智能体系统的任务分配、通信优化以及并行处理提供了解决办法，依靠局部信息不断迭代去逼近全局问题的最优解，为大规模多智能体系统的协作与调度提供了高效的算法支持。多智能体系统基于相关理论，依靠智能体之间的协作克服单一智能体解决复杂任务时的局限，达成系统级目标的同时提升整体系统的灵活性与效率。

2.2 分布式任务规划理论与建模

分布式任务规划属于多智能体系统领域里关键的研究范畴，是可达成多个智能体于分散化网络环境里对复杂任务给予分配、执行以及协同的系统性办法，其流程如图2-2所示。和传统的集中式任务规划方式相比较而言，分布式任务规划更注重每个智能体的自主决策能力、对局部信息的运用以及对动态变化与不确定性的实时反应。随着计算机网络、机器人、无人机以及物联网等技术持续发展，分布式任务规划理论逐

渐成为支持多域、多平台、多场景协同运行的关键支撑技术。

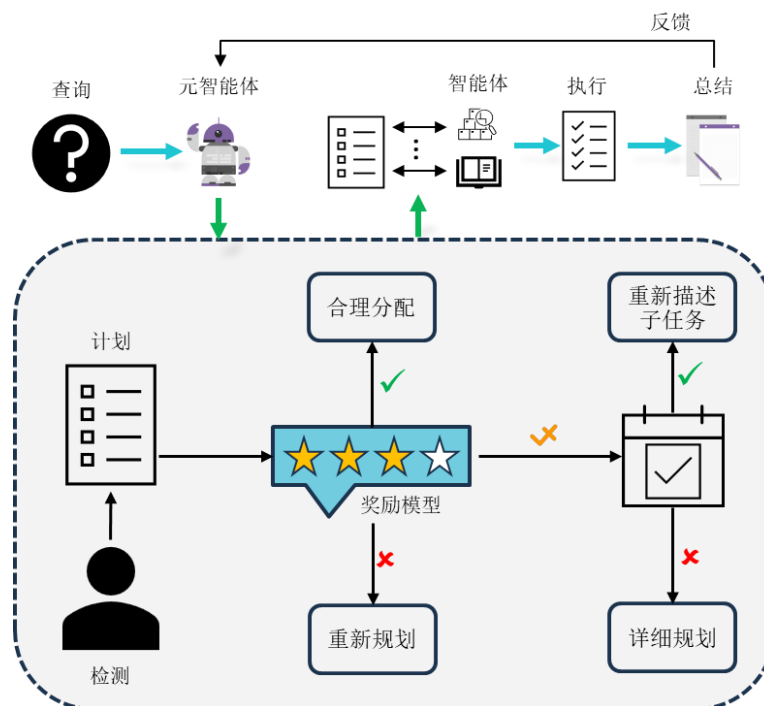


图 2-2 多智能体任务分配、执行和反馈流程

分布式任务规划理论的形成以及发展和多个学科领域有着十分紧密的交融。博弈论为多个利益体之间的行为策略分析给予了可行的理论框架，自利型智能体之间怎样达成协作或者竞争这一方面，博弈论可量化地描述策略演化、均衡解以及激励机制。优化理论和分布式算法比如分布式梯度下降、交替方向乘子法等为分布式环境里求解大规模约束优化问题奠定了方法学基础。控制理论中的一致性方法为多智能体之间依靠局部通信达成全局共识提供了数学工具，广泛应用于分布式状态估计、分布式决策以及网络化控制等领域。此外，人工智能与机器学习，尤其是多智能体强化学习已经成为分布式任务规划的关键补充。MARL 在不完全信息以及环境动态的前提下，借助分布式的试错学习和策略更新，让智能体可自适应地进行协同行动。各个学科在分布式任务规划里都有不同程度的渗透，它们共同构成了分布式任务规划的理论支柱。

在开展分布式任务规划研究工作之前，要先对系统进行精确建模，覆盖智能体的状态、任务类型、执行成本、奖励或效用函数、通信拓扑以及各类约束条件等方面。在多目标场景当中，要依据具体的应用需求来设计多目标目标函数，比如实现效率最大化、能量最小化、负载平衡等，并且探讨不同目标之间的冲突以及权衡情况。为保证分布式算法可有效执行，需在系统层面设计合理的通信协议，以便智能体可以按照需求交换任务、资源或者状态信息。协议可限定通信频率、数据内容格式以及安全机制，同时还应考虑网络带宽、拓扑变化以及延迟问题给系统整体性能带来的影响。在

存在恶意节点或者不可靠节点的情形下，还需要思考如何借助共识机制或者冗余策略来维持系统的正常运行状态。分布式任务规划大多时候经由多轮迭代或者多阶段学习收敛到某种均衡解或者局部最优解，要对算法的收敛过程展开理论分析，明确其收敛速度、收敛条件以及可能出现的局部极值陷阱或者震荡现象。对稳定性的研究还包括在面对扰动或者局部失效时，系统能否自动恢复或者重构任务规划方案。当多智能体系统中的每个个体都有自利性或者拥有各自不同的目标时，合理设计激励机制可让个体在局部决策时兼顾系统整体效益。当多个任务或者多个资源并行分配时也要留意负载均衡，防止部分智能体长期处于不利位置。在涉及商业机密或者敏感信息的多智能体系统里，需要把隐私保护纳入任务规划流程。另外还要部署必要的安全手段来防止网络攻击或者数据篡改，以此维持分布式任务规划算法的鲁棒性与可信性。

任务分配问题通常被定义为满足资源和能力的约束的同时在多个智能体之间分配任务以实现最大化全局收益。假设系统包含 N 个智能体和 M 个任务，智能体集合为 $\mathcal{A} = \{a_1, a_2, \dots, a_N\}$ ，任务集合为 $\mathcal{T} = \{t_1, t_2, \dots, t_M\}$ ，其任务分配问题可以被建模为一个优化问题。每个任务 t_j 的状态由以下向量表示：

$$\mathbf{s}_j = [d_j, r_j, p_j], \quad (2.1)$$

其中 d_j 是任务的截止时间， r_j 是任务所需资源量， p_j 是任务的优先级。智能体 a_i 的能力向量表示为：

$$\mathbf{c}_i = [\text{capacity}_i, \text{speed}_i, \text{cost}_i], \quad (2.2)$$

分别对应资源上限、执行速度和执行成本。

多智能体系统的分布式特性要求智能体在分配任务时进行优化。任务分配问题可以被形式化为一个最大化全局收益的问题：

$$\max \sum_{i=1}^N \sum_{j=1}^M R_{ij} x_{ij}, \quad (2.3)$$

其中， $x_{ij} \in \{0,1\}$ 表示任务 t_j 是否被分配给智能体 a_i ， R_{ij} 表示智能体 a_i 完成任务 t_j 所产生的收益。收益 R_{ij} 可以定义为：

$$R_{ij} = w_1 p_j - w_2 \frac{r_j}{\text{capacity}_i} - w_3 \text{cost}_i \quad (2.4)$$

其中， w_1, w_2, w_3 是权重参数，用于平衡任务优先级、资源需求与执行成本。

为了满足分布式系统的限制，任务分配问题通常需满足以下约束条件：1.任务唯一分配：每个任务最多被分配给一个智能体， $\sum_{i=1}^N x_{ij} \leq 1, \forall j \in \{1, \dots, M\}$ 。2.智能体资源限制：智能体的资源需求不能超过其能力， $\sum_{j=1}^M r_j x_{ij} \leq \text{capacity}_i, \forall i \in \{1, \dots, N\}$ 。

通信以及行为协调乃是达成多智能体协同的关键所在，通信的作用是于智能体之

间分享任务的状态以及环境方面的信息，而行为协调指的是智能体依据所共享的信息来调整自身的行为，达成目标的一致性。多智能体系统大多时候借助通信图 $G = (\mathcal{A}, \mathcal{E})$ 去呈现智能体彼此之间的信息交互，其中节点集合 $\mathcal{A}$ 代表智能体，边集合 $\mathcal{E}$ 代表智能体之间的通信连接。通信效率对协同的成效有着直接的作用，过多的通信有可能致使系统开销增多，而通信不足则会对协同质量产生影响，行为协调要保证智能体的行为不会出现冲突。比如在路径规划里，多个智能体或许会同时经过交叉路口，这个时候就得依靠协商机制来避免冲突。

分布式任务分配需要在局部信息的基础之上达成全局的优化。智能体 a_i 的局部优化问题可运用拉格朗日乘子法建模成：

$$\mathcal{L}_i(x_i, \lambda_i) = \sum_{j=1}^M R_{ij}x_{ij} - \lambda_i \left(\sum_{j=1}^M r_j x_{ij} - capacity_i \right), \quad (2.5)$$

其中 λ_i 为智能体 a_i 的拉格朗日乘子，表示资源限制的权重。通过迭代更新 λ_i ，智能体可以逐步实现对任务分配的动态调整。

2.3 博弈论理论基础

博弈论是一门专门研究在存在多个有理性决策能力的参与方，也就是一般所说的“玩家”或者“博弈方”该如何去进行策略选择以及这些选择可能会带来何种结果的科学。它最初是由数学家冯·诺伊曼以及经济学家摩根斯坦在他们于 1944 年出版的那本名为《博弈论与经济行为》的著作中进行了系统化的提出。从那之后，博弈论发展得越来越快，逐渐成为了在经济学、管理科学、计算机科学、政治学、生物学等诸多领域都被广泛运用的一种分析工具。它是借助对参与者之间潜在的合作、竞争、冲突以及联盟等情况展开研究，帮助我们去理解以及预测在不同的环境之下，人或者智能体将会采取怎样的策略，又会导致怎样的结果。博弈论的基本概念囊括了以下一些关键要素：

参与者：在博弈论当中，首先要明确谁是参与博弈的人，也就是“玩家”或者“博弈方”，这些参与者可以是个人，也可以是企业、政府机构，还可是智能体、机器人或者其他拥有一定决策能力的实体。玩家的理性假设是博弈论的核心前提之一，也就是说玩家会依据自己所可获取到的信息以及评估方法，尽可能地让自身的目标或者收益函数实现最大化。

策略：策略指的是玩家在博弈过程中可以执行的一系列决策或者行动规则，在不同类型的博弈当中，策略所呈现出来的形式是不一样的。比如在一次性博弈，也就是静态博弈里，策略可简单地被理解为一行动的选择。而在动态或者重复博弈中，策略则包含了玩家在每个阶段怎样根据对方以往的行动或者当前的状态来做出应对的规则。策略可以是纯策略，也可是混合策略，混合策略允许玩家按照一定的概率分配来选择可用的行动，以此来提高自己在不确定环境下的收益。

收益：收益函数被用于量化玩家在博弈结果里的得失情况，其可以是金钱，也可以是效用，或者是其他可进行度量的指标，像是资源占有率、满意度等之类的。博弈论借助对不同策略组合下玩家收益分配情形的研究，以此来判定怎样的战略选择对于一个理性玩家来说是最优的或者是最具优势的。收益函数在很大程度上对博弈的性质起到了决定作用，同时也是激励机制设计的基础所在。

信息：信息于博弈论里占据着关键的位置，不同的信息假设会致使不同种类的博弈模型出现。一般而言，若所有玩家对环境状态、对方收益函数以及策略空间都有全面的了解，这就被称作完全信息博弈。要是存在部分信息无法被所有玩家知晓，那便是非完全信息博弈。在非对称信息的条件下，还会出现信息不对称博弈，在这种情况下，某些玩家比其他玩家掌握着更多的环境或者对手信息，有可能产生逆向选择和道德风险等问题。

理性与共识假设：博弈论一般会假定玩家拥有理性，即玩家可依据所获取的信息、自身以及对手的收益函数，挑选出最优的或者是满意度较高的策略组合。在某些较为高级的模型当中，或许还会假定玩家有“共同知识”，也就是说所有玩家都清楚他们处于某一博弈框架之内，而且彼此都知晓对方同样是理性的。这类假设对于理论分析的简化有着一定作用，不过在现实应用时需要结合具体的情境来进行修正或者扩展。

2.4 基于博弈的多智能体分布式任务分配理论

博弈论为剖析多主体之间竞争、协作、冲突以及联盟等行为给予了关键的数理工具，成为分布式任务分配的一个核心理论支撑点。基于博弈的多智能体分布式任务分配理论，把多智能体任务分配过程当作是一个由自利或者合作动机推动的博弈，经由设计合理的激励机制、协议或者规则，促使这些智能体在缺少全局监督的情况下，自主地达成最优或者近似最优的任务分配。

在多智能体系统里，任务分配的本质是把一组需要执行的任务分配给若干智能体，让这些任务在整体性能指标上达成最优的完成效果，常见指标包含完成时间最短化、总收益最大化、能量或资源消耗最小化、可行性与鲁棒性提升等。由于多智能体系统大多时候处于动态环境，并且存在带宽限制、拓扑变化、信息不完全等问题，传统的集中式算法在规模与实时性方面很容易遭遇瓶颈。设计分布式、自治、高效的任务分配机制成为关键挑战。

将多智能体的分布式决策行为看成一个博弈，这意味着每个智能体都是有一定理性或自利属性的“玩家”，它依据自身局部信息以及对其他玩家行动的预期做出策略选择。这里“策略”一般指智能体对某些任务的竞标、报价、参与度或者执行顺序等。凭借合适的博弈模型，可刻画智能体在竞争或协作条件下的战略行为，研究系统整体的平衡点以及激励机制设计。

不同应用场景存在各自特定假设，像智能体间是否允许通信、通信是否可靠，智能体对方收益函数或类型是否知晓，甚至智能体目标函数是否相同或存在利益冲突。依据博弈的分布式任务分配理论，要在这些假设基础上，设计并分析一套可行博弈规则与求解过程，一般以便找到某种稳定解，例如纳什均衡或子博弈精炼均衡，且保证在该均衡下总体收益接近全局最优。若智能体倾向合作，还会关注联盟形成及收益分配等问题。

在非合作博弈背景里，多智能体任务分配的一个常见分析目标是寻找纳什均衡。纳什均衡指在该分配也就是策略组合下，没有任何单个智能体能靠单方面变动策略来提高自身收益。然而纳什均衡不一定意味着系统整体最优，甚至可能出现“囚徒困境”式的效率损失。研究者一般关注混乱代价或最优均衡代价等指标，来量化分散决策导致的效率退化程度，依靠合适机制设计，让个体在保证自身收益时，提升全局效益。

当任务分配伴有某种分布式资源交换或成本分摊时，怎样保证智能体报出的“竞价”或“私有信息”是真实的，大多时候是难点所在。若没有合理激励机制或激励兼容设计，智能体可能会借助谎报自身成本、能力或任务收益来获取更多资源或更少负担，损害系统效率。博弈论提供的机制设计框架，能在一定程度上保证个体真诚报价时获得最大收益，鼓励真实信息披露。

在合作博弈的情景之下，多个智能体有可能会组建起联盟，将各自的资源以及能力合并起来，以此去达成单个智能体所无法完成的任务目标。怎样去衡量联盟里每一位参与者的“边际贡献”，并且把总收益以均衡且稳定的方式加以划分，这是联盟博弈研究的关键所在。Shapley值是一种经常会被用到的分配方案，不过在实际应用时

候或许会面临计算复杂度较高或者缺乏即时性这样的问题，需要进行近似求解或者分段式求解。

拍卖式任务分配是把多智能体分布式任务分配问题转化为一系列拍卖博弈来解决的一种有代表性的方法，其基本流程如下：首先是任务发布，系统或者任务管理者依据当前状态，向所有可能执行任务的智能体发布任务描述。其次是竞价，各个智能体依据自身能力、可用资源、成本函数等，对任务进行出价或者报价。最后是决策与分配，拍卖规则会将任务分配给胜出的智能体，并且确定支付价格。在这个过程当中，依靠精心设计拍卖机制以及支付规则，可鼓励智能体在报价的时候提供真实信息。像维克里拍卖就是一种常见的激励相容拍卖机制，它在一定条件下可让玩家的最优报价为真实估值，对于复杂任务或者多任务并行的情景，可以采用组合拍卖或者多轮拍卖机制。

在没有中央管理者或者缺少可信信息中心的情形中可运用非合作博弈的理论去指导多智能体进行独立决策，重点对个体策略以及全局均衡展开分析，其典型流程如下：首先是定义策略空间，也就是每个智能体可选择执行哪些任务，按照怎样的顺序或者在什么时间段去执行，并且存在相应的收益或者代价函数。其次是信息交换，智能体之间可能会有限度地互通任务需求或者执行状态，以此来预测对手的行为。再次是策略收敛，借助周期性迭代或者同步更新，系统会逐渐收敛到某个纳什均衡或者近似均衡。最后是效率分析，对比均衡解与集中式最优解之间的差距，判断博弈机制的好坏，并且有可能引入惩罚或补偿协议来减少效率损失。

当智能体倾向于建立合约或者联盟时合作博弈提供了可行的分析路径。展开来说，联盟内智能体凭借合并自身资源与执行能力获得高于各自独立行动时的整体收益，接着利用联盟博弈概念对整体收益进行均衡且稳定的分配，要是该分配能保证联盟内部没有人有动力单方面退出那么就称之为稳定的联盟结构。联盟博弈一般需要迭代搜索或启发式算法来完成联盟划分与收益分配，在大规模场景下计算复杂度较高。

当任务分配呈现为持续进行或者重复出现的过程时，单个时间点做出的短期决策有可能对后续智能体之间的信任以及合作关系产生影响，借助重复博弈模型，可在策略当中嵌入激励或者惩罚机制：要是某个智能体在当前回合违背合作意愿、索取不合理收益或者虚报信息，那么在后续回合里将会遭到群体“排斥”或者收益被削减。一些经典的触发策略或者分层惩罚策略可有效地维系长期合作的稳定性。

随着多智能体数量迅速增多，博弈模型的维度以及计算复杂度有所提升。对于异构智能体的能力以及目标函数差异，在博弈建模时也需要加以考虑，怎样在可扩展性

与精确求解之间达成平衡，成为关键的研究方向。一些近似算法、元启发式搜索以及分层次的博弈结构有望缓解大规模问题带来的挑战。在许多实际应用情形下，智能体对于对手的收益函数、状态或者任务资源所掌握的信息并不完整，并且环境可能在执行过程中随时发生变化。贝叶斯博弈及信号传递博弈为这类情境提供了理论工具，但是怎样在分布式情境下保证收敛与有效激励，仍然需要深入研究。多智能体强化学习与博弈论相结合，可使智能体在反复交互中不断修正对对手策略的认知以及自身策略。有些场景甚至需要在线训练，这就要求智能体在复杂的博弈环境中实时学习最优分配策略，此时收敛速度、算法稳定性以及策略可解释性都是值得关注的问题。

2.5 本章小结

本章系统地阐述了和多智能体分布式任务分配相关的核心理论以及研究基础，为后续的研究奠定了较为坚实的理论支撑。一开始介绍了多智能体系统的基本概念以及理论基础，其中涉及了它的定义、特点、控制架构以及典型应用场景，这为多智能体任务规划的复杂性和多样性提供了背景方面的支持。接着重点分析了分布式任务规划理论以及建模，探讨了任务分配的数学描述、优化目标以及约束条件，并且阐述了其在动态环境中的适应性与扩展性。随后经过对博弈论理论基础的详细剖析，揭示出博弈模型在多智能体协作与竞争关系里的关键作用。最后深入剖析了基于博弈的多智能体分布式任务分配理论，呈现出博弈论在任务分配机制设计中的应用潜力。

第三章 多智能体任务分解中的动态角色发现与分配

3.1 引言

多智能体系统中如何实现任务的合理分解与分配是实现高效协作的关键。任务分解涉及任务目标的分析、子任务的生成及分配、执行策略的制定等多个环节。基于集中式控制的任务分解方法使用预定义规则或手工设计任务划分策略，在大规模多智能体系统下表现出计算复杂度高、扩展性差和动态变化适应能力弱的问题。

得益于人工智能技术的发展，强化学习和深度学习逐渐成为解决多智能体协作问题的重要工具。目前多智能体的控制策略主要分为集中式学习^[131]、独立式学习^[132]和集中式训练分布式执行^[133,134]（Centralized Training with Decentralized Execution, CTDE）三种范式。集中式学习将系统视为一个整体，采用单智能体强化学习算法训练，在一定程度上解决了环境非平稳问题。由于其需要全局信息且不可扩展的局限，无法解决无通信、大规模和大动作空间的问题；独立式学习让每个智能体独立训练自己的策略，在一些合作任务中取得了不错表现，忽视了多智能体之间的联系，加剧了学习的不平稳性；集中式训练、分布式执行是一种折中的方法，训练时拥有全局信息，提高学习效率，执行时各自独立决策，一定程度上解决了多智能体学习问题^[135-137]，但随着智能体数量增多，最优联合值函数求解会变得很复杂。

在 CTDE^[138,139]框架的支撑之下，多智能体系统得以借助共享全局信息来展开训练，在执行阶段达成高效协作。不过该框架于实际应用里也遭遇了诸多难题，像怎样合理划分角色、分配任务以此缩减联合动作空间的规模，怎样在角色动态变化的状况下维持稳定性，怎样优化长期目标与短期目标之间的权衡等等。QMIX^[140]身为 CTDE 框架里极为典型的方法，是基于单个智能体的非线性组合，得出联合动作的估值。即便如此，QMIX 方法还是存在着一些局限性。为了处理这些问题，QTRAN^[141]放宽了累加性以及单调性的限制，去除了 QMIX 的结构化约束，然而在后续的 SMAC 任务中并未呈现出优良的性能。Qatten^[142]改良了联合 Q 值的分解方法，引入了多头注意力机制，弥补了 QMIX 以及 VDN 等算法在联合 Q 值分解方面缺少理论支撑的欠缺。QPLEX^[143]则是在 QMIX 中增添了决斗结构，以此强化对个体动作价值的估量。

MADDPG^[144]针对连续动作空间中的任务推测其他智能体策略来稳定训练过程。在处理大规模任务时, Mean-Field^[145]将智能体周围的其他智能体视为一个具有平均特性的整体, 简化了智能体间的交互, 由于对智能体细节缺乏关注, 无法实现精细的大规模协作。QMIX-ME^[146]通过结合最大熵策略学习和 QMIX 结构, 解决了多智能体系统中的信用分配问题, 为 CTDE 框架的发展提供了新的方向。

在人类社会, 处理问题或者任务时, 人们往往会将大而复杂的问题自上而下地分解为不同层次的子任务, 不同的人担任某种角色去处理不同的子任务^[147], 相当于每个个体只需要探索扮演角色的受约束状态行动空间。因此, 人们会使用与特定子任务或策略相关联的角色来处理任务。如果拥有先验知识和领域知识, 我们可以通过预先定义任务分解和角色^[148]来制定一组将多智能体高层次复杂任务分解为低层次简单任务的角色。在实践中, 预先定义的角色并不总能满足任务需求, 任务或团队动态变化也会影响预先定义角色的有效性。团队成员需要在合适的时候转换角色来满足任务动态变化后的需求, 这就需要一个可以动态发现并分配角色的方法。

在大的行动空间中进行大量的探索来学习角色的行为特征的动态角色方法和无角色的方法相比并不一定有优势。在多智能体环境当中, 可对任务给予分解, 从较小的任务着手, 逐步达成整体任务的目标。这种方式需要一个可良好分解多智能体团队任务的模型, 这构成了一项新挑战。动态角色发现与分配在多智能体任务分解里发挥着关键作用, 角色的定义是凭借限定智能体的动作空间以及策略范围, 来降低系统的复杂程度, 同时提升学习效率。传统角色分配方法依赖领域专家的先验知识, 借助静态规则或者模板来确定角色, 然而这种基于静态设计的角色分配难以契合动态环境中频繁变动的任务需求。

在多智能体系统中, 角色的定义一般依据行为差异或者功能分工。不过单纯依靠行为差异或许无法全面呈现角色在任务里的实际贡献。怎样综合考量动作属性与贡献来定义角色, 成为动态角色发现的关键研究方向。多智能体系统中的角色选择要平衡智能体之间的能力差异与任务需求, 同时避免频繁的角色切换给系统稳定性带来的影响。角色选择策略要可动态适应环境变化, 同时兼顾智能体的历史行为与当前状态。角色的动态分配需与相应的策略学习相结合, 保证智能体可在其角色范围内高效执行任务。角色策略学习的目标是通过限制动作空间, 提高学习效率以及策略的泛化能力。多智能体系统中的任务分解不仅需要细化任务目标, 还需将子任务合理分配到不同的角色中。如何通过优化任务分解与角色匹配过程, 提升系统整体性能, 是动态角色分配的核心挑战之一。

角色分配问题一般会被构建成一个分布式部分可观测马尔科夫决策过程 (Decentralized Partially Observable Markov Decision Process, Dec-POMDP), 该过程在全局回报函数的限制条件下, 借助角色划分以及选择策略来使系统性能达到最大化。在动态角色发现与分配的研究工作里, 强化学习以及动作表示属于关键技术。强化学习依靠奖赏信号来推动智能体学习最优策略, 这为角色分配以及策略学习给予了理论方面的支撑。动作表示是把离散动作映射到连续向量空间, 以此可更全面地把握动作属性以及差异, 提高角色定义的精确程度。

很多工作受到角色分工的启发, 通过将任务进行分解, 并由相同角色的智能体去处理相同的子任务, 降低总体设计的复杂性^[149-155]。这些系统需要使用先验知识^[156]对角色和相关子任务进行定义^[157,158]。在特定场景下预先定义可以有效解决问题, 在实践中却无法保证每次都能获得先验知识, 其泛用性受到很大限制。目前, 很多工作都是为了研究角色在不同情况下的泛用性。Wilson^[159]使用贝叶斯策略搜索方法建立了无模型的角色发现算法。Wang^[160]设计了一个专业化的目标, 提供了一种灵活的、通用的机制来鼓励角色的出现。这些试图在泛用性方面做出贡献的方法需要在完整的行动空间中进行大量探索, 大部分的探索行为可能是无效的, 效率低下的行为会浪费大量资源, 增加了机制的复杂度。同时, 最近的许多工作使用角色来衡量不同智能体的特征^[161-163], 如何准确和完整地定义智能体的角色仍在探索之中。在^[164]中, 角色被定义为分层 RL 框架中的高层选项。Christianos^[165]将角色定义为随机政策的环境影响相似度。这些定义都集中在智能体行为某些方面的差异, 不能完全衡量它们。

为了解决上述挑战, 本章提出一个基于角色发现与分配的多智能体团队任务分解框架。本章利用一个动作编码器根据动作属性构建向量表征, 并结合动作贡献形成动作表征。利用行动表征结果对行动进行聚类, 最终发现不同角色。每个时间步, 利用轨迹编码网络表示智能体的历史信息, 计算与角色表征的相似性获得角色选择的分类分布, 并学习角色选择策略。对于学习策略, 扮演同一角色的智能体可以分享学习过程, 网络估计行动的效果和贡献, 将行动的 Q 函数值加权到角色的 Q 函数中。同时, 本章在发现角色的学习过程中使用一个正则器来更好地分离角色, 增加角色之间的表征差异; 并在角色选择策略的学习过程中使用一个正则器来避免智能体频繁改变导致的训练不稳定。

当智能体处于一个合作任务时, 本章通过学习行动差异和动作贡献, 将行动聚类来分解联合行动空间, 一个动作空间组成一个角色, 智能体只需要学习满足某种功能的行动子集。例如, 在一场足球比赛里, 球员可以分为守门员, 后卫, 前锋等。每名

球员根据自身扮演的角色，动作的选择也会受到约束，守门员只需要探索防守时如何移动。为了对角色进行更好的区分和选择，本章利用行动差异和行动贡献的知识，将角色划分为长期角色与短期角色，通过奖励视野研究角色差异引起的损失。确定了角色后在角色奖励视野的基础上训练角色选择方法。角色选择模型在角色的受限行动空间中协调角色选择。角色策略也在受限的行动空间中进行探索，并且和行动表征相关联。本章将多智能体的合作问题在空间上分解成几个受限空间的学习问题，降低了学习的复杂性。在 SMAC^[166]环境中对本章方法进行了测试。测试结果显示，DRDA 在 6 张地图上都不同程度地提高了性能，在难和极难地图中表现都很优异，胜率超过基线，本章的方法比基线方法收敛得更快。消融实验进一步揭示了 DRDA 中的三个组件对性能不同程度的影响。

本章的工作在行动差异的基础上，根据行动的贡献对行动进行划分，从更全面的角度来定义角色，测量不同智能体之间的角色距离。本章利用行动差异和行动贡献将角色划分为长期角色与短期角色。在角色选择中通过奖励视野研究角色引起的损失，使得角色奖励更加合理。本章根据行动差异和行动贡献对完整的行动空间进行分解并且不固定角色发现，使得动态角色发现成为可能。

3.2 基于角色的任务分解模型构建

本章主要思路是基于行动差异和行动贡献来定义角色，不同动作对环境变化和对智能体的影响程度不同，根据这些因素来分解并形成不同的角色动作空间，智能体扮演的不同的角色存在角色距离。然后通过考虑奖励视野的角色选择器给智能体赋予角色，再通过角色策略在角色动作空间上选择策略。这增加了智能体的探索过程，可以让智能体获得先验知识，同时缩小了智能体的行动空间，在需要探索的环境上有很好的效果。

本章将多智能体合作任务建模为分散部分可观察马尔可夫决策过程^[167,168]，该过程由元组 $G = \langle N, S, A, P, R, \Omega, O, n, \gamma \rangle$ 组成，其中 N 是 n 个智能体的有限集合， $s \in S$ 是环境的真实状态。在每个时间步 c ，每个智能体 $i \in N := (1, \dots, n)$ 都有一个由观察函数 $O(s, i): S \times N \mapsto \Omega$ 确定的独立观察 $o_i \in \Omega$ 。智能体 i 选择一个动作 $a_i \in A$ ，形成联合动作空间 $a \in A^n$ 。过渡函数 $P(s'|s, a): S \times A^n \times S \mapsto [0, 1]$ 定义下一个状态 s' 。所有智能体共享相同的奖励函数 $r = R(s, a)$ ， $\gamma \in [0, 1]$ 是折扣因子。每个智能体都有局部的行动-观察

历史 $\tau_i \in T \equiv (\Omega \times A)^*$ 。

本章在 CTDE 方案的基础上, 设计一个基于角色的多智能体团队任务分解框架, 将多智能体的团队合作任务分解为不同层次下的子任务, 每个子任务都与一个角色相关联。每个智能体在一个时间步内扮演一个角色, 在相同时间步内扮演相同角色的智能体可以共享信息, 共同学习解决与角色相关联的子任务的策略。将团队目标高层次地分解为子目标是很重要的, 因为它是团队成员能够快速了解团队并解决问题的一般方法。通过角色将主要目标分解为其子目标, 不指定这些子目标是如何实现的或由谁实现的。例如, 在星际争霸的对抗场景中的目的是消灭对方成员, 消灭对方成员会分为移动、包围、攻击等子任务。

本章给定一个多智能体合作任务 G , 一组角色集合 Ψ , 角色 ρ_j , 子任务 g_j 。 $G = \langle N, S, A, P, R, \Omega, O, n, \gamma \rangle$ 。角色 $\rho_j \in \Psi$ 由一个元组 $\langle j, r_{i,j}, g_j, \pi_{\rho_j} \rangle$ 定义, j 是角色的标识; $r_{i,j}$ 是角色的奖励函数; $\pi_{\rho_j} : T \times A_j \rightarrow [0, 1]$ 是与子任务 g_j 相关联角色的角色策略; $g_j = \langle N_j, S, A_j, P, R, \Omega_j, O, \gamma \rangle$, $N_j \subset N$, $\cup_j N_j = N$, 智能体在时间步 c 只能扮演一个角色: $N_j \cap N_k = \emptyset$ if $j \neq k$ 。角色 ρ_j 的受限行动空间为 A_j , 不同角色的行动空间可出现重合情况: $A_j \subset A$, $\cup_j A_j = A$ and $|A_j \cap A_k| \geq 0$ if $j \neq k$ 。

在不同场景之下期望可动态找寻到一组角色 Ψ^* , 该角色组可使全局预期收益函数 $Q^\Psi(s_t, a_t) = \mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} [\sum_{i=0}^{\infty} \gamma^i r_{t+i} | s_t, a_t, \Psi]$ 实现最大化。全局预期收益会受到角色划分以及角色选择这两方面因素的影响。在本章所构建的机制里, 角色划分与角色选择均并非固定不变, 该机制可被视作是一种动态分组机制。借助对角色限制的行动空间展开探索, 学习角色策略。当智能体选择了一个角色时, 解决子任务的策略也同时确定。

3.3 动态角色发现与分配的方法

在团队合作完成一项任务时, 个体的角色并不是固定的, 在不同的状态下, 每个角色所包含的个体数也不是固定的。例如, 在搜救任务中, 一开始所有的人员都为搜寻者, 去寻找搜救目标的位置, 当找到搜救目标的位置后, 一部分人员成为救援者实施救援行动, 另一部分人员继续担任搜寻者寻找下一个目标。在多智能体合作任务中, 通过这样的方式可以实现智能体之间的协同。动态角色发现和分配 (Dynamic Role Discovery and Assignment, DRDA) 的学习框架如图 3-2 所示。本文首先介绍如何根据

行动来发现一组角色来分解多智能体合作任务。然后，本文讨论智能体如何根据角色适配性来选择角色。最后，在智能体选择完角色后，本文提出角色的策略，角色策略与行动表征相关的。

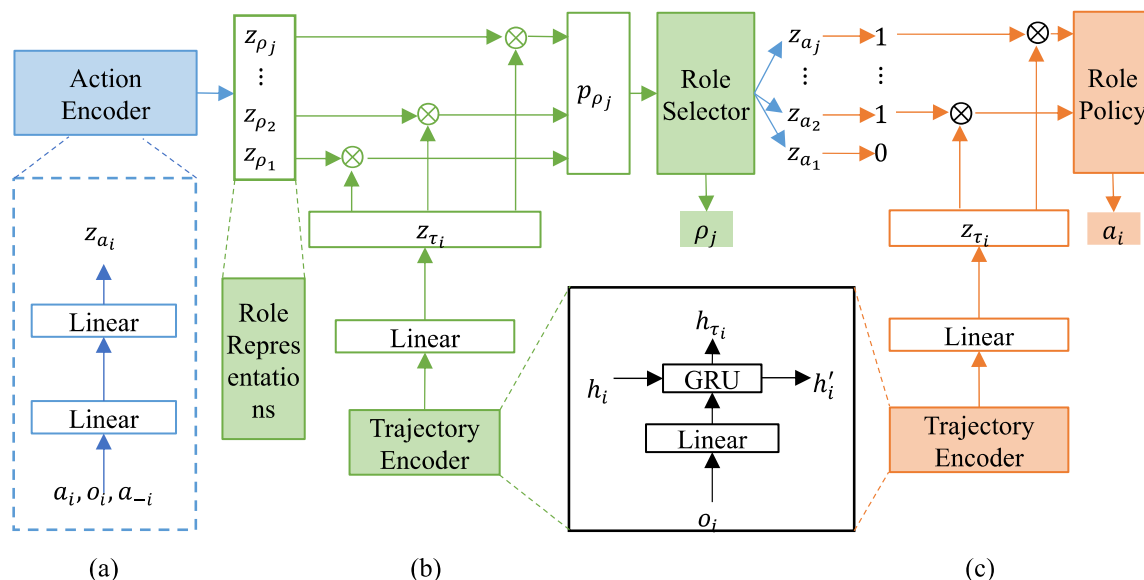


图 3-2: DRDA 整体框架。(a) 动作表征学习结构。(b) 智能体的角色选择结构。(c) 角色的策略学习结构。

3.3.1 基于行动表征的角色发现

角色划分方案在很大程度上决定了机制的效果，如何确定角色分类的方法是至关重要的。一般来说不同智能体在同一时间段会根据状态输出不同的行动，这些不同的行动可以表示为不同的角色。但是基于行动差异定义的角色无法反映角色不同的视野距离、动作空间和奖励函数。因此，行动差异不能作为定义角色的唯一标准。本章在行动差异的基础上，引入行动价值来定义角色类型差异，称为动作贡献。基于奖励函数来估计动作的 Q 值或状态值， Q 值为单个动作的贡献，同时也是动作对团队的贡献。

在很多场景下，智能体采取的行动虽然不同，但行动对周围环境的影响高度相似。例如，在足球比赛中，球员在进攻过程中互相传球，每个时间步球员采取的动作不会相同，在改变互相传球这个行动之前球员互相传球的行动影响高度相似，两个球员扮演的角色实际上也很相似。由于基于时间步长发现的角色数量会很庞大，角色之间的差异却很小，本章提出了一个正则器来保持角色之间的差异，维持角色区分度。

通常情况下复杂的任务会涉及一组具有不同职责的角色。例如在足球比赛中，不同的球员会担任不同的角色进行比赛。之前的工作往往建立在先验知识和领域知识的基础上，通过预先定义来分解任务，在很大程度上限制了工作的扩展性。为此，本文

学习能够反映行动对环境和其他智能体的影响的行动表征。本文首先根据行动所具有的属性对动作进行表征并聚类，确定行动所属的行动空间，根据表征结果计算动作贡献，相同空间中角色贡献相似的保持聚类结果，贡献不同的标记为离群点的动作。具有类似行动效果的行动空间可以看作相同的角色，行动效果不同的行动空间可以看作不同的角色，据此本文确定行动所属的行动空间及最终的角色分类。

如算法 3-1 所示，本章使用图 3-1(a)中的两层线性网络模型来学习参数为 θ_e 的动作编码器 $f_e(\cdot; \theta_e)$ ，编码器 $f_e(\cdot; \theta_e)$ 将独热编码动作(one-hot actions) a_i 从 $\mathbb{R}^{|A|}$ 映射到 $\mathbb{R}^m$ 然后，本章利用 $f_e(\cdot; \theta_e)$ 为每个动作 a_i 构建一个向量表示 $x_{\phi_i} = f_e(a_i; \theta_e) \in \mathbb{R}^m$ ，同时输出行动 a 在这个空间中的动作表征 $z_a = Q_a x_{\phi}$ 。通过预测值 $\tilde{o}'_i, \tilde{r}_t$ 和真实值 o_i, r_t 的差值计算损失函数值，并对以下网络进行更新：

$$\mathcal{L}_e(\theta_e, \xi_e) = \mathbb{E}(o, a, r, o') \sim \mathcal{D} \left[\sum_i \|\tilde{o}'_i - o'_i\|_2^2 + \lambda_e \sum_i (\tilde{r}_t - r)^2 \right] \quad (3.1)$$

其中， λ_e 是超参数，影响智能体的训练重心，调整具体的值可以改变预测下一步观测、预测收益的重要程度。 $\mathcal{D}$ 是一个重放缓冲区。

Algorithm 3-1: Role delineation learning algorithm

Input: tuple $G = \langle N, S, A, P, R, \Omega, O, n, \gamma \rangle$; A group of roles $\rho_j \in \Psi = \langle m, r_{i,m}, g_j, \pi_{\rho_j} \rangle$;

sub-tasks $g_j = \langle N_j, S, A_j, P, R, \Omega_j, O, \gamma \rangle$; action encoder f_e .

Output: Action representation z_a ; loss $\mathcal{L}_e(\theta_e, \xi_e)$

```

1 Initialize replay buffer  $\mathcal{D}$  and sample  $(o, a, r, o')$ 
2 for  $i \in N := \{1, \dots, n\}$  do
3     while not terminated do
4         for  $a_i \in A$  do
5             Give  $K$  roles
6             Randomly draw some data from the dataset to form a small batch and using the encoder
               $f_e(a; \theta_e)$  to cluster the actions according to their latent representations
7             Get the action representation  $z_a$ 
8             Use action representation to calculate action contribution  $Q_{\pi}(s, a_k)$ 
9             Predict the next observation  $\tilde{o}'_i = p_o(z_{a_i}, o_i, a_{-i})$ .
10            Predict the next global reward  $\tilde{r}_t = p_r(z_{a_i}, o_i, a_{-i})$ 
11            Update action encoder  $f_e$  by minimizing the loss  $\mathcal{L}_e(\theta_e, \xi_e)$ 
    
```

如果角色没有差异，那么会出现大量行动效果高度相似的角色，这会增加框架的复杂度。因此为了减少这些重复的角色出现，提现角色之间的差异，维持角色之间的区分度，本章提出一个最大化角色表示之间的 L2 距离的正则器：

$$\mathcal{L}_a(\theta_e) = \mathbb{E}_{\mathcal{D}} \left[- \sum_{i \neq j} \|x_{\phi_i} - x_{\phi_j}\|^2 \right] \quad (3.2)$$

本章运用经验回放缓冲区中的样本 (o, a, r, o') 进行监督学习，给定 K 个 roles，也就

是 K 个类别，每个角色都有完整的动作空间。在收集样本并训练预测模型一段时间后，根据行动表征用 **K-Means** 进行聚类，使用样本对群集质心进行更新。异常值标记为 outliers，outliers 的动作加到所有的角色里面，更新聚类结果，得到不同的类别就是不同的角色。新的动作表征包含着动作效果信息，每个聚类结果代表一个角色，角色的受限行动空间就是一个类别中的动作，具有相同的动作效果。据此本文确定行动所属的行动空间及最终的角色分类。行动效果可以通过行动奖励和局部观察的变化来衡量。通过监督学习，隐动作表征 z_a 包含着和原有动作 a 一样的动作效果（状态转移&回报值）。训练过程中，动作表征通过不断学习适应环境的动态变化，训练完成之后，角色和对应的动作空间固定。因此，角色发现的总体优化目标是：

$$\mathcal{L}_e(\theta_e, \xi_e) = \mathcal{L}_e(\theta_e, \xi_e) + \lambda_a \mathcal{L}_a(\theta_e) \quad (3.3)$$

其中， λ_a 是正则器的正系数。

3.3.2 基于表征的角色选择

考虑到不同角色之间的差异性以及智能体本身的行为习惯，本文要设计一个基于角色与智能体表征相似性的角色选择策略。本文角色分为短期角色与长期角色两种类别。本文研究了奖励视野所引起的角色奖励损失，在折扣因素中体现。这是因为在团队中，有的角色是长期奖励角色，没有短期的收益，使用相同的奖励函数不合理。不同类别的角色的奖励视野将会有所区别：一些类别的角色偏向于即时的回报（例如，射杀眼前的猎物），一些类别角色则偏向于长期的回报（例如，守卫营地）。角色选择策略是一个传统的 **Q** 网络，输入本地行动观察历史，输出角色的 **Q** 值。每过 c 个时间步，角色通过角色选择器选择一个角色，每次选择相当于确定接下来 c 个行动的可能动作集合，角色的 **Q** 值与角色表征密切相关。角色会在给定的时间范围内发挥作用，根据奖励视野和奖励函数获得不同的奖励。

如算法 3-2 所示，有了动作表征，本文可以将角色表征 z_{ρ_j} 建立在角色 ρ_j 可用行动的平均表示上，把角色包含的动作空间的表征求平均：

$$z_{\rho_j} = \frac{1}{|A_{\rho_j}|} \sum_{a_k \in A_{\rho_j}} z_{a_k} \quad (3.4)$$

其中， z_{ρ_j} 为角色 ρ_j 的表示， A_{ρ_j} 是角色 ρ_j 的受限行动空间。本文使用图 3-1(b)的结构学习角色选择，首先输入智能体 i 的观测 o_i ，经过一个由共享的线性层和 **GRU**^[169]组成的共享轨迹编码器，将局部行动观察历史 τ 编码为一个向量 h_τ ，这两个网络的参数为 θ_{τ_β} 。角色选择器 $f_\beta(h_\tau; \theta_\beta)$ 是一个参数为 θ_β 的全连接网络，将向量 h_τ 映射到 $z_\tau \in \mathbb{R}^d$ ，长度

与行动表征相同，对于角色选择器，观测状态为 τ_i 时，智能体 i 选择角色 ρ_j 的预期收益为 $Q_i^\beta(\tau_i, \rho_j) = z_{\tau_i}^T z_{\rho_j}$ 。

Algorithm 3-2: Role selector learning algorithm

Input: Action representation z_{a_k} ; agent observation o_i ; role selector f_β .

Output: role representation z_{ρ_j} ; loss $\mathcal{L}_\beta(\theta_{\tau\beta}, \theta_\beta, \xi_\beta)$.

```

1 Initialize replay buffer  $\mathcal{D}$  and sample  $(o, a, r, o')$ 
2 for  $i \in N := \{1, \dots, n\}$  do
3   while not terminated do
4     for  $\rho_j \in \rho$  do
5       Set role representation  $z_{\rho_j}$ 
6        $h_\tau \leftarrow o_i$ 
7       role selector  $f_\beta$   $z_\tau \in \mathbb{R}^d \leftarrow h_\tau$ 
8       if agent  $i$  select a role  $\rho_j$  then
9         The expected payoff value  $Q_i^\beta(\tau_i, \rho_j)$  of agent  $i$  selects role  $\rho_j$  when the observed state is  $\tau_i$ 
10        Update the role selector  $f_\beta$  by minimizing the TD loss  $\mathcal{L}_\beta(\theta_{\tau\beta}, \theta_\beta, \xi_\beta)$ 
    
```

考虑到智能体观察历史和角色表示的相似性，对于每个智能体 i ，本章计算行动-观察历史表示 h_τ 和角色表示 $z_\rho := [z_{\rho_j}]_{j=1}^k$ 的相似性，即相似性 $(h_\tau, z_{\rho_j}) = Q_i^\beta(\tau_i, \rho_j) / (\|h_\tau\| \|z_{\rho_j}\|)$ 。本章使用 $Q_i^\beta(\tau_i, \rho_j)$ 来近似余弦相似度。然后对余弦相似度采用 softmax 函数，得到角色选择的分类分布 $p(\rho|h_\tau, z_\rho) := [p(\rho_j|h_\tau, z_\rho)]_{j=1}^k$ 。 $p(\rho_j|h_\tau, z_\rho)$ 是智能体 i 选择角色 ρ_j 的概率：

$$p(\rho_j|h_\tau, z_\rho) = \frac{\exp(Q_i^\beta(\tau_i, \rho_j))}{\sum_{i=1}^n \exp(Q_i^\beta(\tau_i, \rho_j))} \quad (3.5)$$

其中， $\exp(-)$ 是指数函数。为了使角色选择过程具有可训练性，本章使用直通 Gumbel-Softmax 估计器来对角色 ρ_j 进行抽样。

为了更好地协调角色分配，解决多智能体学习中的角色分配问题，本章在 QMIX 方法的基础上利用 Q_i^β 估计联合作用价值函数全局 Q 值 Q_{tot}^β ，以当前状态为输入，利用超网络^[170]生成混合网络的参数 ξ_β 。用前面 c 步的回报收益 $R_{LSTRR}^{(c)}$ 加上 c 步后最优的角色选择的 Q_{tot} 值，减去当前预测的 Q_{tot} ，平方后关于经验回放缓冲区中的所有历史经验取期望作为损失函数。通过从重放缓冲区采样多个转换并时间差分误差（Temporal Difference Error, TD）损失来学习更新角色选择器 f_β ：

$$\begin{aligned}
 \mathcal{L}_\beta(\theta_{\tau\beta}, \theta_\beta, \xi_\beta) &= \mathbb{E}_D \left[\left(R_{LSTRR}^{(c)} + \gamma \max_{\rho'} \bar{Q}_{tot}^\beta(s_{t+c}, \rho'; \theta^-, \xi^-) \right. \right. \\
 &\quad \left. \left. - Q_{tot}^\beta(s_t, \rho_t; \theta, \xi) \right)^2 \right] \quad (3.6)
 \end{aligned}$$

其中, ρ 是所有可以选择的角色。 θ^-, ξ^- 是在多次迭代中保持恒定的目标网络参数。 $\bar{Q}_{tot}^\beta$ 是一个目标网络, $\rho = \langle \rho_1, \rho_2, \dots, \rho_j \rangle$ 是智能体可以选择的角色集合, 期望值用重放缓冲区 D 的均匀样本估计。

$$\begin{cases} Q_{tot}^\beta((s, \rho; \theta, \xi)) = \sum_{i=1}^n Q_i^\beta(\tau_i, \rho_i; \theta_i) \\ \frac{\partial Q_{tot}^\beta(s, \rho; \theta, \xi)}{\partial Q_i^\beta(\tau_i, \rho_i; \theta_i)} \geq 0, \forall i \in N \end{cases} \quad (3.7)$$

得益于式(3.7)中的单调性约束, 最大化联合 Q_{tot}^β 和最大化个体 Q_i^β 等同, 最佳个体动作与最佳联合动作保持一致。由于角色类别导致角色特定的奖励范围, 在训练的每一个时间步骤 c , 都有以下正则器:

$$R_{LSTRR}^{(c)} = \frac{1}{K_2} \sum_{j=1}^{K_2} \left(Q_i^\beta(\tau_i, \rho_j) - \sum_{c=0}^{T-c} \mu_j^t r^{(t+c)} \right)^2; \text{ for } K_2 \leq K \quad (3.8)$$

其中, $LSTRR$ 代表长短期奖励角色。这里 $Q_i^\beta(\tau_i, \rho_j)$ 是对与角色 ρ_j 相关的 Q 值的估计, $R_j = \sum_{c=0}^{T-c} \mu_j^t r^{(t+c)}$ 是角色 ρ_j 的折现奖励。这个正则器在集中训练过程中使用, 同时知道奖励的情况。在不丧失一般性的情况下, 本文假设 $\mu_1, \mu_2, \dots, \mu_{K_2}$ 是一个递减序列 (从长期到短期的视野)。

每个时间步, 智能体 i 将选择一个角色。当一个智能体在相邻的时间步中频繁改变角色时会使得训练过程不稳定。为了平滑智能体的角色选择并稳定训练, 本章引入了最小化相邻时间步的角色选择分布之间的 KL 背离的正则器 $\mathcal{L}_h(\theta_e, \theta_h)$:

$$\mathcal{L}_h(\theta_e, \theta_h) = \mathbb{E}_D \left[\sum_a D_{KL}(p(\rho|h_\tau, z_\rho) || p'(\rho'|h'_\tau, z_{\rho'})) \right] \quad (3.9)$$

其中, $p'(\rho'|h'_\tau, z_{\rho'})$ 是下一时间步的角色选择分布, $D_{KL}(\cdot || \cdot)$ 是 KL 发散算子, 总和在所有智能体中进行。

考虑到式(3.7)中的正则化, 角色选择的总体优化目标是:

$$\mathcal{L}_\beta(\theta_{\tau\beta}, \theta_\beta, \xi_\beta) = \mathcal{L}_\beta(\theta_{\tau\beta}, \theta_\beta, \xi_\beta) + \lambda_h \mathcal{L}_h(\theta_e, \theta_h) \quad (3.10)$$

其中, λ_h 是这正则器的正系数。

3.3.3 与表征相关的角色策略选择

智能体选择角色后, 在接下来的 c 个时间步中不会改变角色, 行动空间也会受到限制。每个角色 ρ_j 都有一个由受限行动空间定义的角色政策 $\pi_{\rho_j} : T \times A_j \rightarrow [0, 1]$ 。本章如图 3-1(c)所示学习每个角色的策略。一般来说, 扮演同一角色的智能体可以互相

学习策略参数，不同的角色的政策参数是不同的。

Algorithm 3-3: Role policy selection algorithm

Input: Action representation z_{a_k} ; agent observation o_i ; Role Policy Selector f_{ρ_j} .

Output: role policies π_{ρ_j} ; loss $\mathcal{L}_\rho(\theta_{\tau_\rho}, \theta_\rho, \xi_\rho)$

```

1 Initialize replay buffer  $\mathcal{D}$  and sample  $(o, a, r, o')$ 
2 for  $\rho_j \in \rho$  do
3   while not terminated do
4     for  $a_k \in A_{\rho_j}$  do
5       if agent  $i$  select action  $a_k$  according to  $\varepsilon$ -greedy then
6         The role policy  $f_{\rho_j}(h_\tau; \theta_{\rho_j})$  maps  $h_\tau$  to  $z_\tau$ 
7         Get  $r(s_t, a_t) = \sum_{j=1}^J r_t^j$  and the state  $s_{t+1}$ 
8         Set  $Q_i(\tau_i, a_k) = z_{\tau_i}^T z_{a_k}$ 
9         Update role policies  $f_{\rho_j}$  by minimizing the TD loss  $\mathcal{L}_\rho(\theta_{\tau_\rho}, \theta_\rho, \xi_\rho)$ 
10     $s_t$  is the terminate state

```

与角色选择策略类似，学习角色策略是一个可以直接估计每个动作 Q 值的深度 Q 网络，基于行动效果的 Q 值可以充分利用角色表示的差异信息。如算法 3-3 所示，本章使用和角色选择相同的共享轨迹编码器，将局部行动观察历史 τ 编码为一个向量 h_τ 。角色策略 $f_{\rho_j}(h_\tau; \theta_{\rho_j})$ 是一个全连接网络，将 h_τ 映射到 z_τ ，得到观测的表征 $z_\tau \in \mathbb{R}^d$ ，观测的表征 z_τ 和角色的表征 z_{a_k} 做内积得到的 Q 值是智能体 i 选择一个原始行动 a_k 的价值。智能体选择角色 ρ_j 后，在观测状态为 τ_i 时执行动作 a_k 的价值函数 $Q_i(\tau_i, a_k)$ 。本文估计智能体 i 选择一个原始行动 a_k 的价值为：

$$Q_i(\tau_i, a_k) = z_{\tau_i}^T z_{a_k} \quad (3.11)$$

通过全局奖励学习 Q_i ，将局部 Q 值输入参数为 ξ_ρ 的 QMIX 式混合网络，以估计全局行动值， $Q_{tot}(s, a)$ 。用回报收益 r 加上最优的动作选择的 Q_{tot} 值，减去当前预测的 Q_{tot} ，平方后关于经验回放缓冲区中的所有历史经验取期望作为损失函数，最小化 TD 误差损失来学习更新角色策略 f_{ρ_j} ：

$$\begin{aligned}
\mathcal{L}_\rho(\theta_{\tau_\rho}, \theta_\rho, \xi_\rho) &= \mathbb{E}_D \left[\left(r(s_t, a_t) + \gamma \max_{a'} \bar{Q}_{tot}(s', a'; \theta^-, \xi^-) \right. \right. \\
&\quad \left. \left. - Q_{tot}(s, a; \theta, \xi) \right)^2 \right] \quad (3.12)
\end{aligned}$$

其中， a 是智能体 i 所有可选的动作。 $\bar{Q}_{tot}$ 是一个目标网络， θ_ρ 是所有角色策略的参数，期望值是用与角色选择器相同的重放缓冲器 D 的统一样本来估计。因此，每个智能体只训练选定角色的政策参数。这样，具有相似能力的智能体倾向于选择相同的角色，可以分享他们的经验，加速训练并提高性能。

3.4 实验结果与分析

3.4.1 实验设计

本章选择 SMAC 基准^[166]作为测试平台。SMAC 基准作为 MARL 领域颇为流行的一个基准，借助星际争霸 II 学习环境构建而成，可为解决竞争与合作兼具的多智能体问题搭建起一个相当不错的测试平台。智能体于 SMAC 基准里的行动空间是由一组离散行动所构成的：在上下左右这四个基本方向上移动，并且在每个时间步做出选择，要么攻击一个敌人，要么停止行动，要么不采取任何行动。多智能体仅可对处于攻击范围内的敌人发起攻击，要是地图上存在 n_e 个敌人，那么每个盟友单位的行动空间便包含了 $n_e + 6$ 个离散行动。本章所设定的以便将胜率最大化，也就是赢得的游戏数量与所进行的游戏总数量的比率。在测试过程中，一种方法在规定时限内成功击败所有敌方单位的情节所占的百分比会被称作测试胜利率。

在所有实验中使用 RMSprop (Root Mean Square Propagation) 进行优化，所有超参数设置和结构细节如表 3-1 所示。角色选择器和角色策略从重放缓冲区取样，在完全未滚动的情节上进行端到端训练。目标更新系数(τ)设为 0.005，隐藏层单元数为 64。角色策略和角色选择的混合网络使用相同的架构，混合网络的参数设置与 QMIX^[140]相同。实验使用 SMAC 基准的默认奖励和观察设置^[166]。基线使用作者提供的代码，超参数在 SMAC 基准上进行微调。实验在 NVIDIA RTX 2060 GPU 上进行，训练时间从 12 小时到 48 小时不等。使用 k-means 聚类确定角色行动空间。聚类的数量 k 是一个超参数。在同质化敌人的地图上，将 k 设置为 5，在异质化敌人的地图上，将 k 设置为 7。

表 3-1 超参数设置和结构细节

参数	值
学习率	5×10^{-4}
缩放因子(λ_e)	10
折扣因子(γ)	0.99
探索策略	ϵ -greedy
缓冲区大小 (回合数)	50k
批量大小	32
目标更新系数(τ)	0.005
隐藏层单元数	64
GRU 隐藏状态	64
非线性激活函数	ReLU
使用 ReLU 激活的隐藏层单元数	32
动作表示长度(d)	20
聚类数	5 or 7

3.4.2 性能表现

为了评估方法性能，本节中所有实验都是用不同的随机种子进行的，在选定的地图上用不同的种子对每种方法运行 2 次，并显示了性能的中位数。SMAC 地图被分为简单（easy）、困难（hard）和极难（super hard）三个不同难度的种类。从不同难度的种类中各选取两张地图进行实验。简单地图中选取 3s5z、10m_vs_11m 这两张地图，从困难地图中选取 2c_vs_64zg、5m_vs_6m 这两张地图，极难地图中选取 corridor、27m_vs_30m 这两张地图。地图的详情信息如表 3-2 所示，困难地图和极难地图是高难度探索任务的典型例子。在这 6 张地图中评估方法在 SMAC 实验环境中的性能和表现。本文与基于价值的 MARL 算法（VDN^[171]和 QMIX^[140]）进行比较，考察不同方法在训练步骤内的胜率变化。经过 2M 的训练步骤，本文的方法在 6 个场景中的 4 个场景中至少比基线高出 1/20。

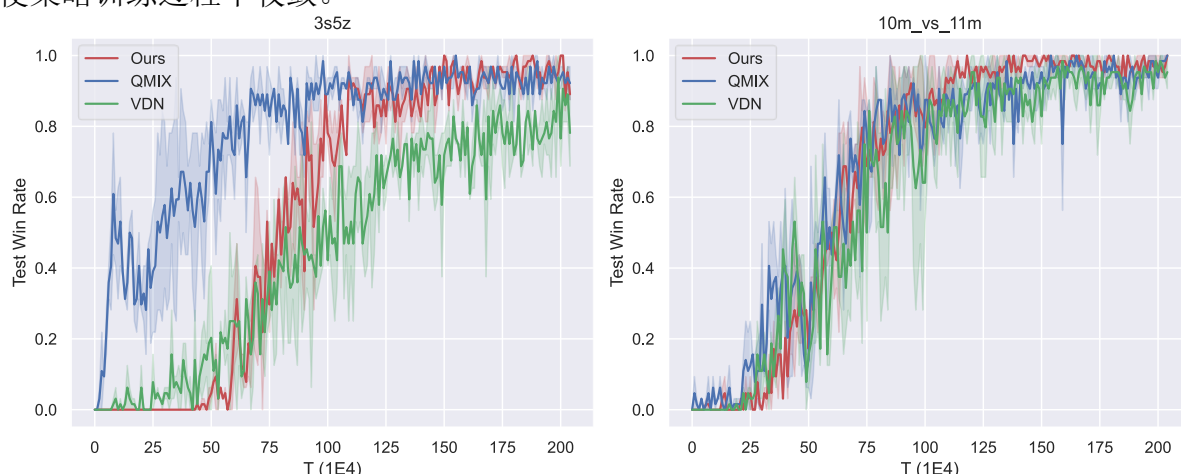
表 3-2 SMAC 地图详细信息

名称	友方单位	敌方单位	类型
3s5z	3 个追猎者 (Stalkers)和 5 个狂热者 (Zealots)	3 个追猎者(Stalkers)和 5 个狂热者(Zealots)	异构且对称(Heterogeneous and symmetric) 同构且不对称
10m_vs_11m	10 个陆战队员(Marines)	11 个陆战队员(Marines)	(Homogeneous and asymmetric)
2c_vs_64zg	2 个巨像(Colossi)	64 只异虫(Zerglings)	微操技巧：定位(Micro- trick: positioning) 同构且不对称
5m_vs_6m	5 个陆战队员(Marines)	6 个陆战队员 (Marines)	(Homogeneous and asymmetric) 同构且不对称
27m_vs_30m	27 个陆战队员(Marines)	30 个陆战队员 (Marines)	(Homogeneous and asymmetric)
Corridor	6 个狂热者(Zealots)	24 只异虫 (Zerglings)	微操技巧：堵截(Micro- trick: wall off)

3s5z 以及 10m_vs_11m 是简单地图里两个颇具代表性的测试场景，3s5z 场景囊括 3 个 Stalkers 以及 5 个 Zealots 这样的异构智能体，而 10m_vs_11m 场景涉及 10 个 Marines 去对抗 11 个敌方 Marines 的同构智能体，这两个场景反映出不同复杂程度下的智能体协作任务：前者侧重于异构智能体之间的任务分工以及协作，后者着重于大规模同构团队的战略协调。

如图 3-3(a)所示，在 3s5z 场景中本文的方法于测试胜率方面呈现出优势。在训练的早期阶段，QMIX 的胜率提升速度最快，曾经领先于本文的方法，然而随着训练的推进，本章的算法开始呈现出更强的学习能力，并且在中后期快速超过 QMIX，最终测试胜率接近 1.0。本章的动态角色分配机制可更高效地引导智能体协作，在更复杂

的任务场景中表现优异。相比之下，VDN的表现相对欠佳，其胜率在训练全过程都维持在较低水平，而且增长速度明显慢于其他方法，VDN在应对异构智能体的角色分配以及协作需求时能力欠缺，这种劣势可能源于其对联合动作空间的简单处理，没能有效利用个体智能体之间的差异性来提升系统性能。另外本章的方法在整个训练过程中表现出更高的稳定性，胜率曲线波动较小，这种稳定性得益于本文算法中的正则化机制，依靠限制智能体频繁切换角色，减轻了因角色分配动态变化而产生的不稳定性。这种特性在复杂场景中非常关键，因为频繁的角色切换可能会降低协作效率，甚至致使策略训练过程不收敛。



(a) 地图 3s5z 中的表现

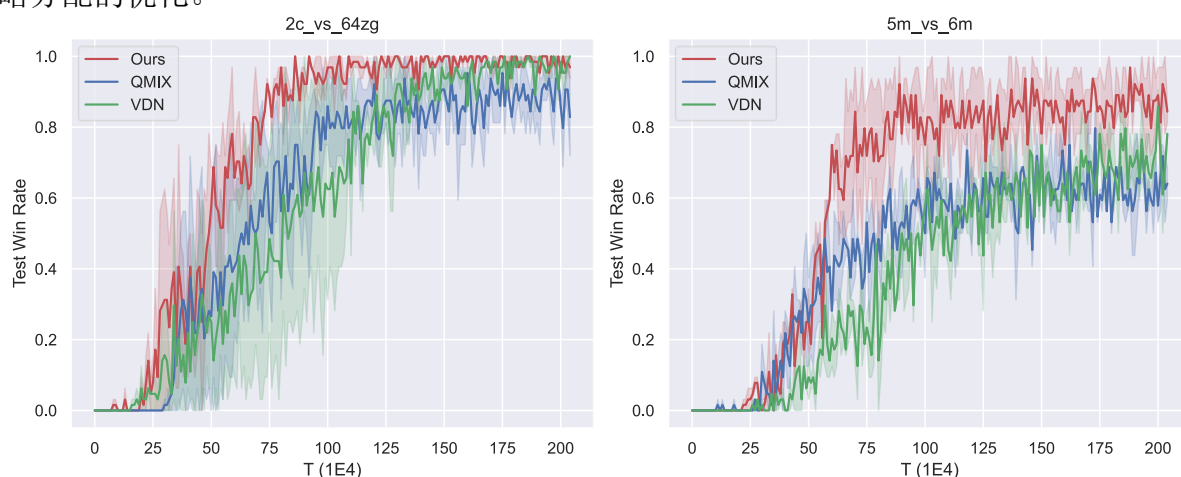
(b) 地图 10m_vs_11m 中的表现

图 3-3 不同方法在简单地图中的对比表现

10m_vs_11m 场景的复杂性主要体现在两方面，一方面是智能体数量有所增加，另一方面是对敌方团队策略的反应需求变得更高。在这一场景当中，本章的方法同样呈现出了强大的适应能力。从图 3-3(b)可看出，在训练初期本文的方法胜率增长较为缓慢，而 QMIX 仍然表现出较快的收敛速度。不过在训练中后期本文的方法胜率迅速提升，最终接近 0.95，表现要优于 QMIX 和 VDN。这样的结果说明，本文的方法借助动态角色发现与分配机制，在较大规模的智能体团队里可协调策略分配，并且利用基于动作表示的模型缩小联合动作空间，提高策略优化效率。与之相比，QMIX 的性能在训练中期趋于平稳，最终胜率停留在约 0.9 的水平。这种差异反映出 QMIX 在处理较高任务复杂性时存在局限性，其联合动作估值策略在智能体数量增加时难以保持足够的扩展性。VDN 在 10m_vs_11m 场景中的表现与其在 3s5z 场景中相似，同样处于劣势，其胜率在整个训练过程中增长缓慢，最终停留在约 0.85 的水平，缺乏对复杂场景中任务分解与协作的有效支持，在智能体数量增多的情况下，策略优化能力显得非常不足。

从实验结果可看出，本章的方法于不同任务复杂性状况下均呈现出较强适应能力与优势，呈现出更高学习效率以及收敛性能。QMIX 的表现较为稳定，不过其胜率上限低于本文的算法，于复杂场景里无法持续优化策略。VDN 的表现最为不佳，无论在异构还是同构智能体任务中，都未能呈现出令人满意的协作效果。QMIX 在早期阶段胜率提升较快，有较强快速学习能力，然而这种优势在训练中后期逐渐被本章的方法超越，这可能是由于依靠动态角色分配机制在策略优化中逐步提升对复杂任务的适应性。

2c_vs_64zg 以及 5m_vs_6m 属于困难地图里两个有着不一样挑战性的场景。2c_vs_64zg 的复杂程度源自于要两个 Colossi 去面对 64 个 Zerglings 时存在数量上的劣势，这突出了智能体的定位策略以及微操能力。5m_vs_6m 则是一个相对均衡的场景，也就是 5 个 Marines 要对抗敌方 6 个 Marines，此任务的关键之处在于团队协作以及策略分配的优化。



(a) 地图 2c_vs_64zg 中的表现

(b) 地图 5m_vs_6m 中的表现

图 3-4 不同方法在困难地图中的对比表现

如图 3-4(a)所示，在 2c_vs_64zg 场景当中，虽然在训练初期三种方法没有拉开明显差距，不过本章的算法在训练前期快速超越 QMIX，并且持续提升，最终在训练结束的时候达到 0.95 的胜率。本章的算法在处理复杂环境之际，可逐渐适应任务需求并优化策略。相比而言，QMIX 的胜率在训练后期趋于平稳，最终停留在 0.8 左右，这意味着其在处理智能体数量悬殊的场景时，优化能力受到限制。VDN 的表现在训练前期是最差的，胜率一直较低，而且增长缓慢，在训练后期超越 QMIX，最终达到 0.85。VDN 在复杂场景里难以应对智能体协作需求。本章算法借助动态角色分配以及基于动作表示的策略优化，有效缩小了联合动作空间。在整个训练过程中，本文的方法胜率曲线平稳且持续上升，显示出更高的鲁棒性。QMIX 在训练后期胜率提升停滞，在复

杂场景中难以优化策略，而 VDN 胜率波动较大稳定性不足。

如图 3-4(b)所示，在 5m_vs_6m 这一特定场景之中，本文所采用的方法依旧呈现出十分出色的性能。从实验最终得出的结果可清晰看出，尽管在训练的初期阶段，QMIX 的胜率增长速度相对较快，然而在训练的中后期阶段，QMIX 却被本章的算法成功超越，最终本章的算法胜率达到了 0.9。QMIX 的最终胜率大约为 0.6，整体表现较为普通，明显比不上本章的算法。VDN 的表现还算可以，最终胜率为 0.7。5m_vs_6m 是一个各方面较为均衡的场景。在这个场景里，团队协作成为了极为关键的因素。本章的算法借助对智能体角色进行动态的调整，使得团队合作效率得到了优化，在后期的胜率较大高于其他方法。QMIX 虽说在均衡场景里表现得还不错，但是在团队协作的优化方面依然比不上本章的算法，在训练初期胜率提升速度很快，这显示出它有较强的快速学习能力，不过随着训练步数不断增加，其胜率增长陷入停滞状态，这也暴露了它在长期策略优化方面存在不足。VDN 的表现证明了它在策略分配以及角色协作方面存在着一定的局限性。而本章的算法尽管在早期学习速度比较慢，但是后期持续增长保证了可拥有更高的胜率上限。

在难度更高的 2c_vs_64zg 场景中，本章的方法呈现出了十分突出的优势，胜率大幅度领先于 QMIX 和 VDN，这充分说明本章的算法在面对复杂任务的时候，可凭借合理的角色分配以及有效的策略优化，将智能体的能力充分发挥出来。而在 5m_vs_6m 场景中，即便任务相对来说比较简单，本章的方法仍然呈现出优越的性能，胜率稳定提高，这显示出它有适应不同任务复杂程度的能力，本章的方法在这两个场景中的最终胜率都接近 0.9 或者超过 0.9，明显高于 QMIX 和 VDN，这种出色的表现得益于算法中引入的基于动作表示的角色发现机制以及动态角色分配策略。而 QMIX 的胜率上限比较低，表现出了十分突出的优化瓶颈。VDN 则在所有场景中的表现都不尽如人意，无法有效地应对复杂的智能体协作需求。

corridor 和 27m_vs_30m 场景都属于极难的地图，主要是为了考察算法在复杂环境下的协作能力以及优化表现。在 corridor 场景里，6 个 Zealots 要对抗 24 个 Zerglings，任务的最关键的是借助微操来达成有效的防御和进攻策略，27m_vs_30m 场景是 27 个 Marines 对抗 30 个敌方 Marines，这个任务需要团队高效协作，以应对敌方数量上的优势。两个场景都在角色分配和策略优化方面对多智能体协作机制提出了严格要求。

如图 3-5(a)所示，在 corridor 场景中本章的方法在训练中后期的胜率表现比 QMIX 和 VDN 要好。从图中可看出，虽然训练初期三种方法的胜率增长趋势相近，但是在大约 200 万步之后，本章的算法开始明显超过其他方法，最终测试胜率接近 0.8。

QMIX 的胜率在训练后期趋于平稳，最终停留在 0.1 左右，而 VDN 的表现远远不如其他两种方法，最终胜率仅达到 0.1。corridor 场景的挑战在于敌方数量的巨大优势，以及智能体要精确控制路径选择和攻击范围。本章的方法依靠动态角色分配和动作表示机制，有效提高了智能体间的协作效率，保证各智能体可依据任务需求快速适应角色转换，这使得系统在复杂场景中表现出了较高的胜率。本章所采用的方法于训练的整个过程当中呈现出了较高的稳定性，其胜率曲线的波动相对较小。与之形成对比的是，QMIX 以及 VDN 的表现欠佳，它们在进行策略优化时，于高复杂性任务里遭遇了瓶颈，呈现出稳定性不足的情况以及在复杂任务场景下的适应性方面的问题。

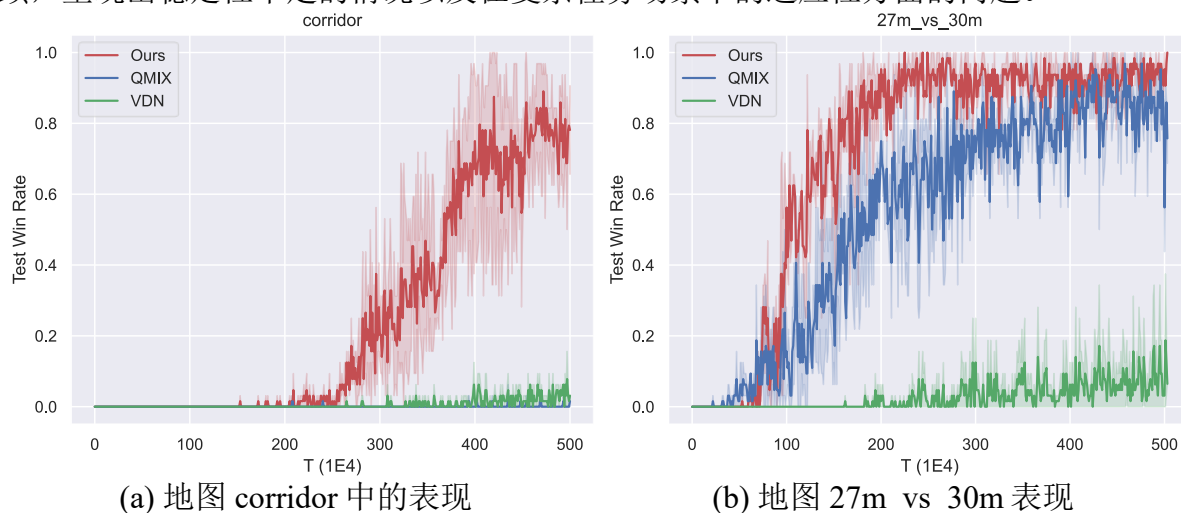


图 3-5 不同方法在极难地图中的对比表现

在 27m_vs_30m 中，本章所采用的方法呈现出了较为突出的优势。从图 3-5(b) 可看出，在整个训练进程里，本章的算法胜率一直处于领先地位，在大概 400 万步之后达到了 0.95 的最终胜率。QMIX 的胜率在训练前期增长得比较快，不过在训练后期渐渐趋于稳定，最终停留在 0.8 左右。VDN 在整个训练过程中的表现不太理想，最终胜率仅仅约为 0.1。27m_vs_30m 的复杂性体现在智能体数量的增多上，还体现在对团队协作有着更高的要求，本章的方法依靠角色分配以及策略优化，保障了智能体之间的协同作用，在复杂场景里实现了性能的提升。相比较而言，QMIX 的策略优化能力在团队规模变大的时候受到了限制，没能充分利用团队成员之间潜在的协作优势。QMIX 在训练初期的胜率增长速度较快，然而它的胜率上限比本文的方法要低，其对复杂策略的优化能力有限。而 VDN 的表现则显示出，它在应对大型团队任务时，策略分配以及角色协作的能力不足。

3.4.3 消融实验

本文的方法由组件 A 受限的行动空间，组件 B 使用行动差异和行动贡献进行聚类，组件 C 将行动表征整合到角色选择和角色政策中这三个组件构成。为了测试组件 A 和 B，本文保持框架中的其他部分不变，让角色动作空间变为包含所有动作的空间或者一个随机动作的空间。对于组件 B，本文在角色行动空间包含所有动作的情况下进行测试。对于组件 C，剔除动作表征用传统的 Q 网络来学习角色选择和角色策略。四种配置分别为：完整的动态角色分配算法（Ours），去除动作表示的版本（Ours No Action Repr），使用完整动作空间（Ours Full Action Spaces），随机限制动作空间（Ours Random Restricted Action Spaces），本文从三种不同难度的地图中各选一张进行消融实验。

从图 3-6 中可以看出，完整的算法在训练结束时的测试胜率接近 1.0，显著优于其他三种配置。这表明，动态角色分配与动作表示的结合，以及基于角色的限制动作空间，是提升系统性能的关键。使用完整动作空间的配置在训练后期的胜率接近 0.7，表现次优。相比之下，去除动作表示的配置和随机限制动作空间的配置表现较差，最终胜率分别停留在约 0.6 和 0.5。这反映了在多智能体任务中合理设计动作空间和角色分配的重要性。

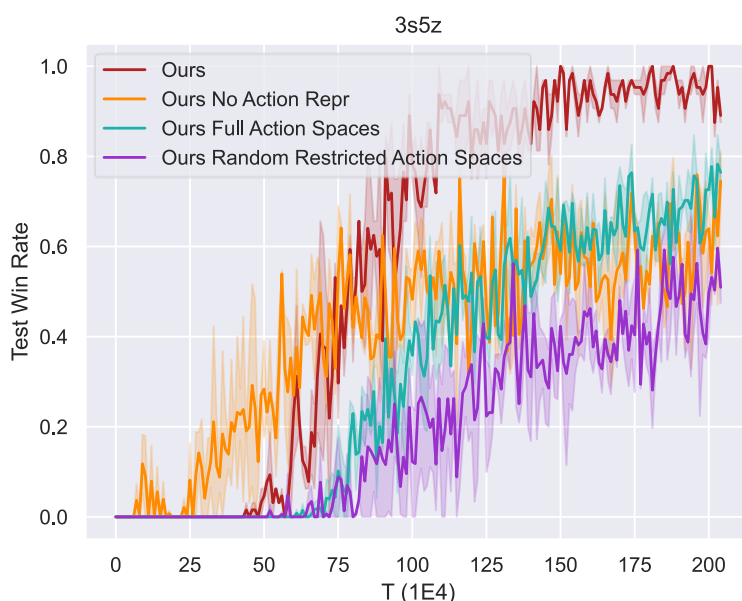


图 3-6 3s5z 中的消融实验

完整算法在训练初期的时候，其胜率增长速度相对比较平稳，不过随着训练步数不断增加，它就明显超过了其他配置，在复杂任务当中实现了高效的策略优化以及有较高的稳定性，完整动作空间被引入之后，联合动作空间的规模有所增加，这就让智能体在训练过程里需要面对更多策略探索方面的难题，使得系统不太容易高效收敛。限制动作空间可有效地减少策略学习的复杂程度，提高学习效率。去除动作表示的配

置在训练后期的胜率比完整算法以及去除动作表示的配置要弱一些,即便没有动作表示,动态角色分配机制对于多智能体协作任务依旧有着比较十分突出的提升作用,动作表示的缺失致使系统没办法充分捕捉动作之间的差异性以及贡献性,限制了角色分配的精确程度以及策略优化的效率。随机限制动作空间的配置表现是最差的,最终胜率仅仅为 0.5,随机限制动作空间有可能导致角色分配出现无效性以及策略优化失效,基于角色进行合理的限制动作空间可给智能体提供明确的策略范围,提升了系统性能。

实验里去除动作表示的配置性能出现了下降,这证实了动作表示在提升系统性能方面有着关键作用。限制动作空间明显减少了联合动作空间的规模,让智能体可在更小的策略空间内对策略进行优化,完整动作空间以及随机限制动作空间的低效表现说明,在多智能体系统中,限制动作空间是提高策略优化效率的关键手段,随机限制动作空间可能会破坏角色分配的有效性,角色与动作空间合理匹配是十分关键的。完整算法最终的胜率接近 1.0,明显要优于其他三种配置。动态角色分配框架中各个组件的协同作用可提升多智能体系统的协作效率以及策略优化能力。实验结果证明动作表示和限制动作空间是动态角色分配框架的关键组件,它们提高了系统的胜率,对策略优化的收敛速度以及稳定性产生了积极影响。在设计多智能体系统时,动态角色分配和动作表示机制可有效提升系统性能,合理设计和限制动作空间对于解决复杂任务有意义。

从图 3-7 可观察到,完整算法的最终测试胜率将近 0.9,比其他三种配置都要好很多,去除动作表示的配置在训练后期的胜率大概是 0.35,表现处于次优水平,使用完整动作空间的配置以及随机限制动作空间的配置表现欠佳,最终胜率分别停留在大约 0.3 和 0.2。

完整算法在训练初期胜率增长速度较为平稳,随着训练的推进,智能体之间的协作能力有了提升,最终胜率达到接近 0.9。动态角色分配与动作表示相结合,能帮助智能体在复杂任务里实现高效的策略优化,该配置在整个训练过程中表现出较高的稳定性,胜率曲线波动较小,完整算法可以优化最终性能,有着较强的鲁棒性。去除动作表示的配置表现次优,即便没有动作表示,动态角色分配机制对多智能体协作任务仍有提升作用。该配置的性能在训练后期出现了收敛停滞,胜率最终稳定在 0.35。动作表示的缺失限制了智能体捕捉动作间差异性和贡献性的能力,影响了角色分配的精确程度。使用完整动作空间的配置在训练中的表现比完整算法和去除动作表示的配置要弱,完整动作空间的引入使得联合动作空间的维度加重,加大了策略探索的难度,最终胜率停留在 0.3 左右。在多智能体系统中,借助限制动作空间来缩小策略搜索范

围是提高学习效率的关键所在，该配置的胜率曲线在训练初期增长缓慢，在高维动作空间中智能体难以稳定地学习到有效策略。随机限制动作空间的配置在四种方法中表现最差，最终胜率仅有 0.2，随机限制动作空间可能致使角色分配无效，让智能体无法形成有效的协作策略。基于角色的合理动作空间限制可给智能体提供明确的优化方向，提升系统性能，该配置的低效表现也反映出随机性对系统稳定性的破坏，在多智能体任务中角色分配和动作空间的设计要有明确的目标。

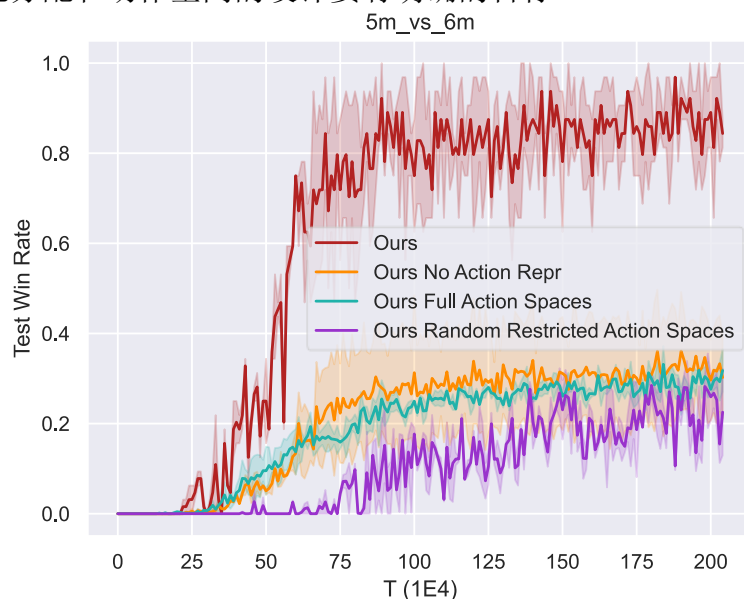


图 3-7 5m_vs_6m 中的消融实验

去除动作表示的配置在表现上虽然良好，然而却比完整算法逊色不少，这验证了动作表示对于提升系统性能有着关键作用，限制动作空间可减少联合动作空间的规模，使得策略优化的效率得到提升。完整动作空间和随机限制动作空间的表现显示，无限制的动作空间以及随机限制的动作空间都不能有效支持策略优化。基于角色的合理限制动作空间可为系统性能的提升提供关键支持。随机限制动作空间的低效表现意味着随机性在角色分配中的引入可能会破坏系统的优化目标，对智能体的协作能力产生影响，角色分配和动作空间设计要依据明确的目标和优化策略。完整算法的最终胜率接近 0.9，比其他三种配置要好很多，呈现出了动态角色分配框架的整体优势。去除动作表示的胜率相对平稳，动态角色分配机制对系统性能的提升有关键作用。完整动作空间和随机限制动作空间的胜率较低，验证了动作空间设计和角色分配对策略优化很关键。完整算法在训练全程表现得很稳定，完整动作空间和随机限制动作空间的胜率曲线波动较大，反映了随机性和高维动作空间对系统稳定性有负面影响。实验结果说明在设计多智能体系统时，动态角色分配、动作表示以及合理限制动作空间是提升系统性能的关键。

从图 3-8 可观察到,完整算法在训练后期的胜率明显高于其他配置,最终快要接近 0.8,去除动作表示的版本表现稍逊一筹,最终胜率稳定在 0.15 上下,未对动作空间进行限制的版本表现最差,胜率接近于 0,随机限制动作空间的版本表现欠佳,最终胜率仅仅达到 0.05。

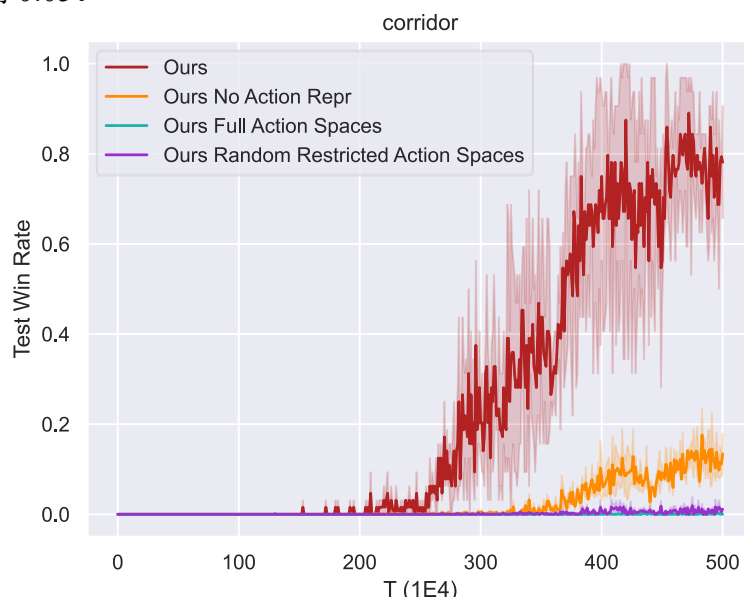


图 3-8 corridor 中的消融实验

在训练开始的阶段,完整算法的胜率增长态势较为平稳,随着训练不断推进,协作效率得到了提高,最终胜率快要接近 0.8,动态角色分配机制依据任务需求实时对智能体的角色以及动作范围作出调整,以此保证各个智能体在通道里可协同完成防守与进攻任务,这样的角色调整使得团队的整体适应性获得了提升。动作表示借助把离散动作映射到连续向量空间,可有效地捕捉动作之间的属性差异以及贡献性,让角色分配变得更加精确。限制动作空间有效地减少了智能体的策略搜索范围,提高了学习效率并且降低了探索噪声。去除动作表示的版本在训练后期表现欠佳,胜率最终稳定在了 0.15。即便没有动作表示,该版本依旧保留了动态角色分配机制,为多智能体协作提供了一定的优化能力,动作表示的缺失致使智能体难以捕捉动作之间的细微差别,限制了角色分配的精确程度,影响了整体策略优化效果,尽管性能比完整算法要差,该版本的胜率曲线在训练过程中相对比较平稳,动态角色分配机制对任务执行依旧有较强的提升作用。未限制动作空间的版本表现明显低于完整算法,最终胜率快要接近 0,完整动作空间的引入增加了智能体的联合动作空间维度,使得策略学习变得更加险阻,动作空间的扩展导致智能体在策略优化过程中需要处理更多的探索噪声,降低了训练效率以及最终性能,该版本的胜率曲线一直在 0 附近,完整动作空间未能有效地支持智能体的策略学习。随机限制动作空间的版本表现最差,最终胜率仅仅为

0.05, 随机性在动作空间限制中的引入可能会破坏角色分配的合理性, 随机限制动作空间可能致使角色分配与动作空间不匹配, 让智能体无法在有效的动作范围内优化策略, 随机限制动作空间降低了系统的优化能力, 对系统的稳定性产生了负面的影响。

完整算法的最终胜率快要接近 0.8, 明显比其他三种配置要好, 去除动作表示的版本次之, 与完整算法存在着较大的性能差距。未限制动作空间和随机限制动作空间的版本都表现不好, 验证了动作空间设计以及角色分配对算法性能的关键性, 动作表示提高了角色分配的精确程度, 为智能体策略优化提供了关键支持。动作空间限制依靠缩小策略搜索范围提高了学习效率以及系统性能, 动态角色分配在提升多智能体协作效率方面有核心作用, 动态角色分配和动作表示的结合是提升多智能体系统性能的关键, 合理设计动作空间对策略优化的成功非常关键, 随机性或无约束的动作空间可能会破坏系统的优化目标, 较大降低性能。运用传统深度 Q 网络方法所呈现出的性能与原始模型最为接近, 其性能表现平均下降幅度达到了 55%, 在不限制角色动作空间的消融之上。可以得出将行动表征整合到角色选择和角色政策中对本文方法的表现有着很大的影响, 受限的行动空间与优越性能的表现密切相关。具有随机限制角色行动空间的方法性能表现是三种消融实验中最差的, 较原始模型性能平均降低了 60%, 这突出了使用行动差异和行动贡献来分解联合行动空间的重要性。总之, 三种消融不同组件的方法在性能表现上均不如未消融的方法。可以看出, 限制角色行动空间和将行动表征整合到角色和策略中, 对本文的方法性能表现起到关键的作用。

3.5 本章小节

现有多智能体角色分配方法大多采用预定义角色或静态映射, 当任务类型、执行状态或环境条件发生变化时, 难以及时自适应地进行角色切换, 为此本章提出一个新框架用于学习动态角色发现与分配, 行动编码器依据每个行动的特性构建向量表示, 基于行动差异和角色差异从更全面的角度定义并划分角色。为合理为智能体分配角色, 考虑角色差异和奖励视野, 提出基于表征的角色选择策略, 使有相似能力的智能体动态扮演相同角色, 引入两个正则器增加角色间的代表性差异, 依靠阻止智能体频繁改变角色稳定训练, 角色选择和角色策略在受限行动空间整合行动表征和角色差异, 实验结果显示, 该方法能实现有效的角色发现与分配, 验证了动作表示和动作空间限制对系统性能提升的关键作用。

第四章 多智能体系统中的动态任务最大化重分配

4.1 引言

当所处的环境或者面临的任務出现变化的时候，处于静态环境中的多智能体系统所采用的任务分配方法，很难迅速做出调整来适应新的任务需求。尽管这些方法解决了任务分配领域的一些问题，但是大部分方法只适用于集中式系统，在动态环境中的应用存在许多问题。

在动态环境里开展任务分配以及重分配工作，是一个颇具挑战性的问题，需要全面考量任务类型的多样情况、智能体的不同特质、环境的难以预测性以及实时性方面的要求。任务的随机性和复杂性进一步加剧了难度，使得任务的动态分配与重分配成为研究的重点。在实际场景中，任务需求和智能体资源大多时候会随着时间的推移而发生改变。比如在灾害救援任务当中，随着救援进程的不断推进，新的任务需求可能会持续出现，原来任务的优先级也可能会跟着调整。怎样依据环境变化快速达成任务的动态重分配，并且在保证系统效率的基础上实现资源利用率的最大化，是需要解决的关键问题。现有的研究在多智能体任务分配领域取得了一定的成果，不过大多集中在静态场景，在动态环境中往往很难应对频繁变化的任务需求。当任务的优先级、截止时间或者资源需求发生变化时，静态任务分配方案可能会造成资源浪费、任务冲突甚至任务失败。如何在动态任务出现之时快速调整分配方案并实现全局的优化，已然成为当前研究的核心问题。动态任务分配与重分配的主要挑战在于可快速识别环境变化并制定出新的任务分配方案，实现任务完成率和系统效率的最大化。任务和资源的动态性可能会让原有的分配方案失去效用，智能体之间的通信延迟以及任务执行过程中的不确定性也会对系统的整体性能产生负面的影响。在多智能体系统中依据任务类型、智能体特性以及环境状态等因素动态调整任务分配^[172]是解决这些问题的关键所在。因此，动态任务分配需要结合任务类型、智能体特性以及环境状态等因素进行灵活的调整。

在动态环境里任务会在空间和时间方面持续发生改变，比如任务的产生、取消以及优先级的调整等情况。智能体的状态同样会随时间而改变，动态性使得系统要可实

时察觉到变化并快速对任务分配策略做出调整。在任务分配期间，系统要在全局最优和局部执行效率之间寻找到平衡。全局最优方案或许需要更繁杂的计算以及更高的通信开销，局部优化方案有可能致使整体性能降低。怎样设计出高效算法迅速响应局部变化并接近全局最优是一项挑战。许多任务之间存在着依赖关系或者资源冲突，一个任务的完成可能要先完成另一个任务，又或者多个任务可能会竞争同一资源。动态重分配过程中要合理处置这些依赖和冲突，避免资源浪费或者任务执行失败。动态环境中的频繁任务重分配可能会让系统不稳定，智能体行为的频繁切换可能会增加执行过程里的能耗和延迟。怎样在适应动态变化的同时维持系统稳定性，是任务重分配需要解决的关键问题。

基于市场的方法由于效率、鲁棒性和可扩展性等多重优势，经常用于解决多智能体系统中的任务分配问题^[173]。拍卖算法^[174,175]作为基于市场的方法被广泛应用于大规模任务分配问题中。在拍卖方法的基础上，基于共识的捆绑算法^[38]结合了拍卖和冲突解决的积极特性产生无冲突的分配。复杂合作约束^[176]和不完美通信^[177]在 CBBA 的基础上进行了修改和扩展。基于性能影响（Performance Impact, PI）算法^[62]是 CBBA 基础上的分布式启发式算法，在解决难度大、时间紧迫的问题时具有更好的性能。PI 算法的动态任务分配方法能够快速处理任务变化，适用于多智能体协作场景。PI 算法的分配结果往往受限于任务分配量的局限性，无法充分利用系统的资源导致全局效率较低。PI-MaxAss 算法^[178]通过转移分配任务来为未分配任务创造可行的时间段，最大化分配任务的数量。Whitbrook A^[61]允许在不确定情况下进行动态在线重新安排和稳健优化并在之后设计了三种变体来提高 PI 的鲁棒性^[179]，改进是以更多的计算时间为代价的。Mi Yang^[180]提出一种分布式方法来处理在执行原始计划期间发生的动态事件。Rahim Ali Qamar^[181]提出时间约束下的最大任务分配算法。这些方法并没有很好解决动态任务出现前后的任务数量最大化分配。

在多智能体系统中，智能体之间存在协作和竞争的关系，而任务分配涉及到如何平衡这种关系。首先，如何使得每个智能体都能够获得一定的任务量，同时避免出智能体之间现冲突和竞争。其次，多智能体系统需要设计一种任务重分配的分布式算法，使得每个智能体都能够独立地执行任务重分配，并且在全局范围内达到最优解。此外，还需要考虑算法的复杂度和实时性等因素，尽快响应不同的动态任务。因此，多智能体系统在动态环境中实现任务最大化分配时面临着动态任务致使初始分配方案失效的问题，如何在处理动态任务的基础上最大化分配方案给算法给予了更高挑战。

本章针对动态任务分配中的关键问题给出了一种创新的任务分配与重分配框架，

借助设计灵活的成本计算机制和动态子团队协作策略，在实现任务最大化分配的同时保证系统在动态环境中的响应效率与鲁棒性。本文的研究重点是怎样结合动态任务的特性，依靠重新分配已分配任务以及优化未分配任务的处理方式来最大化任务分配的总量，提升多智能体系统的整体效用。

本章在动态环境下提出一个多智能体系统中的最大化任务分配的动态任务分配方法，以便在动态任务出现前后实现任务最大化分配。利用一种新的成本计算机制对分配的任务进行可变成本计算，在原始分配的基础上增加任务分配数量。基于动态任务类型，借助由子团队和任务释放机制构成的动态任务分配算法确定任务响应范围，并借助工作量的概念来平衡子团队的工作负荷并挑选候选智能体。在动态任务分配阶段提出动态任务驱动的任务最大化再分配算法，以在任务列表变化后增多任务分配的数量。

本章在 PI 算法的基础上通过转换成本对已分配任务重新分配，将更多未分配的任务纳入任务列表，提高任务分配数量。在出现动态任务后通过最大化任务分配的动态任务分布式框架应对不同类型的动态任务。通过响应范围来控制子团队规模，考虑动态任务对智能体效用的影响，让子团队成员释放出效用较低的任务对动态任务作出响应，提高对动态任务的响应率。子团队在初始分配的基础上降低通信和计算负担，缩短响应时间。动态任务驱动的最大化再分配方案作为重要组成部分最大化动态环境中分配的任务数量。实验结果显示本章的方法在任务分配成功率上取得了不错的效果，在运行时间中表现优异。

4.2 基于动态任务的任务分配模型构建

本章目标是在考虑任务价值的基础上分配尽可能多的任务，同时根据动态任务的类型对任务进行处理，以此解决动态环境下多智能体系统的任务重分配问题。主要思路是首先基于原始任务分配解设计最大化任务分配机制，在 PI 算法的基础上最大化初始分配方案。然后通过动态任务分配算法来处理动态任务。最后通过任务再分配算法最大化方案来最大化变化的任务列表。机制虽然增加了复杂度，但是可以最大化执行任务的数量，同时处理动态任务，在实际环境中具有重要的意义。

本章将多智能体任务分配过程建模为分布式启发式任务分配^[62]，该过程由元组 $G = \langle A, T, D, V, H, S, c_{i,j}, \tau \rangle$ 组成，其中 $A = [a_1, \dots, a_m]^T$ 是 m 个异构智能体的有限集合。

$T = [t_1, \dots, t_n]^T$, 是有界区域内 n 个任务的有限集合, $m \leq n$ 。 $D = [d_1, d_2, \dots, d_n]^T$ 是任务截止时间的集合, 任务受到时间限制, 需要在截止时间 d 前开始执行。任务分配方案 $V = [v_1, \dots, v_m]^T$ 由所有智能体的任务分配列表组成, 其中 v_i 是分配给智能体 a_i 的任务列表, 并按照分配任务的实际开始时间排序。一个带条目 $h_{i,j}$ 的兼容矩阵 H 用来定义智能体 a_i 是否有能力执行任务 t_j 。 $S = [s_1, s_2, \dots, s_n]^T$ 表示执行任务的持续时间, 执行任务所需的时间可能不同。 $c_{i,j}$ 是智能体 a_i 完成任务 t_j 的时间成本。 τ 表示时间。

任务列表 v_i 的规模是由分配给 a_i 的任务数量所决定的, 鉴于智能体存在性能方面的限制, 其最多可分配 L 项任务, 任务列表属于任务集合 T 的一个子集, 任务仅可被分配给有执行能力的智能体, 智能体的有序任务列表彼此不能重合, 也就是说 $v_i \cap v_j = \emptyset$, 对于 $\forall i \neq j$ 的情况均是如此。智能体可分配多个有兼容类型的任务, 不过一次仅能执行一个任务, 分配给智能体的任务会依照特定的顺序去执行, 来让每项任务都可以在各自规定的截止时间之前完成, 时间成本 $c_{i,j}$ 可尽量低。

本章的研究目标是在已知任务信息和环境信息的基础上获得无冲突解, 最大化总效用, 并优化任务分配过程, 使最多的任务被执行, 任务分配问题可写为:

$$\max \sum_i^N \sum_j^{|v_i|} u_{i,j} \quad (4.1)$$

$$u_{i,j}(c_{i,j}, v_i) = \frac{d_j - c_{i,j}}{d_j} R_j \quad (4.2)$$

$$c_{i,j} = \begin{cases} s_{0j}, & j = 1 \\ \sum_{q=1}^{j-1} c_{i,q} + s_j, & 1 < j \leq |v_i| \end{cases} \quad (4.3)$$

Subject to

$$|v_i| \leq L \leq M, R_j > 0 \quad (4.4)$$

其中, $u_{i,j}$ 表示智能体 a_i 在任务列表 v_i 中执行任务 t_j 的效用, 并采用线性递减的评分策略。 R_j 表示 t_j 对整个任务的重要性, 每项任务的静态值不会降低效用。时间成本 $c_{i,j}$ 根据任务列表 v_i 中的任务执行顺序计算得出, s_{0j} 表示从初始位置到 v_i 中第一个任务所需的时间, 智能体 a_i 执行任务 t_j 的时间成本是任务列表中该任务之前所有任务时间成本的累计和执行任务 t_j 所需时间之和。

本章设计一个最大化任务分配的动态任务分配方法应对动态环境中的动态任务, 并通过考虑最新信息来提供可行的重新分配时间表。使用 PI 算法进行最初的任务分配直到无法再添加任务或达到任务分配上限。如果仍有未分配的任务, 将任务重新分配给其他智能体, 为未分配的任务创造空间。出现动态任务时根据动态任务的类型和动

态任务对任务分配解决方案的影响选择处理策略。考虑不同动态任务之间的差异通过不同策略来响应。由于动态环境更具灵活性,通过子团队机制将动态任务的响应范围进行限制,不指定这些任务是如何实现或由谁实现。

本章给定一个动态环境的元组 $\eta_j = \langle \Omega, \psi, k_j, T_{k_j}, (x_j, y_j), \chi_{i,j}, s_j, d_j \rangle, j = 1, \dots, q$ 来表达动态任务的信息,其中 Ω_u 表示子团队, ψ 表示执行过程中动态任务发生的时间。 k_j 表示发生变化的任务的ID,而 $k_j > n$ 表示引入了新任务, $k_j \leq n$ 表示现有任务 k_j 发生变化。 $T_{k_j} = 0$ 表示将任务 t_j 从任务列表中剔除,不同的大于0的正整数代表不同的任务类型。 (x_j, y_j) 表示任务的坐标, $\chi_{i,j}$ 表示智能体 a_i 和任务 t_j 的距离。 s_j 表示任务 t_j 执行持续时间。 d_j 表示任务 t_j 的截止时间。

本章希望可以在短时间内应对动态环境下中所有类型的动态任务并最大化全局预期收益函数 $u_{i,j}$ 中定义的总利润。从基线PI算法得到的原始任务分配解 $V_0 = [v_1, \dots, v_m]^T$ 开始,智能体 a_i 拥有知道所有智能体的调度以及任务和智能体的原始基本信息及调度信息。所有智能体就获胜智能体列表 $\beta_i = [\beta_{i,1}, \dots, \beta_{i,n}]$, $i = 1, \dots, m$ 达成共识,定义为存储分配给智能体 a_i 本地视图中的任务。

4.3 最大化任务分配与动态任务分配方法

本节描述了所提出的解决最大化任务分配方案与动态任务分配问题的方法,其中最大化任务分配分两个部分,首先是对原始任务分配解 V_0 进行最大化分配,然后是动态任务发生后的二次最大化分配。多智能体系统的目标是在考虑现有分配的同时,消耗更少的计算资源来分配尽可能多的任务,并应对执行任务过程中的动态任务。

4.3.1 最大化任务分配的动态任务分布式框架

为了最大化任务分配数量,并有效处理智能体在执行任务期间可能遭遇到的不同类型的动态任务,本章结合考虑最大化任务分配的目标,力图设计一个能够灵活响应各种动态任务的最大化任务分配分布式系统,以确保最大化任务分配数量的基础上,尽可能响应动态任务。

本章假设只有任务出现动态变化,而智能体的状态维持不变。动态任务覆盖新增任务、删除任务以及现有任务的状态变化这三种类型。新增任务是在系统开启执行任务之后才出现的。当完成初始任务分配后,有可能会出现新的任务,这些新任务需要分

配给智能体去执行。新任务的出现或许是由外部事件触发，也可能是系统内部状态发生变化所导致。删除任务是在系统已经开始执行任务之后被移除的任务。任务被删除可能是因为任务已完成、不再需要执行，或者任务的执行条件发生变化不再适用。现有任务的状态变化是指已在任务列表中的任务/其执行条件或要求出现了改变，这可能包含任务优先级的变化、任务截止时间的变更、任务执行资源需求的调整等情况。

图 4-1 呈现了部分动态任务对原始任务分配解决方案所产生的影响。

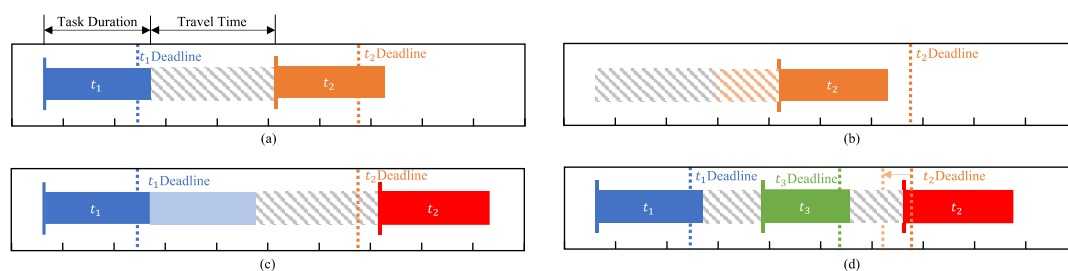


图 4-1 动态事件影响示意图 (a) 初始时间表 (b) 删除任务与移动时间增加 (c) 延长任务时间 (d) 插入新任务与截止时间提前

图 4-2 概述了优化任务分配的过程。智能体在前往执行任务列表中有顺序任务的同时对探测半径内的区域进行检测，动态任务出现的时间和状态信息在检测到之前是未知的。智能体可以实时检测任务信息并处理成统一形式的元组，将检测到的值与元组中的旧值进行比较，区分任务的状态信息，并将改变的任务信息传递给其他智能体。由于单个智能体的检测范围有限，检测到动态任务的智能体和分配更改任务的智能体可以不同。由于智能体在执行任务的过程中会不断移动，在 ψ 时刻出现动态任务时，需要更新智能体的位置。本章将执行中的任务和正在执行任务的智能体排除，智能体任务完成后可以从当前位置参与重新分配。

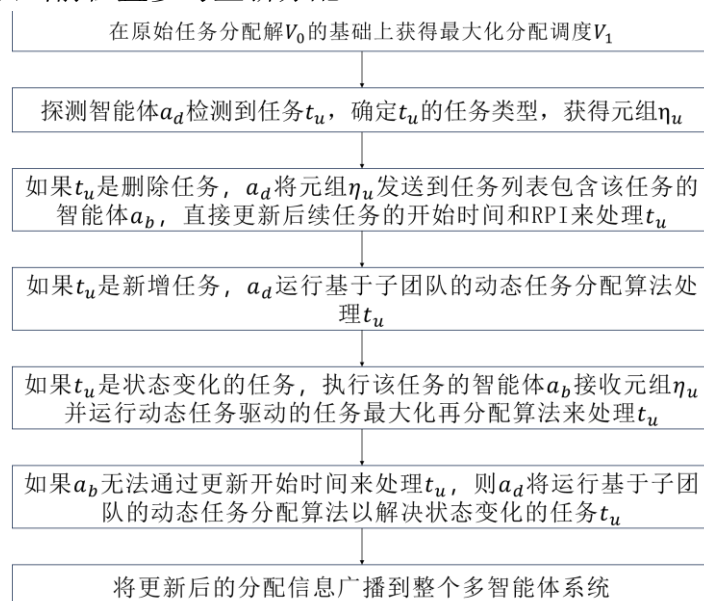


图 4-2 动态任务分配最大化流程图

在本章所阐述的方法里，任务的效用是由多方面因素共同决定的，包括完成任务所能获得的奖励、完成任务所需付出的成本、动态任务自身的类型以及其对于现有方案所产生的影响。当在执行原始分配方案的进程中出现了动态任务时，本章依据任务的不同特点，将其划分为常规任务、高价值任务以及快响应任务这几种类型，而任务的类型会对相应子团队的规模大小产生影响。举例来说，高价值任务往往需要一个规模更大的响应子团队，以便可实现更多方面的协调工作。快速响应任务因为其截止日期相对较早，由更小的响应子团队来负责处理，来提高响应的成功率。当探测到新任务出现时，优先安排单个智能体而非所有智能体去应对动态任务，减少计算以及通信方面的负担，探测智能体还会和邻居节点进行信息共享，并依据子团队机制来形成子团队。

Algorithm 4-1: Dynamic Event Handling Algorithm

Input: a_d, t_u, η_u
Output: v_i

- 1 Run MTAA to maximizing the original task allocation solution
 - 2 Detect the dynamic event type of task t_u and obtain tuple η_u
 - 3 **if** *Dynamic event is a task deletion* **then**
 - 4 Send tuple η_u to agent a_b
 - 5 Detect the agent v_b previously allocated to task t_u
 - 6 **else if** *Dynamic event is a new task* t_u **then**
 - 7 Detect agent a_d and execute the SBDTA to handle task t_u
 - 8 **if** *Dynamic event is a continuously changing task* t_u **then**
 - 9 Agent v_b previously assigned to task t_u receives tuple η_u
 - 10 Run DT-TMRA to handle task t_u
 - 11 **if** *agent v_b cannot handle the task* **then**
 - 12 Detect agent a_d and further execute SBDTA to resolve changes in task t_u
 - 13 Broadcast the updated allocation information to the entire multi-agent system
-

如算法 4-1 所示，当探测智能体 a_d 在检测范围内检测到任务 t_u 时，确定 t_u 的任务类型并获得元组 η_u 。通过比较元组来确定 t_u 的类型，通过算法 4-2 所示的分布式框架基于最大化分配调度 V_1 处理动态任务 t_u 并根据动态任务的类型选择处理策略。当动态任务属于删除任务时， a_d 会把元组 η_u 发送至任务列表中有此任务的智能体 a_b ，直接对后续任务的开始时间以及 RPI 进行更新，以此来处理 t_u 。要是类型为新增任务， a_d 便直接运行 SBDTA 去处理 t_u 。若 t_u 是状态发生变化的任务，那么执行该任务的智能体 a_b 会接收元组 η_u 并运行 DT-TMRA 来处理 t_u 。要是 a_b 没办法借助更新开始时间来处理 t_u ， a_d 就会运行 SBDTA 去解决状态变化的任务 t_u ，更新后的分配信息会被广播至整个多智能体系统。

4.3.2 基于原始任务分配解的最大化分配方案

由于 PI 算法禁止通过牺牲总效用来增加分配数量，导致许多潜在可分配的任务无法被有效分配。为了实现对原始任务分配解 V_0 进行最大化分配的目标，本章提出了一种基于原始任务分配解的最大化分配方案。该方案通过降低已分配任务的成本重新纳入智能体的任务列表，最大程度地分配任务数量。利用 PI 算法进行初始任务分配获得原始任务分配解 V_0 ，未分配的任务通过重新分配给其他智能体，为未分配的任务创造新的分配机会。过程中需要选择重新分配的任务和适用的任务成本计算机制。

算法 4-2 在 PI 算法的基础上计算每个任务的 $\varphi_{i,q}^\wedge$ 找到可包含在 v_i 中的可重分配任务列表 b ，可重分配任务列表中存在已分配任务和未分配任务。在满足所有约束条件后，使用基于分布式蜂群算法的任务选择，通过迭代将任务添加到智能体任务列表中。随着将任务添加到任务列表中，任务的时间成本向量 $c_{i,j}$ 也会更新。在第 i 个智能体的 v_i 中添加任务，直到任务列表已满或无法添加其他任务。达到任务上限或无法再添加任务时，更新智能体 ID 向量 β_i ，得到的初始最大化任务分配解 V_1 。

Algorithm 4-2: Maximizing Task Allocation Algorithm

Input: $w_q^\oplus; w_q^\ominus; v_i; \pi; c_{i,q}$
Output: task list: v_i

```

1 Initialize
2 while  $v_i \leq L$  do
3   Compute  $\varphi_{(i,q)}^\wedge$ 
4   if inclusion of candidate tasks then
5     while exist reassignable tasks do
6       Select the next task  $t_q$  based on SBDTA
7       roulette wheel method
8       if task  $t_q$  can be added to  $v_i$  then
9         Add task  $t_q$  to  $v_i$ 
10        Update time cost vector  $c_{i,j}(v_i \oplus l, t_q)$ 
11      else
12        Update the redistributable task list
13    Update  $c_{i,j}, \beta_i, v_i$ 
14 Update  $\varphi_i^\sim$ 

```

任务 t_q 的融合性能影响（Inclusion Performance Impact, IPI）定义为在任务列表 v_i 中的位置 l 插入任务 t_q 后任务时间成本的总体增加。剔除性能影响（Removal Performance Impact, RPI）是执行移除任务 t_q 的成本度量，较高的 RPI 意味着任务贡献度较高，任务分配较好。如果任务列表中没有任务，则任务 t_q 的 IPI 等于时间成本。智能体 a_i 的任务列表 v_i 中任务 t_q 的 IPI 计算公式为：

$$w_{i,q}^{\oplus} = \min_{1 \leq i \leq |v_i|+1} \left[\sum_{j=l}^{|v_i|+1} c_{i,j}(v_i \oplus l, t_q) - \sum_{j=l}^{|v_i|} c_{i,j}(v_i) \right] \quad (4.5)$$

其中, $v_i \oplus l, t_q$ 表示在智能体 a_i 的任务列表 v_i 的 l 处插入任务 t_q 。任务的 IPI 存储在 IPI 向量 $\varphi_i^{\oplus}: w_1^{\oplus}, w_2^{\oplus}, \dots, w_q^{\oplus}, \dots, w_m^{\oplus}$ 。在任务列表已满或无法再添加任务后, 智能体将计算从任务列表 v_i 中移除任务 t_q 的 RPI:

$$w_{i,q}^{\ominus} = \sum_{j=l}^{|v_i|} c_{i,j}(v_i) - \sum_{j=l+1}^{|v_i|} c_{i,j}(v_i \oplus t_q) \quad (4.6)$$

本章将 $w_{i,q}^{\oplus}$ 转换为 $\varphi_{i,q}^{\wedge}$, $\varphi_{i,q}^{\wedge}$ 是任务对性能的影响, 但对未分配任务和已分配任务的计算方法不同, 计算方法如下:

$$\varphi_{i,q}^{\wedge} = \begin{cases} \wp_{i,q} - w_{i,q}^{\oplus} / \wp_{i,q}, & \text{if task is already assigned} \\ w_{i,q}^{\oplus}, & \text{if task is unassigned} \end{cases} \quad (4.7)$$

未分配任务的 $w_{i,q}^{\oplus}$ 和 $\varphi_{i,q}^{\wedge}$ 值相同, 已分配任务的 $\varphi_{i,q}^{\wedge}$ 低于任务的 $w_{i,q}^{\oplus}$ 。在通信和冲突解决阶段也要计算转换后的 RPI, 即 $\varphi_i^{\sim}$:

$$\varphi_{i,q}^{\sim} = \begin{cases} \wp_{i,q} - w_{i,q}^{\ominus} / \wp_{i,q}, & \text{if task is already assigned} \\ w_{i,q}^{\ominus}, & \text{if task is unassigned} \end{cases} \quad (4.8)$$

未分配任务的 $w_{i,q}^{\ominus}$ 和 $\varphi_{i,q}^{\sim}$ 值相同, 而已分配任务的 $\varphi_{i,q}^{\sim}$ 低于任务的 $w_{i,q}^{\ominus}$, $\varphi_{i,q}^{\sim}$ 低于 $w_{i,q}^{\ominus}$ 值会导致任务分配给 $\varphi_{i,q}^{\sim}$ 最低的智能体。通过方式创建空缺以包括未分配的任务。

任务选择基于效用概念, 采用分布式蜜蜂算法, 同时考虑了任务的成本和优先级。从可重分配任务列表中选择任务 t_q 的概率为:

$$P(b_i, t_q) = \xi_{i,q}^{\alpha} \lambda_{i,q}^{\omega} / \sum_{j=1}^{|v_i|} \xi_{i,j}^{\alpha} \lambda_{i,j}^{\omega} \quad (4.9)$$

其中, $|v_i|$ 是任务列表 v_i 中包含的任务数, $\lambda_{i,q} = 1 / \varphi_{i,q}^{\wedge}$ 。任务优先级 $\xi_{i,q} = (d_{i,q} - \tau) / d_{i,q}$, 与任务截止时间相关联, 任务的截止时间越近, 优先级就越高。已分配和未分配的任务可能会返回不同的结果。任务选择使用的是概率函数调整 α 和 ω 使未分配的任务具有更高的概率。这样, 经过几次任务重新分配迭代后, 分配了尽可能多的任务给智能体。

4.3.3 基于子团队的动态任务分配算法

为了使多智能体系统可以重新分配动态任务, 本章提出一种基于子团队的动态任务分配算法, 包含增量子团队和任务释放机制, 在原始分配方案的基础上牺牲部分求解质量, 快速进行任务重新分配并减轻通信和计算负担。

本章通过子团队机制让部分智能体和任务参与重新分配过程，并根据任务释放机制让子团队成员优先释放部分对总分贡献较低的任务，通过权衡协调与收敛速度响应动态任务，提高任务重新分配的成功率。智能体 a_d 探测到任务 t_u 后，以智能体 a_d 为中心，在 a_d 的通信范围内寻找可以加入子团队的智能体，如果存在可以加入子团队的智能体，那么子团队通信范围可以进一步扩大。子团队半径内的智能体组成子团队 Ω_u ，其他智能体不参与重新分配。如果通信范围内不存在可以加入子团队的智能体，优先由智能体 a_d 处理改任务。子团队内部根据效用决定是否插入新任务及执行新任务的智能体。根据任务释放机制可以删除现有分配方案中的部分任务，在尽可能小的影响总体效用的基础上，将新任务插入任务列表。

子团队半径 r_u 受到任务价值、任务紧急程度和任务执行时间等因素的影响。增量比 κ 控制增量的频率，每次增量的范围不大于智能体的最大通信范围，逐步增加子团队半径 r_u ，找到能恰好分配动态任务的最小子团队半径 r_u （此时子团队半径可大于智能体最大通信范围）。随着子团队半径的增大，更多的智能体被纳入子团队参与重新分配，一旦分配动态任务 t_u 的无冲突解决方案出现，重分配过程终止并输出可行解及其对应的响应范围，但最终响应范围应在动态任务的最大响应范围内：

$$r_u = \kappa * CR_{max} \left(\zeta_1 \frac{R_u}{R_{max}} + \zeta_2 \frac{d_u - \psi}{d_{max} - \psi} + \zeta_3 \frac{s_u}{S_{max}} \right) \quad (4.10)$$

其中， CR_{max} 指的是智能体的最大通信范围， ζ_1 、 ζ_2 、 ζ_3 作为权重，用于平衡任务价值、任务紧急程度以及任务执行时间对单位半径所产生的影响。本章引入了工作量来衡量智能体的工作负荷情况。对于智能体 a_i 而言，其工作量 W_i 是由任务数量以及所有任务的执行时间共同确定的。在挑选候选智能体的时候，本章不仅要考虑它们与动态任务之间的距离，也要考虑它们的工作量：

$$\min_{1 \leq i \leq |\Omega_u|} [\delta_1 W_i + \delta_2 \chi_{i,j}] \quad (4.11)$$

$$W_i = \sigma |v_i| \sum_{i=1}^{|v_i|} s_i \quad (4.12)$$

对于动态任务 t_u ，本章定义了元组 $\theta_u = (t_u, \Omega_u)$ 来存储动态任务及其对应的子团队，使用集合 T_{rel} 来存储 Ω_u 中所有被释放的任务及未分配的动态任务。对于智能体 $a_i \in \Omega_u$ ，将任务列表 v_i 中 RPI 值最小的任务释放，更新智能体的工作量以反映任务的删除。子团队 Ω_u 中的智能体 a_j 优先释放原计划任务列表中 RPI 最低的 $n_{j,rel}$ 个任务：

$$n_{j,rel} = |a_j| - \mu \frac{\sum_{i=1}^{|\Omega_u|} |v_i|}{|\Omega_u|} \quad (4.13)$$

其中， μ 用于限制任务发布的比例。根据智能体的工作量和子团队的平均工作量来选

择任务。根据工作量确定删除任务的数量可以进一步平衡子团队 Ω_u 中智能体的工作量。

Algorithm 4-3: Subteam-Based Dynamic Task Allocation Algorithm

Input: $\kappa; V_1$
Output: v_i

- 1 Initialize the increment ratio: $\kappa \leftarrow \kappa_0$
- 2 **while** *not reaching termination conditions* **do**
- 3 $V_r \leftarrow V_1$
- 4 a_d determines a subteam Ω_u for t_u by r_u
- 5 Initialize $T_{rel} \leftarrow t_u$
- 6 **for** *agent a_j in Ω_u* **do**
- 7 Remove tasks through task releasing mechanism
- 8 Update β_j and γ_j
- 9 Communicate with agent in Ω_u
- 10 Resolve conflicts and remove conflicting tasks
- 11 Reach consensus on γ_j and β_j
- 12 Store highest RPI value for deleted tasks $\omega_{i,u}^\ominus$
- 13 Remove changed tasks with lower RPI values
- 14 Check the updated winning agent ID $\beta_{i,u}$
- 15 **if** *winning agent is itself* **then**
- 16 Add tasks
- 17 **else**
- 18 Remove tasks from the task list
- 19 Execute DT-TMRA
- 20 **while** *task exchanges and inclusions* **do**
- 21 Trigger new communication
- 22 Increase the increment ratio κ

SBDTA 方法见算法 4-3。首先, 增量比率从初始增量比率 κ_0 开始。如果变更后的任务尚未分配, 则将当前方案设为未发生动态任务的初始最大化任务分配方案。然后, 探测智能体 a_d 根据子团队机制确定组成响应子团队 Ω_u 的子团队成员, Ω_u 中的智能体通过任务释放机制选择并移除部分任务。 Ω_u 中智能体相互通信, 交换判定信息并更新获胜智能体 ID 向量 β_j 、获胜 RPI 向量 γ_j 和时间戳 (确定最新的信息), 以解决冲突。完成子团队内部的通信后, 智能体就 T_{rel} 中任务的更新获胜 RPI 向量 γ_j 和获胜智能体 ID 向量 β_j 达成一致。然后, 智能体删除冲突任务, $\omega_{i,u}^\ominus$ 存储子团队 Ω_u 中删除 t_u 的智能体的最高 RPI 值, 智能体从任务列表中删除 RPI 值较低的已更改任务 t_u , 每个智能体检查更新后的获胜智能体 ID $\beta_{i,u}$ 。如果获胜智能体是自己, 则智能体添加任务, 否则从任务列表中删除 t_u 。至此, 子团队 Ω_u 就更改后的任务 t_u 达成共识。然后执行 DT-TMRA, 进一步将更多任务纳入任务列表, 直至无法再纳入更多任务。所有智能体反复执行两个阶段后, 再次触发新的通信。当没有更多任务可以交换和包含时, 重新分

配结束，从而使总效用最大化。所有子团队成员通过迭代任务最大化再分配算法和通信重新分配 T_{rel} 中的任务，直到获得无冲突解决方案。增量比 κ 不断增加，直到 t_u 被分配或响应半径超过任务范围，SBDTA 流程终止。

4.3.4 动态任务驱动的任务最大化再分配算法

对动态任务做出响应后，智能体的任务列表会出现变化，原先的最大化任务分配方案或许不再有效。基于此，本节给出一个动态任务驱动的最大化再分配方案，此方案能实现任务分配的最大化。

Algorithm 4-4: Dynamic Event Task Reassignment Algorithm

Input: T_{rel}, t_u, η_u

Output: v_i

- 1 Initialize sets: t_u^{old}, t_u^{new}
 - 2 Select agent a_b based on the task handling strategy
 - 3 Remove t_u^{old} from task list v_b
 - 4 Update start times and RPI for tasks after t_u^{old} in v_b
 - 5 **for** each $t_k \in t_u^{old}$ **do**
 - 6 Update task information in v_b
 - 7 **for** each $t_k \in t_u^{new}$ **do**
 - 8 Calculate the impact of inserting t_k at each position l
 - 9 Find the position l with the maximum impact
 - 10 **if** $l \neq \emptyset$ and $\omega_{b,u_{new}}^{\oplus} > 0$ **then**
 - 11 Insert t_k at position l in $v_b^{\ominus u_{old}}$
 - 12 **else if** $\omega_{b,u_{new}}^{\oplus} < 0$ **then**
 - 13 Insert t_k at the position with minimal impact on local utility in $v_b^{\ominus u_{old}}$
 - 14 Set the RPI of t_k^{new} as the IPI at insertion: $\omega_{b,u_{new}}^{\ominus} = \omega_{b,u_{new}}^{\oplus}$
 - 15 Update start time and RPI of subsequent tasks in v_b
 - 16 Store local performance vectors for v_b
 - 17 **Return**
 - 18 Updated task assignment and local performance vectors
 - 19 Update the RPI vector $\gamma_i = [\omega_{i,1}^{\ominus}, \dots, \omega_{i,m_i}^{\ominus}]$
-

如算法 4-4 所示，鉴于特定任务状态发生改变，定义两个集合管理 T_{rel} 中的任务： t_u^{old} 用来管理未改变的任务， t_u^{new} 用来管理已更新的任务。依据任务处理策略，挑选智能体 a_b 接收与任务 t_u 相关的修改元组 η_u 。接着借助任务释放机制把 t_u^{old} 从任务列表 v_b 中去除，同时对后续任务的开始时间和 RPI 作出调整。在确定于任务列表中插入 t_u^{new} 的位置时，会找出对本地性能影响最大的位置 l ，这个位置对应最佳 IPI。任务列表 v_i 中任务 t_k 的 IPI 是：

$$\omega_{i,k}^{\oplus} = \max_{1 \leq l \leq |v_i|+1} \left\{ u_{i,k}(v_i \oplus l, t_k) + \sum_{r=l}^{|v_i|} [u_{i,r+1}(v_i \oplus l, t_k) - u_{i,r}(v_i)] \right\} \quad (4.14)$$

如果在 $v_b^{\ominus u_{old}}$ 中没有合适的位置 l 来插入 t_u^{new} , 或者插入它违反了时间限制, a_b 无法处理任务 t_u , 本章就设置 $l = \emptyset$ 。如果存在一个有效的位置, 并且 $\omega_{b,u_{new}}^{\oplus} > 0$, 则插入会提高智能体的效用, 那么 $l \neq \emptyset$ 。但是, 如果 $\omega_{b,u_{new}}^{\oplus} < 0$, 则插入会降低效用。在情况下, 为了迅速处理任务变化, 本章允许牺牲一些效用, 并在对本地效用影响最小的位置插入任务。将任务 t_u^{new} 插入到更新后的任务列表 $v_b^{\ominus u_{old}}$, 无论 IPI 值是正还是负, 本文将新任务 t_u^{new} 的 RPI 设置为插入时的 IPI:

$$\omega_{b,u_{new}}^{\ominus} = \omega_{b,u_{new}}^{\oplus} \quad (4.15)$$

然后, 列表中的后续任务将根据新的开始时间和 RPI 通过以下公式进行更新:

$$\omega_{i,k}^{\ominus} = u_{i,b}(v_i) + \sum_{r=b+1}^{|v_i|} [u_{i,r}(v_i) - u_{i,r-1}(v_i \ominus l, t_k)] \quad (4.16)$$

智能体 a_i 任务列表中任务 t_k 的 RPI 保存在获胜 RPI 向量 $\gamma_i = [\omega_{i,1}^{\ominus}, \dots, \omega_{i,m_t}^{\ominus}]$ 中。RPI 的计算方法是执行被移除任务与后续任务之间的得分差异。尽管从任务列表中删除任务会导致智能体在执行已删除任务时的局部得分下降, 但由于任务已删除, 后续任务的时间成本提前, 后续任务的本地得分将提高。如式(4.14)所述, 从 v_i 中移除任务 t_k 的 RPI 反映了执行移除任务的损失和推进后续任务的收益。计算囊括了任务对智能体总体得分的贡献及其对本地效用的影响。

4.4 实验结果与分析

4.4.1 场景和仿真设置

采用虚拟的研究场景^[178]评估所提方法性能, 实验覆盖两类异构无人飞行器, 即侦察无人飞行器与攻击无人飞行器, 侦察无人飞行器以每秒 30 米的恒定速度运行, 攻击无人飞行器则以每秒 50 米的速度移动, 且在整个实验过程中速度保持稳定不变, 任务同样分为两类, 一类是只能由侦察无人机执行的侦察任务, 另一类是只能由攻击无人机执行的攻击任务。本文假定任务和无人机在这两类任务中平均分配, 任务于跨度为 10000 米 $\times$ 10000 米 $\times$ 1000 米的三维空间里随机生成, 无人机则随机放置在相应的二维地平面上, 完成侦察任务的时间设定为 300 秒, 而攻击任务所需时间为 350 秒。

借助改变无人飞行器和任务的数量来构建不同测试条件,利用不同种子值确定无人飞行器和任务的位置以及任务截止日期。案例配置是任意的,对算法功能并不关键,本文设计了两种情况开展一系列实验,用以验证其对每种组合的有效性,场景 1 涉及任务的添加与删除,场景 2 包括在添加任务的同时改变现有任务的状态,另外本文使用不同种子生成了三种情况,每种动态任务由五个任务构成,其特征参数各不相同。针对每一个处于不断变化状态的任务,本章借助 20s、500s、1000s 以及 1500s 这些出现时间值,以此来引入该任务的不同阶段,在测试的进程当中,每项任务的截止时间都是于特定的时间范围之内进行随机分配的,并且依据不同的任务与智能体比例,设定了各不相同的最大截止时间 (s_{max})。为了避免智能体的时间安排出现过于紧凑的情况,所有任务的最长截止时间都被设定为 2000s,任务持续时间提前 150s、300s 以及 600s,任务持续时间延长 30s、60s 以及 120s,这分别代表着三种动态情形,基线是依靠使用所提供的代码来给予实现的,超参数在实验过程中进行了相应的微调。所有方法均使用 MATLAB R2020a 实现,并在 Intel i7-12700F CPU, NVIDIA RTX 3070Ti GPU 上运行。表 4-1 概述了所测试的无人机和任务的 10 种不同组合。

表 4-1 无人机和任务的不同组合

No.	1	2	3	4	5	6	7	8	9	10
m	6	8	10	12	14	6	8	10	12	14
n	18	24	30	36	42	36	48	60	72	84

4.4.2 实验结果对比分析

本章分析了 PI 算法和本文方法在不同场景下不同组合的任务分配成功率和成本。从运行时间和解决方案质量对本文提出方法与 CBBA、基线 PI、带重新分配版本的 PI (PI-Whitbrook) 和动态分组分配 (DGA-Chen) 的性能进行了比较分析。CBBA 和基线 PI 在引入动态事件时缺乏重新分配模块,在任何情况下都无法配置新的解决方案。PI-Whitbrook 可以添加和删除新任务。DGA-Chen 没有关注动态事件的类型,利用重新运行策略对没有重新分配模块的方法进行实验。

图 4-3 展示了在不同案例中 PI-Original 与本章方法的平均任务分配成功率随组合 (No.) 变化的表现。图 4-3 (a) 展示了 PI-Original 与 Proposed 方法在场景 1 中的平均任务分配成功率。数据表明组合从 1 到 10 逐渐增大时,两种方法的平均任务分配成功率均呈现一定程度的下降趋势。在场景 1 中 1-5 号 PI 算法的平均成功率为 83%, 6-10 号 PI 算法的平均成功率为 78%, 该算法在不同组合下能够维持一定的任务分配能力,在复杂环境下仍存在局限性,难以进一步提升分配成功率。本文提出的方法在整个范

围内表现显著优于 PI-Original，对于编号 1-5 本文方法实现了 88% 的平均成功率，6-10 号本文方法实现了 83% 的平均成功率。在场景 1 中本文方法在任务分配成功率上始终优于 PI-Original，在组合编号较大的情况下本文方法优势更加明显，成功率比 PI-Original 高出约 5%。Proposed 方法中动态任务重分配机制和任务释放策略在任务数量增加时能够更有效地分配和优化资源利用。

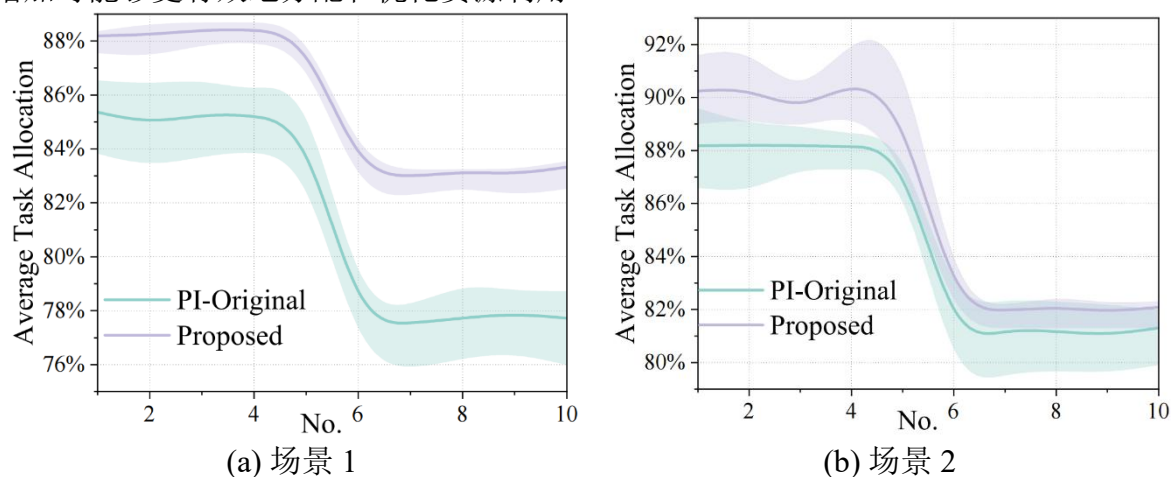


图 4-3 不同组合下的任务分配成功率

图 4-3(b)呈现出 PI - Original 与 Proposed 方法于场景 2 中的任务分配成功率情况，在场景 2 里，PI 原始算法的任务分配成功率相较于场景 1 略高一些，其中 1-5 号 PI 算法的平均成功率为 88%，6-10 号 PI 算法的平均成功率为 81%。PI 算法在场景 2 这种任务类型下，依旧存在分配效率方面的限制。Proposed 方法在场景 2 中，1-5 号实现了 90% 的平均成功率，6-10 号实现了 82% 的平均成功率，Proposed 方法在不同场景下可适应不同组合，并且维持着较高的任务分配成功率。在场景 2 中，Proposed 方法的分配成功率一直高于 PI-Original，在任务编号较大时，Proposed 方法相较于 PI-Original 平均高出约 4% 的成功率，这证实了动态任务处理机制的有效性。

两种方法在场景 1 和场景 2 中的整体表现趋势相近，Proposed 方法在这两个场景中都呈现出更高的分配成功率。不管是场景 1 还是场景 2，Proposed 方法始终体现出更强的动态适应能力。随着组合的改变，Proposed 方法的分配成功率提升幅度比 PI-Original 略高，在任务负载增加时可分配资源，使任务完成数量达到最大化。本文所提出的动态任务释放机制以及灵活的任务成本调整，有效地优化了任务分配过程，克服了 PI - Original 在处理复杂任务时的局限性，这种改进在任务复杂性和环境动态性增加时表现得非常明显。从两个场景的实验结果可得出，Proposed 方法在动态任务分配中的表现优于传统的 PI 原始算法，其核心优势体现在对任务类型和环境动态变化的敏感性以及在高任务负载下的资源优化能力。这些实验结果验证了 Proposed 方法的有效

性和鲁棒性，为多智能体系统在动态环境中的应用提供了更优质的解决方案。

图 4-4 呈现出 PI 原始算法以及本章所提及方法在不同组合情形下的成本变动状况，其中平均成本指的是在任务分配进程里智能体完成任务所需耗费的时间或者资源量。图 4-4(a)展示了场景 1 中这两种方法的任务分配平均成本情况，PI-Original 方法在组合产生变化时，其平均成本范围大致处于约 150 至约 180 之间。这种平稳的增长态势体现出 PI 算法以让系统总时间成本达到最小为目的的策略，当任务数量增多时可有效地维持成本的相对稳定状态，本文所阐述的方法在组合发生变化时平均成本高于 PI-Original，成本从约 200 至约 220 呈现出同样的稳定趋势。本文方法平均成本较高，是因为该方法的任务分配策略核心以便使任务分配数量最大化而非让多智能体系统的时间成本最小化。Proposed 方法在所有组合条件下的成本都高于 PI-Original，最大成本差距将近 10%，这意味着 PI 算法在时间成本优化方面有更较大的优势，本文方法由于优先考虑任务分配的最大化导致了更高的时间消耗。

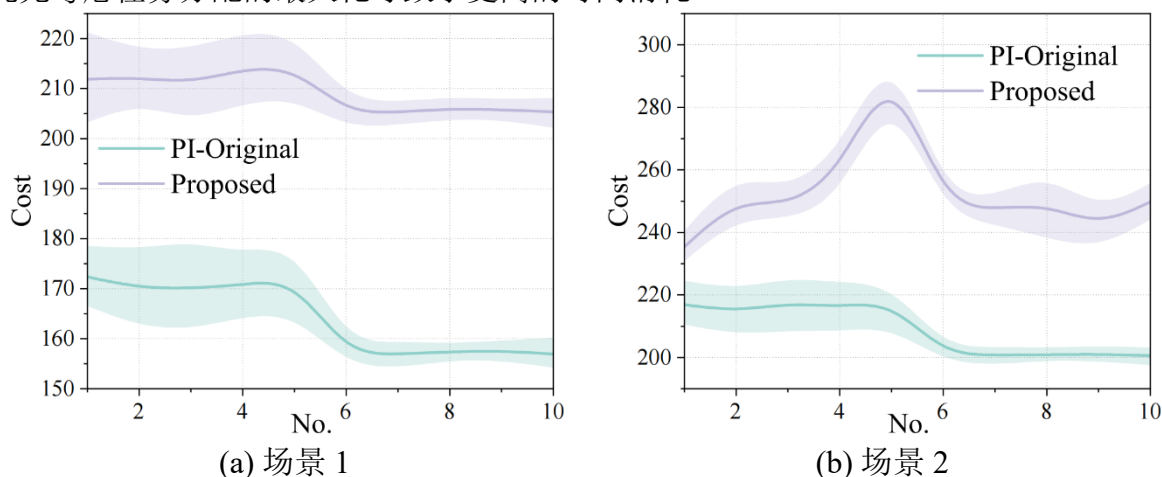


图 4-4 不同组合下的相关成本

图 4-4(b)呈现出了场景 2 里两种方法的任务分配平均成本情况，场景 2 有着更为复杂的任务动态以及资源需求，PI-Original 方法的平均成本处在大约 200 至约 220 这个范围。在任务复杂性更高的场景当中，PI 算法的时间成本优化策略依旧可对整体增长加以控制，成本增长幅度相较于场景 1 略高一些。Proposed 方法的平均成本从约 230 至接近 290，Proposed 方法在场景 2 中的成本更高，场景 2 中的任务分配要求更为复杂，使得在资源分配以及任务最大化过程里消耗了更多时间，场景 2 中 PI-Original 的平均成本依旧低于 Proposed 方法，两者之间的成本差距有所扩大。虽然 PI 算法在时间成本优化方面仍然有优势，不过在复杂场景中其优化能力的提升相对有限，Proposed 方法在复杂场景中优先分配更多任务的策略使得时间消耗增加。

PI 算法把使全局时间成本达到最小当作目标，在各类任务动态里呈现出比较高的

效率，这种策略的局限之处在于存在牺牲任务分配数量的可能性，对系统整体任务完成能力造成了限制，随着组合的改变，PI-Original 和 Proposed 方法的平均成本都出现了上升态势，Proposed 方法的上升幅度更大，Proposed 方法在任务数量增多时更侧重于让任务分配数量实现最大化，而不是对时间成本进行优化，PI-Original 方法在时间成本方面有优势，不过其分配策略或许会让部分任务没能得到有效分配。Proposed 方法优先考虑任务分配的全面性，提高了系统的任务完成能力以及资源利用率。

图 4-4 的实验结果说明 PI-Original 和 Proposed 方法在任务分配平均成本方面存在差别，PI-Original 凭借时间成本最小化策略达成了更低的平均成本，Proposed 方法因为优先让任务分配最大化而致使时间成本更高。这两种方法在性能上体现出不同的权衡：前者适用于对时间敏感的场景，后者更适合任务覆盖率以及系统效用优先的场景，与本文的方法相比，PI 算法的任务分配平均成本较低，平均成本的上升归因于 PI 算法依据整个多智能体系统的时间成本最小化来确定任务的优先级，本文方法优先考虑的是任务分配的最大化，并非整个多智能体系统时间成本的最小化，最终导致了平均时间成本的增加。

图 4-5 展示了在场景 1 和场景 2 中，不同方法在处理动态任务信息和计算最终解决方案所需时间（运行时间）的对比。运行时间表示处理动态任务信息的计算时间，以及最后一个智能体获得最终收敛解决方案所需的时间。对比的算法包括 PI 原始算法（PI-Original）、PI 改进算法（PI-Whitbrook）、动态分组分配算法（DGA）、基于共识的捆绑算法（CBBA）以及本文提出的方法（Proposed）。

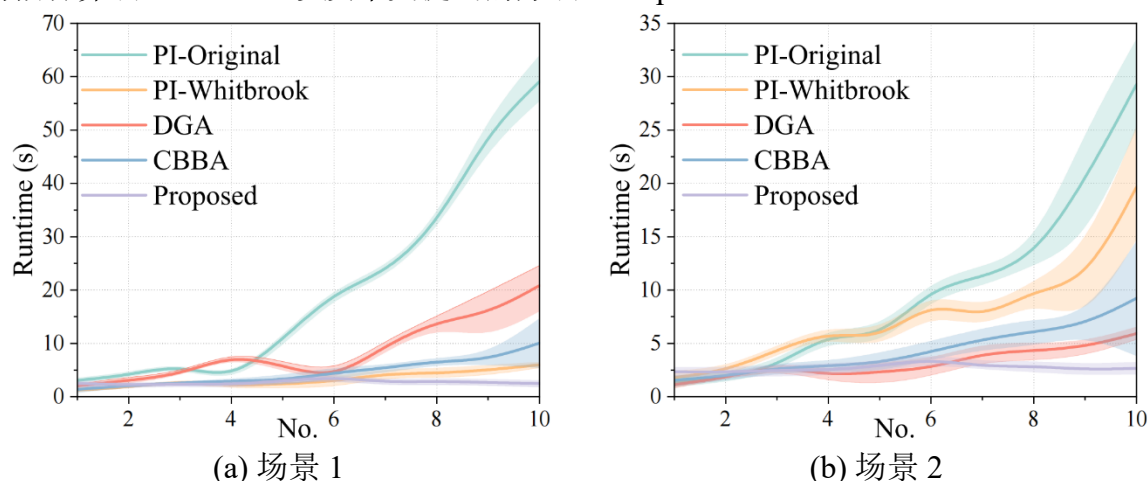


图 4-5 不同组合下的运行时间

图 4-5(a)呈现出在场景 1 里不同组合情形下五种方法的运行时间状况，PI 原始算法的运行时间在所有组合当中都是最高的，开始的时候大概是 3 秒，并且随着组合编号的增加逐渐攀升至接近 60 秒，这种方法在任务数量增多之际，由于全局搜索所带来

的计算复杂程度过高, 很难契合动态环境的实时性需求。PI-Whitbrook 的运行时间有所下降, 随着组合编号的增长从大约 2 秒增加到 5 秒左右, 该算法对任务分配过程进行了部分改进优化, 不过在任务量较大的时候依然存在一定的计算瓶颈。DGA 的运行时间相较于 PI 方法降低, 在组合编号为 1 的时候大约是 2 秒, 在组合编号为 10 的时候上升至大约 20 秒。分组机制在一定程度上减少了全局计算需求, 然而在较为复杂的场景下仍旧需要更多的通信和协调时间。CBBA 的运行时间处于 DGA 和 PI - Whitbrook 之间, 初始的时候为大约 2 秒, 组合编号增大之后增长至大约 10 秒, 任务捆绑和共识机制在动态任务场景中, 当任务数量较大时需要更多的协调和分配时间。Proposed 方法在场景 1 中的运行时间一直保持最低, 初始的时候运行时间大约为 2 秒, 组合编号增加到 10 时也仅仅增加到大约 2 秒, 本文方法的动态子团队机制和任务释放策略减少了通信和计算开销, 达成了高效的任务分配。

图 4-5(b)中, PI-Original 的运行时间显著高于他方法, 从编号 2 的约 5 秒增加至任务编号 10 的超过 30 秒。PI 原始算法在复杂动态场景中的计算复杂度过高难以适应动态环境的实时性要求。PI-Whitbrook 的运行时间略低于 PI-Original, 从约 3 秒增加至 20 秒左右。尽管优化减少了一部分计算开销在任务复杂性增加的情况下性能提升有限。DGA 在场景 2 中的运行时间仍保持较低水平, 初始时约为 3 秒后上升至约 5 秒。与场景 1 相比性能略有提升。CBBA 在场景 2 中的运行时间表现相似, 初始时约为 2 秒后上升至约 10 秒, 任务捆绑机制对动态任务分配具有一定的适应性, 通信和协调需求限制了运行效率的进一步提升。Proposed 方法在场景 2 中的运行时间依然最低, 运行时间仅从约 2.5 秒增加至约 2.5 秒, 在处理动态任务分配时能够快速响应并生成收敛的解决方案。

基线 PI 算法收敛到无冲突解决方案所需的运行时间最长。PI-Whitbrook 和 CBBA 有所改进但运行时间仍较高。DGA 将已更改的任务分配给聚类获得的小组成员表现出较低的运行时间, 将先前分配给其他小组成员的任务纳入其中需要更多的计算和通信。Proposed 方法在所有编号和场景中始终保持最低的运行时间, 由于增量子团队的形成机制, 只有有限数量的无人机参与重新分配, 只有一小部分任务从原计划任务列表中释放出来重新分配。因此, 子团队 Ω_u 中的智能体只在整个子团队内传送已释放任务的分配信息。动态子团队分配和任务释放机制有效减少了通信和计算需求, 与其他四种算法相比 Proposed 方法收敛到无冲突解决方案所需的运行时间要少得多。Proposed 方法中子团队的大小是由增量比、静态值和不断变化的任务截止时间决定的, 计算过程不受问题规模的影响, 运行时间不像上述三种算法呈现出明显的变化。无论是在任务

复杂性较低的场景 1 还是任务复杂性较高的场景 2，Proposed 方法始终展现出高效性和稳定性。

图 4-6 展示了五种算法（PI-Original、PI-Whitbrook、DGA、CBBA 和 Proposed）在不同案例中不同组合下解决方案质量。本章将基准 PI 算法的解质量作为参考，给出了每种算法的得分与基准得分之间的偏差。

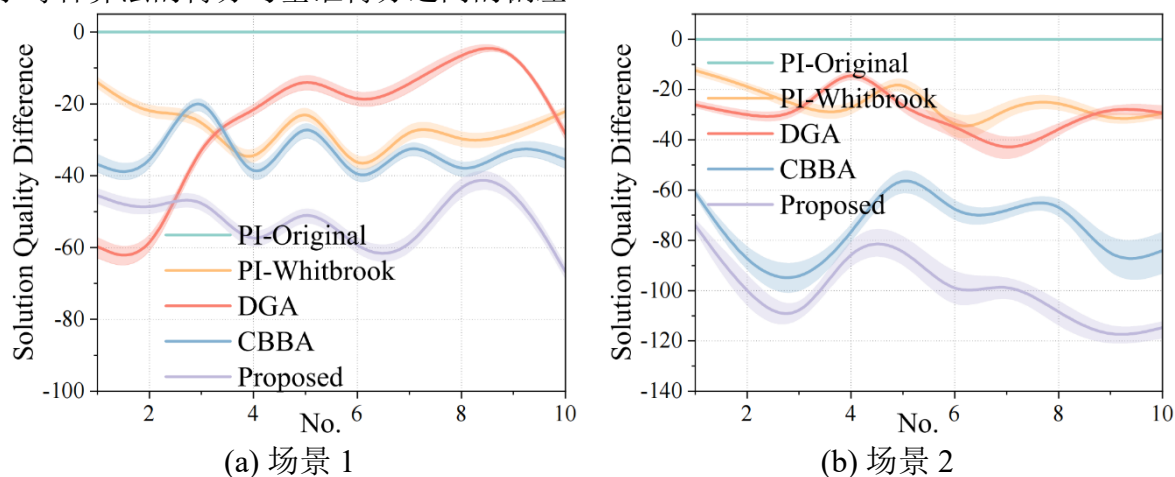


图 4-6 不同组合下的解决方案质量

图 4-6(a)呈现出了在场景 1 里五种算法解决方案质量的偏差情况，PI-Original 作为基准算法，其解决方案质量偏差一直为 0，该算法的解质量得益于全局优化策略，可在多次运行当中保持稳定，提供最高的得分，不过高计算复杂度对动态场景的适用性产生了影响。PI-Whitbrook 的解决方案质量偏差较为稳定，处于-20 到-40 之间。此算法对 PI-Original 的优化有所简化，虽然计算效率有所提升，但在解决方案质量方面稍显逊色。DGA 的解决方案质量得分相对于基准的偏差略大一些，大约在-10 至-60 之间，在处理动态任务时，可能存在因分组策略导致的资源利用效率不足的情况，这对整体解决方案的得分造成了影响。CBBA 的偏差在-20 到-40 之间波动，任务捆绑策略在一定程度上提高了解决方案质量。Proposed 方法在场景 1 中的解决方案质量偏差最大，大概在-50 至-60 之间，由于 Proposed 方法优先考虑任务分配的最大化非全局得分的最优化，仅有部分智能体参与了重新分配过程。Proposed 方法的设计目标并非追求得分最优，而是借助提升任务覆盖率来实现更高的任务完成率，偏差降低是可以接受的。

图 4-6(b)展示出了场景 2 中五种算法解决方案质量的偏差，PI-Original 在场景 2 中的解决方案质量得分维持为 0，PI-Whitbrook 在场景 2 中的偏差处于-20 至-40 之间，解决方案质量逐渐降低。在更为复杂的场景中，PI-Whitbrook 的简化策略致使优化效果受限。DGA 在场景 2 中的偏差范围约为-20 至-40，DGA 分组策略在任务复杂性增加时，在多智能体协调过程中的效率较低。CBBA 的偏差在场景 2 中约为-60 至-100 之

间，在复杂场景中，协调成本使得得分下降。**Proposed** 方法在场景 2 中的偏差范围约为-80 至-110，表现比场景 1 要差，由于场景 2 中的任务复杂性更高，导致部分智能体的重新分配过程未能优化得分。与其他算法相比，**Proposed** 方法的解决方案质量表现仍在可接受范围内，其设计目标也不是得分最优，而是最大化任务分配数量。

不管是在场景 1 还是场景 2 中，多次运行基准 PI 算法始终可获得最高的解决方案质量，全局优化策略在动态任务分配中有有效性，PI-Whitbrook、CBBA 和 DGA 在效率与质量之间达成了一定平衡，**Proposed** 方法尽管在解决方案质量上的表现不如其他算法，但在任务覆盖率最大化方面呈现出了独特优势。性能取舍使得在注重任务分配数量和系统灵活性的场景中有一定特点。

4.5.本章小结

本章提出了一种多智能体系统动态任务分配方法，该方法可应对动态环境的复杂性，以便实现任务分配数量的最大化，同时还引入了一种全新的成本计算机制，这种机制可允许进行可变成本计算，实现原始任务分配方案的最大化。针对动态任务专门设计了基于子团队的动态任务分配算法，在考虑任务类型、子团队以及任务释放机制等因素的基础上，达成了工作负载的平衡。另外还提出了一个动态任务驱动的任务最大化再分配方案，当任务列表出现变化之后，继续维持任务分配数量的最大化。此方法在不同案例之下呈现出了优异的性能，在某些情形下甚至超越了传统方法。仿真结果显示，该方法在多智能体系统里处理动态任务时性能出色，有较高的任务分配成功率、合理的成本、较短的运行时间，以及令人满意的解质量。

第五章 基于均衡性的多智能体分布式任务分配方法

5.1 引言

多智能体系统的性能会在一定程度上受到任务分配方式的影响,智能体消耗的能量覆盖旅行过程中所消耗的能量以及执行任务期间所消耗的能量,每执行一项任务都要消耗一定量的能量,当能量耗尽后智能体就会失效,怎样有效地分配任务以便最大程度地发挥系统的整体效率,保证智能体之间的任务负载处于均衡状态,这是一个颇具挑战性的问题。在多智能体系统里任务分配一般会关注整体效率,然而有可能忽略个体智能体的负载均衡情况,这或许会致使一些智能体负担过重,而其他智能体出现资源浪费现象,需要一个高效的负载平衡方案来有效利用这类系统。

多智能体系统中的负载均衡问题指的是一组独立的智能体共同分享一组独立的任务。本文假设智能体是异构的,每个智能体都有自主性且追求自身利益,本文期望智能体系统可完成更多的任务,从系统的层面来看,在保证智能体负载均衡分配的应当最大程度地提升资源利用率,系统期望最大程度地降低智能体之间的负载失衡状况,防止智能体出现过载或者利用不足的情况,保障整个系统有鲁棒性、可靠性以及稳定性。

人们利用队列模型和博弈论研究了分布式计算中的负载平衡问题,并提出了各种性能指标,如预期响应时间、负载不平衡百分比、用户产生的成本等^[182]。研究人员提出了资源分配问题的集中式解决方案^[183,184]。由于可扩展性和复杂性的原因在分布式环境中集中解决负载平衡问题并不可行。Zhang^[185]提出一种进化多目标优化算法来解决双目标工作流调度问题。Kishor^[186]提出一种基于极值优化的多目标优化算法解决负载不平衡最小化、进程间通信最小化和任务迁移指标三目标负载均衡问题。Talukder^[187]提出一种多目标差分进化算法,可同时优化成本和时间两个相互冲突的目标。Grosu^[188]将分配问题建模为一个非合作博弈,每个博弈方的报酬都是执行任务的预期响应时间。Kameda^[189]考虑了通信网络中的最优流量控制问题,并证明在存在自利用户的情况下,流量分配问题所得到的近边界是低效的。Subrata^[190]提出一种基于非合作博弈的框架来解决计算网格中的分配问题。Tripathi^[191]在分布式数据中心的背

景下考虑了与文献^[188]类似的负载平衡问题，提出了一种以运营成本最小化为目标的非合作负载均衡博弈并获得了纳什均衡结构。

在现有的研究中任务分配算法多以优化系统效率为目标，忽略个体智能体的资源约束和负载均衡问题。容易导致部分智能体过载其他智能体资源利用不足的情况，造成单点故障，影响系统的整体性能。研究人员提供了多目标负载均衡问题的非合作解决方案。Li^[192]引入多智能体强化学习方法，开展云边端协作背景下的任务负载均衡调度研究，将任务调度视为一个竞争性的多智能体系统。采用分散式部分可观测马尔可夫决策过程来模拟奖励最大化问题，并设计了一种基于注意力通信的多智能体强化学习算法来解决该问题。Baert^[193]提出了一种自适应多智能体系统，减法器智能体在作业期间进行交互，任务的重新分配基于协商，以减少负载最大的减法器的工作量，减少运行时间。Ebrahim^[194]提出了一种采用多智能体强化学习的全分布式负载均衡解决方案，它能智能地分配物联网工作负载，优化等待时间，同时在网络中提供公平的资源利用率，利用迁移学习进行终身自我适应，以适应环境的动态变化。Omidi^[195]将资源分配的公平性和效率放在首位，利用基于令牌的机制来确保公平性。Wang^[196]将随机学习自动机和连续时间优化技术相结合，提出一种用于实时适应性和动态负载均衡的创新解决方案解决多智能体资源管理所面临的复杂挑战。

效率和负载均衡大多数时候呈现出相互矛盾的优化目标，权重系数对两者间平衡的调节会影响算法的实用性。随着智能体数量以及任务复杂度的增加，任务分配算法的计算复杂性呈指数式增长，怎样在高效性与准确性之间达成平衡成为一项关键课题。在实际应用场景里，任务数量、类型以及智能体的状态有可能随时间发生变化，如何设计动态响应机制来适应环境变化，是另外一个关键的研究方向。在多智能体系统中任务分配的负载均衡一般意味着所有智能体的资源利用率趋向于均衡，在追求负载均衡时可能会致使某些低效智能体被分配到超出其处理能力的任务，拖累整体系统的性能。如果单纯追求系统效率，任务可能会集中分配给性能最优的智能体，使得这些智能体超负荷运行。负载均衡和效率之间的冲突是设计任务分配算法时必定要面对的核心挑战。

本章设计了一种负载均衡的任务分配机制，权衡智能体之间任务分配的负载均衡以及系统效率，优化智能体的资源利用率并防止个体过载。该方案考虑缩短智能体的响应时间以及提高系统的利用率，同时避免出现某些智能体资源利用率过高而另外一些智能体资源利用率过低的状况。为智能体设计了一个名为均衡性变化的指标，这个指标可量化任务分配造成的负载不平衡，运用加权和方法把均衡性变化指标与预期响

应时间结合起来,将组合指标当作新的目标函数。把问题表述为非合作负载平衡博弈,本章提出了一种基于均衡性的分布式任务分配方法来计算博弈的纳什均衡。在保证系统整体效率的同时实现智能体之间的负载平衡,优化任务分配策略以减少任务从发布到完成的时间。

5.2 分布式任务分配系统模型

5.2.1 系统模型

本章考虑一个由 m 个异构智能体和 n 个任务组成的分布式多智能体系统,这些智能体独立并行工作。智能体呈现出异构的特性,也就是说它们有各不相同的性能表现, $A = [a_1, \dots, a_m]^T$ 是 m 个异构智能体所构成的有限集合, $T = [t_1, \dots, t_n]^T$ 是处于有界区域内 n 个任务形成的有限集合,本章将智能体集合标记为 $M = 1; 2; \dots; m$,把任务集合标记为 $N = 1; 2; \dots; n$,并且 $m \ll n$ 。任务列表属于任务集合 T 的子集,任务只能被分配给有能力执行它的智能体,各个智能体的有序任务列表可能出现重合的情况。智能体可被分配多个有兼容类型的任务,不过一次仅能执行一个任务。分配给智能体的任务会依照特定的顺序去执行,来让每项任务都可在各自规定的截止时间之前完成,使得时间成本 $c_{i,j}$ 尽可能地低。 $S = [s_1, s_2, \dots, s_n]^T$ 表示执行任务所需要的持续时间,执行不同任务所需的时间是有可能不一样的。

智能体 $j \in M$ 的处理能力 P_j 体现了智能体 j 的综合处理能力以及工作效率等方面,囊括了智能体的性能、能量以及其他与之相关的能力,借助 P_j 可推导出智能体在不同任务负载情形下的响应时间以及工作效率。智能体 j 的可用能量或者资源总量是 E_j ,这可确定智能体是否拥有足够的资源来完成任务,任务 $i \in N$ 的任务复杂度或者负载需求是 L_i ,不同的任务可能有着不同的复杂度或者负载需求,凭借 L_i 可计算出智能体 j 的响应时间以及整个系统的响应状况。智能体 j 移动到任务 i 所在位置的移动距离是 D_{ij} ,结合智能体的移动速度以及移动能耗等情况,可以计算出移动能耗。智能体 j 的平均移动速度是 V_j ,结合移动距离 D_{ij} ,可用来计算智能体从当前位置抵达任务位置所需要的移动时间。智能体 j 的单位距离移动能耗是 $C_{move,j}$,结合移动距离 D_{ij} ,可估算出完成该任务所需的移动能量消耗。

任务规模存在区别,有的任务需要多个智能体才能完成,本章将大的任务分解为

小的任务。 x_{ij} 表示任务*i*分配给智能体*j*的工作量比例, $0 \leq x_{ij} \leq 1$, $x_{ij}=0$ 表示任务*i*不分配给智能体*j*, $x_{ij}=1$ 表示任务*i*分配给智能体*j*, $0 < x_{ij} < 1$ 表示任务*i*分配给多个智能体。任务规模不同, 智能体执行任务所需的成本与消耗也不同。决定 x_{ij} 值后, 任务*i*分配给智能体*j*。任务*i* $\in M$ 的执行负载 L_{exec} 表示智能体执行任务时所需的能量。智能体移动负载 L_{move} 表示智能体移动时所需的能量。智能体总负载 $L_{\text{total}} = L_{\text{exec}} + L_{\text{move}}$ 表示智能体完成任务所需的总能量, 包括任务执行和移动。本文给定一组任务和一组独立工作的智能体, 将智能体建模为一个 $M/M/1 - FCFS$ 队列^[197]。任务随机地以泊松过程到达某个智能体, 执行时间呈指数分布且随机, 多个任务按照先到先得(First Come First Serve, FCFS)原则依次执行。由于所有智能体都被建模为 $M/M/1$ 队列, 因此, 为了保持稳定, 智能体的负载约束条件为 $\sum_{i=1}^m \lambda_i < \sum_{j=1}^n \mu_j$ 。

5.2.2 预期响应时间模型

智能体*j*的工作量分配配置 x_j 是向量 $x_j = (x_{j1}, \dots, x_{jm})$, 本章将其称为智能体*j*的策略。所有智能体的共同战略被称为系统的策略概况。联合策略分配 X 是任务策略向量 $X = (x_j, \dots, x_n)$, 本章用 $X = (x_j, x_{-j})$ 表示, 其中 x_{-i} 代表除智能体*j*之外的所有其他智能体的策略。当且仅当策略配置文件 $X = (x_j, x_{-j})$ 满足 $0 \leq x_{ij} \leq 1, \forall i \in M; \forall j \in N$, $\sum_{j=1}^n x_{ij} = 0 \vee 1, \forall i \in M$, $\sum_{i=1}^m x_{ij} L_i < P_j, \forall j \in N$ 等约束条件时, 本章称其为可行策略配置文件。本章要确保系统为智能体分配非负的负载量并保证智能体上的执行负载必须小于其负载率, 同时需要确认任务是否被分配。

给定一个可行的策略配置文件 $X = (x_j, x_{-j})$, 智能体*j*完成任务*i*的预期响应时间 $\mathcal{D}_j(x_j, x_{-j})$:

$$\mathcal{D}_j(x_j, x_{-j}) = \sum_{i=1}^m x_{ij} \frac{L_i}{P_j} + \sum_{i=1}^m \delta(x_{ij}) \frac{D_{ij}}{V_j} \quad (5.1)$$

其中, $\delta(x_{ij})$ 为指示函数, 当 $x_{ij} > 0$ 时取值为 1, 表示智能体*j*会移动到任务*i*的位置; 否则为 0。

基于各任务的响应时间 R_j 的平均值得到任务的平均响应时间 $\bar{R}_{\text{task}}$ 。假设共有*m*个任务, 每个任务的响应时间为 R_{ij} , 则任务的预期平均响应时间为:

$$\bar{R}_{\text{task}} = \frac{1}{m} \sum_{i=1}^m R_{ij} = \frac{1}{m} \sum_{i=1}^m \left(\frac{D_{ij}}{V_j} + \frac{L_i}{P_j} \right) \quad (5.2)$$

智能体*j*从当前位置移动到任务*i*的移动时间 $T_{\text{move},ij}$, 智能体任务执行时间 $T_{\text{exec},ij}$ 。智能体*j*的移动能量消耗 $E_{\text{move},ij}$, 总能量消耗 $E_{\text{total},ij}$ 包括移动能量消耗和任务执行能

量消耗（假设任务执行能量消耗由任务复杂度和处理能力确定），则：

$$E_{total,ij} = C_{move,j} \times D_{ij} + \frac{x_{ij}L_i}{P_j} = E_{move,ij} + \frac{x_{ij}L_i}{P_j} \quad (5.3)$$

5.2.3 智能体均衡性变化评价模型

给定联合负载均衡策略配置文件 $X = (x_j, x_{-j})$ ，让 $\sum_{i=1}^m x_{ij}L_i$ 成为分配给智能体 j 的总任务量。给定联合策略配置文件 $X = (x_j, x_{-j})$ ，智能体 $j \in N$ 的利用率定义为智能体 j 的总负载与其工作能力的比率，即：

$$\rho_j(X) = \frac{\sum_{i=1}^m (x_{ij}L_i + c_{i,j})}{P_j}, \quad (5.4)$$

其中， ρ_j 的取值范围从 0 到 1。值为 0 表示智能体 j 完全未被使用，值为 1 表示智能体 j 的利用率为 100%，即智能体 j 已超载。

当且仅当所有智能体的利用率相同时，即 $\rho_j(X) = \rho_l(X), \forall j, l \in N$ ，联合负载均衡策略配置文件 $X = (x_j, x_{-j})$ 被称为均衡利用率策略。当智能体的联合策略配置文件为均衡利用策略时，智能体的利用率被视为均衡利用率。本章将反映智能体之间负载不平衡的均衡性变化定义如下：

$$\zeta(X) = \sum_{j=1}^n (\rho_j(X) - \bar{\rho}(X))^2, \quad (5.5)$$

其中， $\bar{\rho}$ 表示所有智能体的平均利用率。

负载分配决策是由智能体自主做出的，从任务的角度对智能体之间的负载不平衡进行量化。给定联合负载均衡策略配置文件 $X = (x_j, x_{-j})$ ，设 $\sum_{k=1, k \neq i}^m x_{kj}\lambda_k$ 为除 i 之外的所有智能体 j 的总负载。智能体 j 在任务 i 处的可用能量 P_j^i 确定为：

$$P_j^i = E_j - \sum_{i=1}^m \delta(x_{ij}) C_{move,j} \times D_{ij} - \sum_{i=1}^m \frac{x_{ij}L_i}{P_j} \quad (5.6)$$

智能体 j 对任务 i 处的利用率 ρ_{ij} 定义为任务 i 分配给智能体 j 的负载（即 $x_{ij}\lambda_i$ ）与智能体 j 在任务 i 处可用能量之比，则智能体的平均利用率估算为：

$$\bar{\rho}^i(X) = \frac{1}{n} \sum_{j=1}^n \frac{(x_{ij}L_i + c_{i,j})}{P_j^i}. \quad (5.7)$$

当任务 i 将其负载分给智能体时，造成的智能体均衡性变化为：

$$\zeta_j(x_j, x_{-j}) = \frac{1}{n} \sum_{j=1}^n \left(\frac{x_{ij}L_i + c_{i,j}}{P_j^i} - \frac{1}{n} \sum_{j=1}^n \frac{(x_{ij}L_i + c_{i,j})}{P_j^i} \right)^2. \quad (5.8)$$

本章计算利用率的平方偏差，但如果系统中智能体数量 n 较大，则总的平方差会

随之增大。均衡性变化 $\zeta_j(x_j, x_{-j})$ 衡量任务 i 所造成的智能体之间的负载不平衡。随着该标准值的减小, 智能体利用率方面的均衡性增加。因此, 在此模型中, 目标是找到一个可行的策略配置文件 $X = (x_j, x_{-j})$, 使得每个智能体 $j \in M$ 的 ζ_i 值最小。

5.3 均衡性博弈的纳什均衡

5.3.1 均衡性问题优化

本章的目标是最小化任务的预期响应时间及最小化每个智能体的均衡性变化值, 这两个目标是相互冲突的。当系统专注于最小化任务的预期响应时间时, 会尽可能地利用更快的智能体。由于对更快智能体的偏好, 负载不平衡的影响被忽视, 更快的智能体变得过载, 较慢的智能体则负载不足。当考虑最小化智能体的均衡性变化值时, 系统会均衡利用速度较慢和较快的智能体, 资源的平等利用可能会增加每个任务的执行时间。

如果一个多目标优化问题涉及相互冲突的目标, 则不可能找到一个可以同时优化所有目标的单一解决方案。在多目标优化的背景下, 最优性的概念与一组解决方案相关联, 目标之间的最佳权衡可以使用帕累托最优的概念来定义。策略配置文件 $X^* \in \Omega$ 是帕累托最优的, 当且仅当不存在支配 X^* 的 $X \in \Omega$ 满足如下条件:

$$\forall k \in [o], f_k(X) \leq f_k(X^*) \wedge \exists k \in [o], f_k(X) < f_k(X^*). \quad (5.9)$$

其中, o 表示优化问题中的目标数量, Ω 是可行策略空间。换句话说, 如果不存在另一个策略配置文件在不恶化其他目标的情况下改进至少一个目标, 则策略配置文件 X^* 就被称为帕累托最优。

本章的目标是找到一个可行的策略轮廓, 使得这两个目标之间达到平衡的折衷, 并且同时优化这两个目标的值。因此, 要找到一个可行的解决方案, 本章需要为这两个目标分配一个相对的偏好。针对这一目标, 本章对两者进行了综合考虑。为了综合这两个目标, 本章采用加权求和的方法。在该方法中, 本章将每个目标乘以一个权重因子, 并将这两个目标加总得到一个综合目标函数。权重因子表示各目标的相对重要性。负载均衡的目标是最小化任务的预期平均响应时间和最小化智能体均衡性变化, 将两个目标结合后, 本章得到:

$$\min_{x_j} f_j(x_j, x_{-j}) = w_1 \bar{R}_{task} + w_2 \zeta_j(x_j, x_{-j}), \quad (5.10)$$

其中, 权重因子 $0 < w_1, w_2 < 1, w_1 + w_2 = 1$ 。 w_1 的值越大, 表明对任务(即任务的期

望响应时间)的关注度越高; w_2 的值越大, 表明对智能体的均衡使用关注度越高。因此, 选择适当的 w_1 和 w_2 值取决于在设计分布式系统时确定的目标。

因此, 在给定了任务在智能体之间的建模和分布的情况下, 本章现在将正式阐述本章试图解决的问题。给定一个由 m 个智能体和 n 个任务组成的异构分布式系统以及智能体的目标函数 f_j , 找到一个可行的联合策略概要 $\mathcal{X} = (\mathbf{x}_j, \mathbf{x}_{-j})$, 使得每个智能体的目标函数值最小。

智能体 j 的性能指标由其目标函数 f 给出, 智能体 j 试图找到一种使其目标最小化的负荷分配策略。由于目标函数取决于所有智能体的策略, 因此智能体受所有参与智能体策略的影响, 而不是受其策略的影响。本章将负载均衡问题建模为一个非合作博弈, 非合作的负载均衡博弈形式化地定义为一个元组 $G = \langle \mathcal{M}, (\mathbf{x}_i)_{i \in \mathcal{M}}, (f_i)_{i \in \mathcal{M}} \rangle$, 其中 $M = \{1, \dots, m\}$ 为玩家集合, $\mathbf{x}_j$ 为玩家 $j \in \mathcal{M}$ 的策略轮廓, 每个玩家 j 的惩罚函数为 f_j 。在本章当中智能体被看作是玩家, 对于每一个玩家 $j \in M$ 而言其惩罚函数是 f_j , 每一个玩家 j 的策略概貌是 $\mathbf{x}_j = (x_{11}, \dots, x_{1n})$ 。本章把负载均衡问题表述成最小化问题并且将其称作惩罚函数, 如果

$$i \in \mathcal{M}: \mathbf{x}_j^* \in \underset{\mathbf{x}_{ij}}{\operatorname{armin}} f_j(\mathbf{x}_j, \mathbf{x}_{-j}) \quad (5.11)$$

那么负载均衡博弈 G 的联合策略 $\mathcal{X}^* = (\mathbf{x}_j^*, \mathbf{x}_{-j}^*)$ 就被称作纳什均衡。在这个博弈里玩家 j 给定其他玩家的策略记为 $\mathbf{x}_{-j}$, 其以便挑选一个策略 $\mathbf{x}_j$, 让其惩罚 $f_j(\mathbf{x}_j, \mathbf{x}_{-j})$ 达到最小, 这个博弈的解概念就是纳什均衡, 处于纳什均衡状态时没有任何一方借助选择不一样的策略来降低自身的惩罚, 而其他玩家的策略则维持不变。

从博弈 G 的定义中可以看出, 每个玩家的目标都是找到一个能使其成本函数值最小的策略。此外, 对于每个智能体来说, 最优的策略配置文件就是博弈的纳什均衡。因此, 计算智能体 i 的最优策略可以转化为确定一个智能体与 n 个任务的最优策略的问题 (对于智能体 i 来说, 其他人的策略是固定的)。对于每个玩家 $j \in \mathcal{M}$, 本章将其表述为非线性优化问题:

$$\mathbf{x}_j^* \in \underset{\mathbf{x}_j}{\operatorname{argmin}} f_j(\mathbf{x}_j, \mathbf{x}_{-j}) \quad (5.12)$$

假设其他人的策略是固定的, NLP_j 的决策变量是智能体 j 的策略, 即 $\mathbf{x}_j$ 。当每个智能体的策略都是对其他智能体策略的最佳响应时, 就可以找到均衡策略曲线 $\mathbf{X}$ (即纳什均衡)。考虑到其他智能体的策略是给定的, 智能体 j 的最佳响应定义为 $\mathbf{x}_j$ 的策略, 它能为智能体 j 提供最小的惩罚, 非线性优化问题的解就是玩家 j 的最优响应。

5.3.2 最佳响应计算算法

首先探讨算法的一些预处理步骤，当生成每个玩家 j 的策略之后，便会对惩罚函数值展开评估，同时也会评估所有智能体的可用资源。如算法 5-1 所示，在首个 for 循环里，算法针对智能体 i 生成并修复多个随机的任务分配策略，以此探索可能的策略空间。在第二个 for 循环中，算法借助重组机制去更新在策略生成阶段所获取的任务分配策略，改进智能体系统的任务分配方案。运用更新与修复机制持续改进这些策略，让智能体在分配任务时更加均衡，最终得以形成一个更优的负载分配方案。在算法的最后部分，挑选惩罚值最小的任务分配策略当作智能体的最优响应，智能体 j 只会选取惩罚值比当前状态更低的策略，或者维持原状态，而绝对不会选择更糟的策略。算法为每个智能体保证了策略的改进，直至达到均衡状态。算法的剩余部分借助迭代来生成候选策略、修复不契约束的策略并且选择惩罚值最小的策略，最终获得每个智能体的最优任务分配方案。如此可保证系统在负载均衡和效率方面取得最优的折衷结果。

Algorithm 5-1: Candidate-Based Recombination and Repair Algorithm for Agent j

Input: P_j, E_j, L_i, x_i

Output: x_i^{OPT}

```

1 Initialize  $x_i^{OPT} \leftarrow x_i$                                 ▷ Generate  $NP$  random feasible strategies
2 for  $k = 1$  to  $NP$  do
3   Generate a feasible random strategy
4   Evaluate penalty  $f(v_i^k, x_{-i})$                             ▷ Update strategies through optimization
5 for  $k = 1$  to  $NP$  do
6   Transform the current strategy  $v_k$  into a new strategy  $u_k$ 
7   Evaluate penalty  $f(u_i^k, x_{-i})$ 
8   if  $f(u_i^k, x_{-i}) < f(v_i^k, x_{-i})$  then
9     Replace the old candidate strategy  $v_i^k$  with the newly generated strategy  $u_i^k$ 
10    Update its penalty value  $f(u_i^k, x_{-i})$  to that of the new strategy  $f(v_i^k, x_{-i})$     ▷ Select the best
        strategy
11 Select the candidate strategy with the lowest penalty  $f(v_i^k, x_{-i})$  ▷ If improvement exists, update the
    optimal strategy
12 if  $f(v_i^k, x_{-i}) < f(x_i, x_{-i})$  then
13   Set the current candidate strategy  $v_i^k$  as the new optimal strategy  $x_i^{OPT}$  for agent  $i$ 

```

每次迭代中，算法生成一个随机的任务分配策略 v_i^k ，该策略描述了任务 i 如何在不同的智能体之间分配。随机策略的形式为：

$$v_i^k = (v_{\{i1\}}^k, v_{\{i2\}}^k, \dots, v_{\{in\}}^k) \quad (5.13)$$

其中， $v_{\{ij\}}^k$ 表示任务 i 分配给智能体 j 的工作量比例。要让生成的策略契约束条件，就得对任务分配做归一化处理，促使各分量之和等于 1。

在生成策略之后，要保证其符合所有约束条件，要是生成的策略不符合约束，那

就运用修复机制给予调整,要是任意负载分量 v_{ij}^k 超过了允许的上限,那就把该负载分量调整为最大允许值,再将剩余的负载按比例重新分配给其他智能体。修复机制能保证生成的策略都是可行的,在策略生成并经过修复后计算策略的惩罚值 $f(v_i^k, x_{-j})$ 。惩罚值用于衡量任务分配策略的质量,一般和系统的目标函数有关,惩罚值越低,说明该策略在任务分配方面越优化,智能体系统的整体性能也就越好。

在每次迭代时,为改进当前随机生成的策略 v_i^k ,算法会借助部分重组生成一个新策略 u_i^k 。新策略由当前策略 v_i^k 与上一轮的优胜策略 v_i^p 依照一定规则组合而成,凭借比较两者的惩罚值来判断哪一个策略表现更优,进行策略更新。为此从集合 $\{1, 2, \dots, n\}$ 中随机挑选一个索引 id ,并生成一个随机数 $r \in [0, 1]$ 。新策略 u_i^k 通过对两个策略进行部分重组得到,使得智能体 j 的任务分配策略能够融合不同策略的优点,进一步改进负载分配,生成方式如下:

$$u_{ij}^k = \begin{cases} rv_{ij}^k + (1-r)v_{ij}^p, & j \geq id \\ v_{ij}^p, & j < id \end{cases} \quad (5.14)$$

新生成的策略 u_i^k 要接受可行性检验,目的在于保证其符合所有约束条件。这些约束条件囊括相等约束与不等式约束,新生成的策略存在违反这些约束的可能性,要进行相应修复。要是 u_i^k 违反了相等约束,那就得对策略给予修复,以此保证契合相等约束。为查验新策略 u_i^k 是否契合总和为1的约束,需先计算偏移量 δ^k :

$$\delta^k = \sum_{j=1}^n u_{ij}^k - 1 \quad (5.15)$$

其中, $\delta^k \neq 0$ 表示当前策略的各分量之和不为1,需要进行修复。为此,可按以下规则对每个分量 u_{ij}^k 进行比例调整,对负载分配分量进行修复,使其总和回到1:

$$u_{ij}^k = \begin{cases} u_{ij}^k \times \left(1 + \frac{\delta^k}{\sum_{j=1}^n u_{ij}^k}\right), & \text{if } \delta^k > 0, \\ u_{ij}^k \times \left(1 - \frac{\delta^k}{\sum_{j=1}^n u_{ij}^k}\right), & \text{if } \delta^k < 0, \\ u_{ij}^k, & \text{if } \delta^k = 0. \end{cases} \quad (5.16)$$

策略中的负载分量被按比例进行调整,从而确保满足相等约束。

修复完成之后,算法会再度核查策略是否契合全部不等式约束条件,同时针对修复后的策略 u_i^k 计算惩罚值,要是 u_i^k 的惩罚值比当前策略 v_i^k 的惩罚值小,那就用 u_i^k 替代 v_i^k ,新策略在改善负载均衡方面有更好的性能。

算法会计算所有更新策略 v_i^k 的惩罚值,从中挑选出惩罚值最小的策略当作候选策略。要是这个候选策略的惩罚值 $f(v_i^k, x_{-j})$ 小于原始策略 $f(x_j, x_{-j})$ 的惩罚值,那么就

将智能体的最优策略更新为该候选策略，这就是说要是新生成的策略在任务分配上比原始方案更具优化性，智能体就会采用这个新策略作为其当下的最优响应。

算法最终会返回智能体 i 的最优任务分配策略 x_i^{OPT} ，这个策略是在多次生成并修复任务分配方案之后，选取惩罚值最小的方案作为最终的负载分配策略，其目的在于在保证负载均衡的同时使系统的总惩罚值最小化，较小的惩罚值意味着任务负载在智能体之间的分配更为合理，系统的整体性能更优。

算法中新生成的策略列表 S' 里每个元素的惩罚值都比之前的相应策略列表 S 更优，这意味着经过持续的比较与选择，算法可保证最终的策略是当前可行策略中惩罚值最小的，也就是负载分配达到最优化。

5.3.3 基于纳什均衡的分布式负载均衡算法

由于所有玩家都是自私的并试图最小化自己的损失，本章设计了一种分布式负载均衡任务分配算法，每个参与者计算其他参与者的给定策略来完善自己的策略，每个玩家都可以更新策略以最小化惩罚函数。

如算法 5-2 所示，算法起始于比例分配方案，为各个智能体分配任务。在初始阶段，会挑选一名智能体作为“启动玩家”，此智能体开始计算策略的最佳响应，智能体依照预先设定的顺序开展任务策略更新，启动玩家率先执行最佳响应算法。随后其他智能体依次跟进，智能体借助最佳响应算法计算其他智能体的策略，以此确定最佳任务分配策略。智能体更新策略后，会把新策略的信息传递给其相邻的智能体，若接收到的消息标记为“STOP”，该智能体要停止策略更新，此时所有智能体达成一致，否则标记为“CONTINUE”，意味着其他智能体需继续进行策略更新。在更新策略时，每个智能体要对所有可行策略加以评估，挑选出使系统惩罚值最小的策略作为当下的最优策略。

每当智能体更新策略，都要评估惩罚函数的变化。依靠计算新策略与旧策略的惩罚值差异，来判定该策略是否提升了系统的整体性能。新策略评估完毕后，智能体会将更新后的策略信息传递给相邻的下一个智能体，继续开展下一轮迭代。这个过程持续进行，直至所有智能体达到纳什均衡，也就是没有智能体可改变自身策略来降低自身损失。

算法在智能体之间进行多轮迭代，每一轮里每个智能体依据其他智能体的策略，运用最佳响应算法来使自身的惩罚函数最小化。当所有智能体的策略在连续几轮中都

未发生变化时，算法收敛至一个平衡状态，即达到纳什均衡状态，此时系统中的任务分配策略不再改变，每个智能体的任务分配处于较优状态。

每个智能体被视作自私的，只考虑如何最小化自身的惩罚函数，这契合实际的多智能体竞争环境。算法有分布式特性，智能体之间可并行更新自身策略，这种机制可提升算法效率，并保证系统能快速收敛至稳定状态。当所有智能体达到稳定状态时，系统达到纳什均衡，此状态下的任务分配在整体上是最优的。算法持续更新每个智能体的策略，促使任务在智能体之间的分配达到最佳平衡，每个智能体依据其他智能体的策略动态调整自身的任务分配，最终达成系统整体的最优负载平衡。

Algorithm 5-2: Equitability-Aware Load Balancing Algorithm for finding Nash Equilibrium

Input: P_j, E_j, L_i, x_i

Output: Final strategies at Nash Equilibrium

```

1 Choose an agent  $i_{\text{start}}$  as the starting player                                ▷ Initialization
2 Set  $i \leftarrow i_{\text{start}}$ 
3 Set all  $x_i$  to initial proportional allocations
4 flag  $\leftarrow$  CONTINUE
5 while flag = CONTINUE do
6      $x_i^{\text{old}} \leftarrow x_i$                                                     ▷ Main Loop
7      $x_i^{\text{new}} \leftarrow \text{BESTRESPONSE}(x_i^{\text{old}}, x_{-i})$ 
8     Compute  $\Delta f = f(x_i^{\text{new}}, x_{-i}) - f(x_i^{\text{old}}, x_{-i})$ 
9     if  $\Delta f < 0$  then
10          $x_i \leftarrow x_i^{\text{new}}$ 
11     else
12         Keep  $x_i$  unchanged
13     if No agent has improved in the last round then
14         flag  $\leftarrow$  STOP                                                    ▷ Check if all agents have stopped improving
15     else
16         flag  $\leftarrow$  CONTINUE
17     Send  $(x_i, \text{flag})$  to agent  $(i + 1)$                                     ▷ Pass updated info to the next agent in the ring
18      $i \leftarrow i + 1 \pmod{m}$ 
19 Return  $\{x_1, x_2, \dots, x_m\}$  as the equilibrium strategies                ▷ Algorithm terminates when flag = STOP

```

5.4 实验结果与分析

5.4.1 实验设计

为了对不同算法得到的解决方案进行定性测量，本章使用了三个性能指标^[186]对方法进行评价。均衡性指数（Equilibrium Index, EI）衡量用户期望响应时间的均等性，

是用以衡量多智能体系统里各个智能体工作负载均衡状况的关键指标，该指数越高，任务在所有智能体间的分配就越均匀，各智能体的负载也就越相近，其定义如下：

$$FI(R) = \frac{(\sum_{i=1}^m R_i)^2}{m \sum_{i=1}^m R_i^2} \quad (5.17)$$

其中， R 是向量 $R = (R_1, R_2, \dots, R_m)$ ，其中 R_i 是玩家 j 的预期响应时间。EI 的值在 0 和 1 之间，当 EI=1 时，每个玩家的响应时间相同。

全局响应时间（Global Response Time, GRT）衡量的是整个系统的预期响应时间，可用于衡量整个多智能体系统处理任务时的效率，全局响应时间越低，系统整体响应任务请求的速度就越快，效率也就越高。其定义如下：

$$GRT = \frac{1}{\lambda_T} \sum_{i=1}^m \lambda_i R_i \quad (5.18)$$

其中， λ_T 是系统中的总任务到达率，即 $\lambda_T = \sum_{i=1}^m \lambda_i$ 。

负载不平衡百分比（Percentage Load Imbalance, PLI）衡量的是智能体上分配负载的均匀程度，能衡量任务在所有智能体之间的分配均衡程度，负载不均衡度越低，任务在智能体之间的分配就越均匀，智能体负载相对更平衡，随着系统利用率上升，负载不均衡度的变化是一项关键的观察指标，可用于判断系统能否有效应对不同负载下的资源分配问题。其定义如下：

$$PLI = \left(\frac{r_{max}}{\bar{r}} - 1 \right) \times 100\% \quad (5.19)$$

其中， r_{max} 表示智能体的最大利用率， $\bar{r}$ 表示智能体的平均利用率。

每个玩家的 NP 以及可接受的容差阈值是需要的。 ϵ 作为停止条件这两个控制参数。在本章进行的所有实验中，这些参数的值都是固定的，即每个玩家的 NP 设为 $2n$ ， $\epsilon = 10^{-4}$ 。对于每次实验运行算法 20 次，取每次运行的平均值作为最终结果。

5.4.2 权重组合比较

在本节中模拟提出的 EALB，并提供数值结果来证明 EALB 的功效。在这部分实验中，本章研究了权重向量 w_1 和 w_2 对本章提出的算法性能的影响。系统利用率 ρ_T 设为 0.7（即 70%）。权重组合 (w_1, w_2) 对应不同的任务或策略的权重分配比例。本章选择了五种不同的权重组合 (w_1, w_2) ，权重组合依次为(1,0)、(0.7,0.3)、(0.5,0.5)、(0.3,0.7)、(0,1)。这些组合表示在优化过程中，目标函数中对两种不同指标的关程度如何变化。(1,0)表示完全关注第一个指标，而(0,1)则表示完全关注第二个指标。

从图 5-1 可明晰地看出，EI、GRT、PLI 这三种性能指标在不同权重组合 (w_1, w_2)

下的实验数值。从 (1,0) 到 (0,1) 的权重组合产生变化, 这意味着随着对第二个指标的关注程度渐渐提高, 相应的值也出现了改变, 权重向量组合的改变对均衡指数以及全局响应时间所产生的影响较为轻微, 然而负载不平衡百分比却受到了较大程度的影响。经过对曲线变化展开分析, 可以发觉在某些权重组合状况下, 或许存在一个平衡点, 可让多种方法的值更为接近, 两种指标的权衡取得了较为理想的成果, 随着 w_1 值的减小 (或 w_2 值的增大), 算法会变得日益关注智能体的均衡利用。

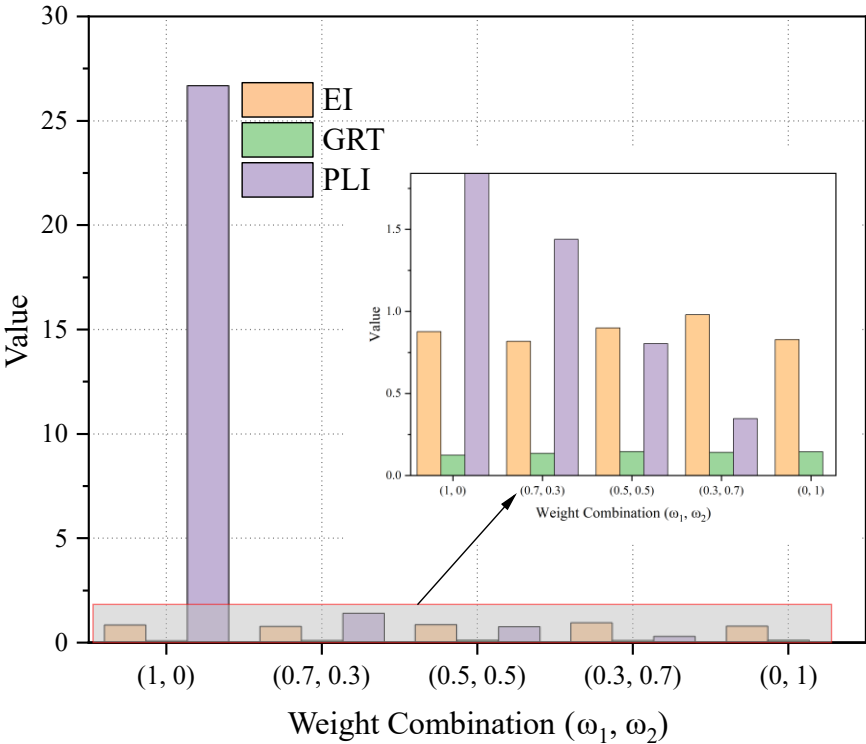


图 5-1 不同权重组合下的算法性能

不同的权重组合会对各个策略的表现产生影响, 可找到一个相对均衡的点。凭借对 EI、GRT、PLI 这三种指标加以比较, 可以判断哪一种在不同的权重组合下表现更为出色。要是以此挑选最优的策略, 那就分析不同组合下的最小惩罚值以及与之对应的权重分配方案, 以此保证系统的整体性能处于最优状态, 权重组合会对算法的性能造成的影响。从图中可看到在一些权重组合下性能变化幅度较大, 在某些权重组合下, 所有方法的性能趋于相同, 这样的组合或许代表着一个有效的折中方案。 $w_1 = 0.7, w_2 = 0.3$ 的 EALB 结果接近最优。因此, 在接下来的所有实验中都使用这组权重因子值。

5.4.3 性能比较

在这组实验中本章将 EALB 与全局优化方案 (GOS)、非合作方案 (NCS) 和基于

Q 学习的非合作方案 (QNCS) 进行比较研究。GOS 是确定全局响应时间的基准方案之一。NCS 是基于非合作博弈概念的玩家最优方案。QNCS 是基于非合作博弈设置的方法。本章设计一个由 10 个任务和 25 个智能体组成的异构系统, 参数设置与原文相同。系统利用率 ρ_T 设为 0.7, 权重 $(w_1, w_2)=(0.7, 0.3)$ 。

图 5-2 展示了本章算法和其他三种算法在三个不同指标上的表现。EALB 在任务分配的均衡性方面优于其他算法, 能够更好地实现任务的均衡分配, 保证不同智能体的工作负载差异较小。EALB 的 GRT 值比 GOS 高出约 2%, 低于 NCS 和 QNCS。在 PLI 指标上 EALB 显示出显著的优势, 比 NCS 减少 91%, 比 QNCS 减少 88%, 比 GOS 减少 31%。相比于其他算法 EALB 方法在降低系统中智能体之间的负载不平衡度方面非常有效, 能够更好地确保不同智能体之间的负载均衡, 避免某些智能体过载其他智能体负载不足的情况。

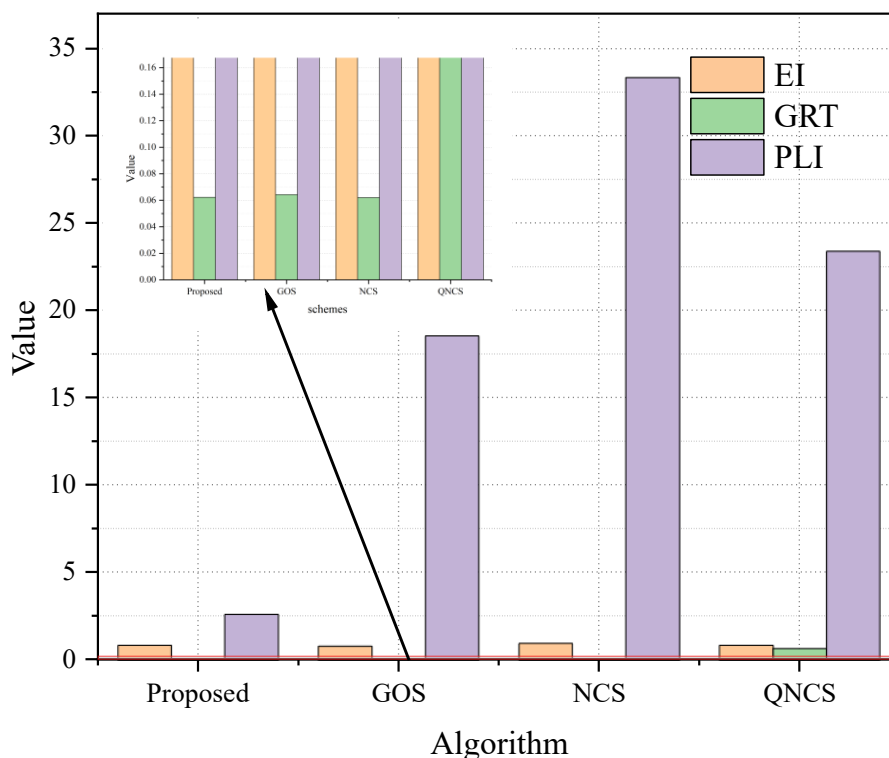


图 5-2 不同指标上不同算法的表现

EALB 设计目标是最小化响应时间, 要尽可能减少智能体之间的利用率差异。与 GOS、NCS 和 QNCS 不同, EALB 方法不会让较慢的智能体利用率过低, 也不会让较快的智能体负担过重。EALB 保证了每个智能体的响应时间较为一致, 并将系统中的负载不平衡降至最低。EALB 在 GRT 指标上的表现不如 GOS, 在提高系统整体均衡性的同时以一定程度的响应时间增加为代价。EALB 在追求任务分配均衡性的过程中牺牲了一部分系统的响应效率。

在 EI 和 PLI 指标上 EALB 的表现优于所有其他算法，表现出显著的优势。全局响应时间虽然略高于 GOS，优于 NCS 和 QNCS，EALB 在整体响应时间上有一定的折中效果。EALB 在以系统整体响应时间略有增加为代价在降低智能体之间的负载不平衡度和提升均衡性方面具有显著优势。EALB 在三个指标中的两个指标上表现更好，可以认为综合性能优于其他算法。

5.4.4 系统利用率的影响

系统利用率所描述的是所有智能体在整体上对于任务处理能力的运用程度，利用率的提升也就意味着系统里每个智能体的工作量在不断增多，系统中的资源使用率会慢慢趋向于满负荷状态。实验的目标在于观察在不同利用率情形下各类算法的表现状况。在多智能体系统里，不同利用率下的性能表现有意义，它呈现出在各种负载水平时各个算法的适应性与效率。实验构建了一个由 10 个任务以及 25 个智能体构成的异构系统，系统利用率的值在 10%–90% 这个范围内进行变化，权重组合 $(w_1, w_2)=(0.7, 0.3)$ 。

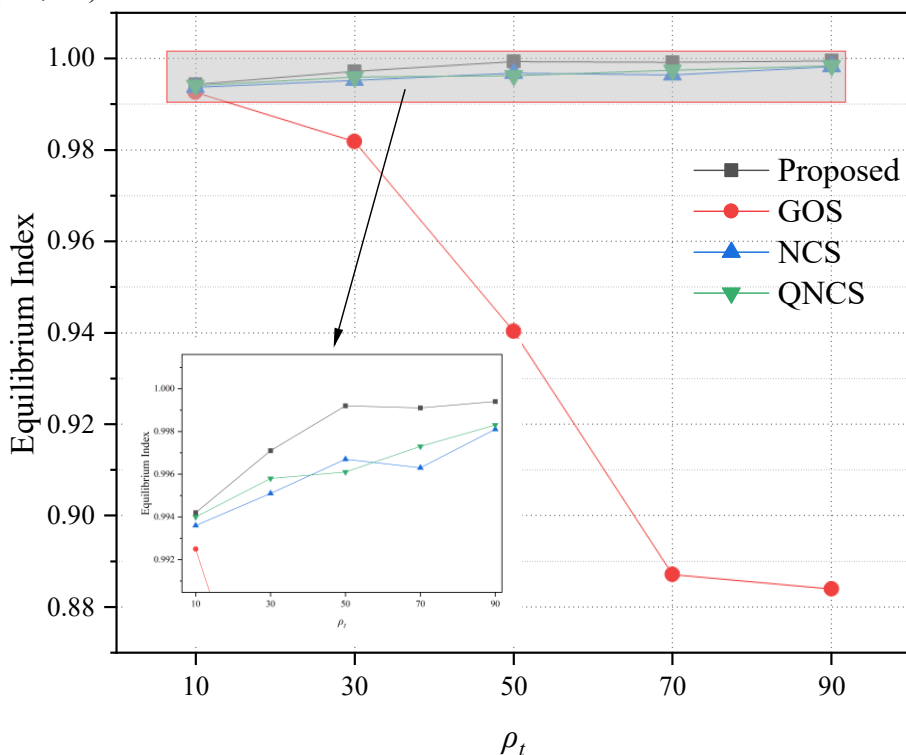


图 5-3 算法在均衡性指数上的表现

如图 5-3 所示，当系统利用率逐步增加时，EALB 的均衡性指数持续稳定上升，最终趋近于 1.0，在负载逐渐加重的情形下，EALB 的任务分配策略依然可达成良好的均衡效果，防止部分智能体负载过重，而其他智能体处于空闲状态。在高利用率的状

况下，EALB 依旧可维持接近 1.0 的均衡性指数。当系统利用率上升时，智能体面临着更大的资源压力，此时 EALB 依旧可高效地进行任务分配，使每个智能体的负载达到均衡。EALB 可在系统利用率不断提高的情况下，最大程度地减少负载不均衡的情况，避免部分智能体出现过载或闲置现象，保证了系统的稳定以及资源的有效利用。GOS 随着系统利用率的增加均衡性指数下降，在高利用率下均衡性指数的波动明显大于其他方法。QNCS 和 NCS 的表现呈现出相似的趋势，表现较为稳定，但未达到 EALB 的水平。

GOS、NCS 和 QNCS 的均衡性指数不如 EALB 稳定，在任务分配的稳定性方面存在欠缺，容易致使某些智能体负载过重，增加系统故障的风险。EALB 在全系统利用率范围内均能维持较高的均衡性指数，可更出色地实现均衡的任务分配，保证每个智能体都承担适量的任务，避免出现资源浪费或过载的情况。

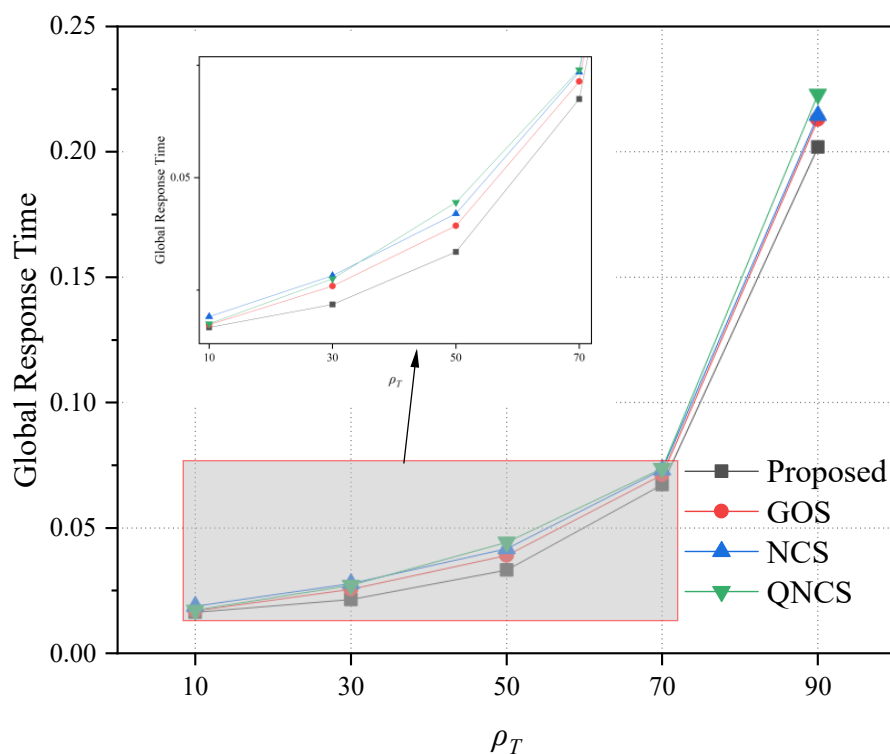


图 5-4 算法在系统利用率变化下的全局响应时间

如图 5-4 所示，EALB 曲线在各个利用率水平上均维持着较低的全局响应时间，GOS、NCS 以及 QNCS 的全局响应时间比 EALB 要高。EALB 在整个系统利用率的范围之内都呈现出相对较低且平稳的全局响应时间，可有效地均衡智能体之间的任务负载，减少因为任务分配不均衡而引发的响应时间延迟。在系统利用率快要达到满载的时候，EALB 的响应时间上升幅度相较于其他算法而言较小，可有效地避免某些智能体出现过载以及其他智能体出现资源浪费的情况，在低到中等利用率的情况下，GOS、

NCS 和 QNCS 的全局响应时间相比于 EALB 是比较高的, 在资源利用方面存在不均衡的状况。当系统利用率接近 1.0 的时候, 这些算法的响应时间会急剧上升, 这反映出这些算法在系统利用率提高时没有可有效地管理和分配任务, 导致部分智能体工作负荷过大, 其他智能体相对处于闲置状态。

EALB 会动态地调整智能体的任务分配, 让每个智能体在不同的利用率下都可合理地分担系统任务, 最大程度地减少响应延迟。在高系统利用率的时候 EALB 仍然可保持较低且稳定的全局响应时间, 可避免某些智能体出现过载, 使得系统内所有智能体都可以接近最优的效率进行工作。

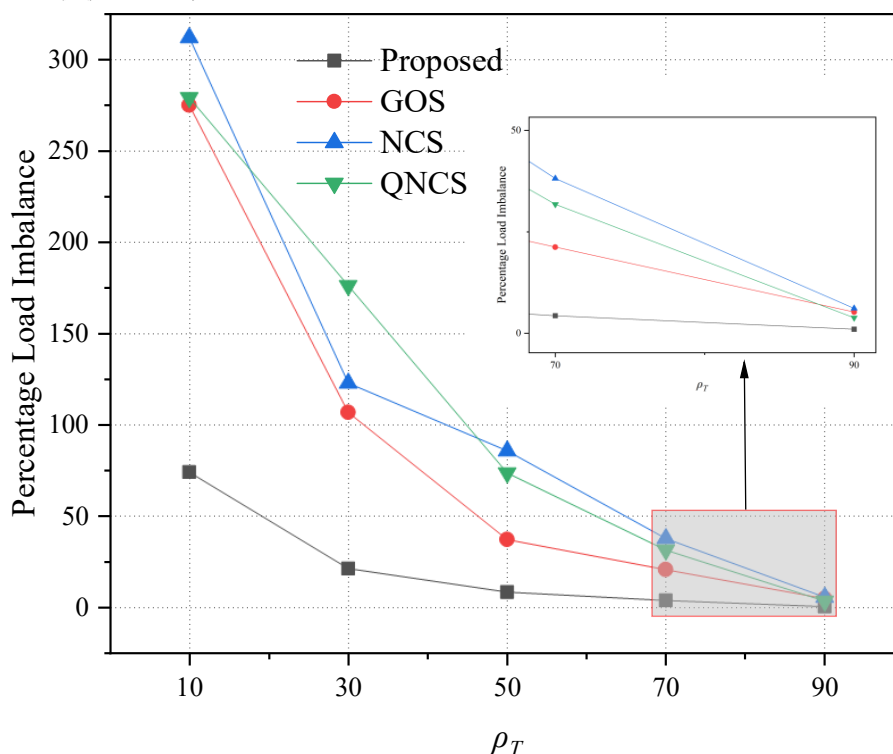


图 5-5 算法在系统利用率变化下的负载不均衡度

如图 5-5 所示, 当系统利用率由低向高逐步递增时, 负载不均衡度整体呈现出下降态势, 任务于智能体之间的分配变得日益均衡, EALB 在整个系统利用率区间始终维持着最低的负载不均衡度, 随着利用率的提高, 负载不均衡度降低, EALB 可在不同负载情形下有效平衡各个智能体之间的任务, 保障系统稳定运行。GOS、NCS 和 QNCS 的负载不均衡度在起始阶段较高, 随着系统利用率的攀升, 负载不均衡度呈下降趋势。在低系统利用率状况下, GOS、NCS 和 QNCS 的负载不均衡度较高, 由于未能均匀分配任务致使某些智能体负载过重或者过轻, 随着系统利用率增加, 这些算法逐渐改善了任务分配的均衡性, 不过最终的负载不均衡度依旧高于 EALB。

在高系统利用率之时, EALB 的负载不均衡度几乎趋近于零, 智能体之间的负载

极为均匀，系统资源的使用效率实现了最大化。尽管 GOS、NCS 和 QNCS 在高利用率下也达成了一定程度的负载均衡，但依旧存在一些不均衡的现象，EALB 在各个负载水平下都可达成良好的任务分配均衡，在高负载时可有效规避任务分配的不均衡问题，维持系统的均衡性与稳定性。

5.4.5 系统异构性的影响

分布式系统的异构程度取决于能力偏度，即智能体最大处理能力和最小处理能力之间的比率。系统的异构性与能力偏度成正比，即偏度值越大的系统异构性越高。在这部分实验中，本章研究了系统异质性对不同负载平衡方案的影响。本章设计了一个由 100 个智能体和 100 个任务组成的系统，具有不同异构程度的不同系统。智能体分为三组，处理能力较大组由 1 至 25 台智能体组成，处理能力适中组由 26 至 75 台智能体组成，处理能力较小组由 76 至 100 台智能体组成。横轴表示系统异质性，值越大代表系统中的智能体在能力、资源、性能等方面的差异性越大。纵轴表示均衡性指数，用于衡量多智能体系统中各个智能体的工作负载是否均衡，越接近 1 表明任务在智能体之间的任务分配越均衡，智能体之间的负载差异较小。

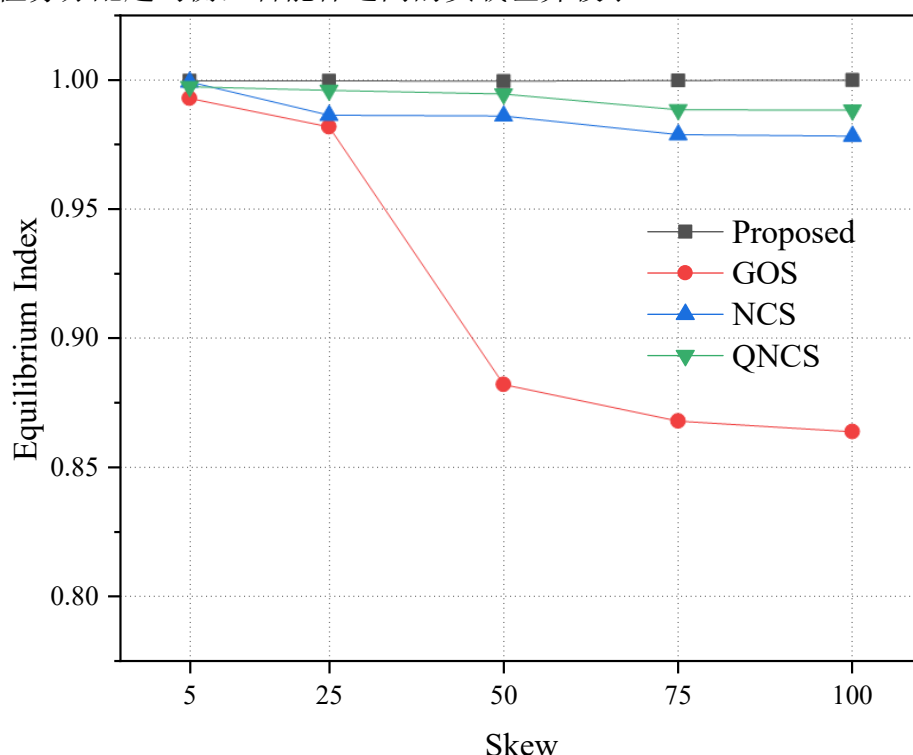


图 5-6 算法在系统异质性变化下的均衡性指数表现

如图 5-6 所示，随着系统异质性的增加各算法的均衡性指数有所变化。EALB 在系统异质性较低时就显示出较高的 EI 指数，EI 指数基本接近 1。随着系统异质性的增

加，GOS 和 NCS 的 EI 指数下降，未能有效调整任务分配导致任务在智能体之间分配不均。QNCS 在 EI 指数方面相较 GOS 和 NCS 有所改进，在高异质性条件下表现略好，稳定性不及 EALB。在异构智能体系统中均衡的任务分配可以防止智能体因过度负载出现故障，避免系统资源浪费。EALB 在智能体性能差异显著的情况下依然能够保持接近 1 的 EI 指数，避免某些智能体过载或负载过低。

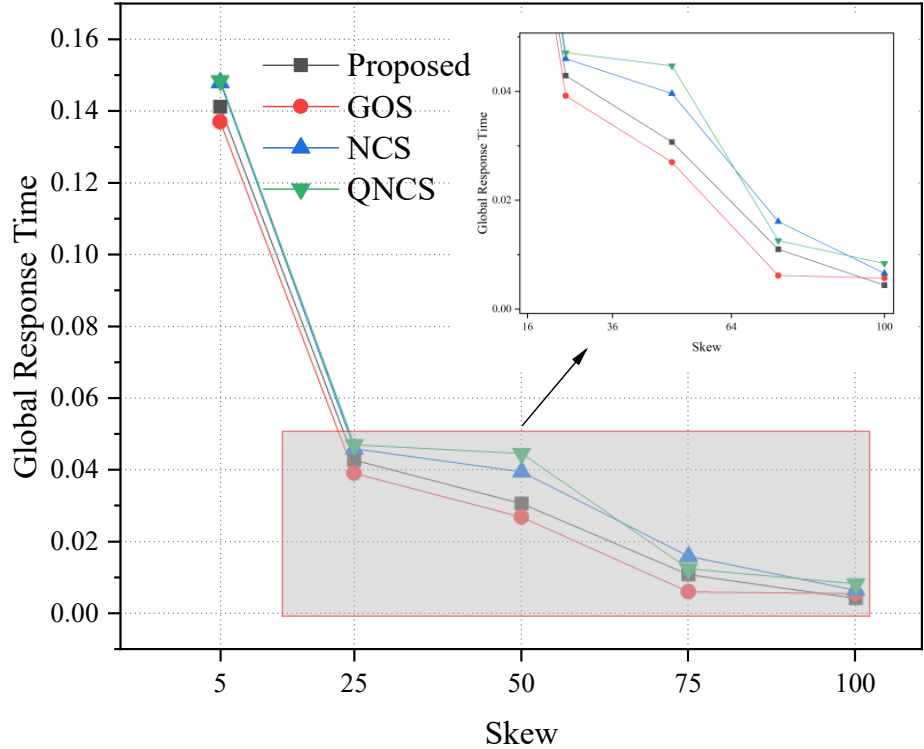


图 5-7 全局响应时间偏差随系统异质性变化的表现

从图 5-7 可以看出，当系统异质性不断增加时，不同算法在全局响应时间偏差方面呈现出不同的表现。EALB 在整个异质性范围内，GRT 指标较低且稳定，可很好地应对智能体之间的性能差异情况。不管系统的异质性怎样变化，EALB 的任务分配策略都可以合理地考虑智能体能力的差异，让各个智能体的响应时间尽可能接近，不会因为性能差异增大而引发十分突出的不均衡现象。随着系统异质性增加，GOS 和 NCS 的全局响应时间偏差也比较小，在高异质性条件下呈现出对异构智能体的适应能力，这些算法都可有效地平衡性能差异十分突出的智能体之间的任务分配，有效控制好系统中智能体响应时间的均衡状态。

在高异质性条件下，EALB 能保持较低的全局响应时间偏差，实现不同能力智能体之间的良好负载均衡，避免出现某些智能体负载过重致使响应时间延长的问题。与之相比，GOS、NCS 和 QNCS 在系统异质性增大时，全局响应时间偏差同样保持较低水平，也能在性能差异较大的环境中有效地均衡任务负载，维持系统整体的均衡状态，

所有算法在异构多智能体系统的任务分配以及全局响应时间均衡方面都表现出色，呈现出了良好的适应性和稳定性。

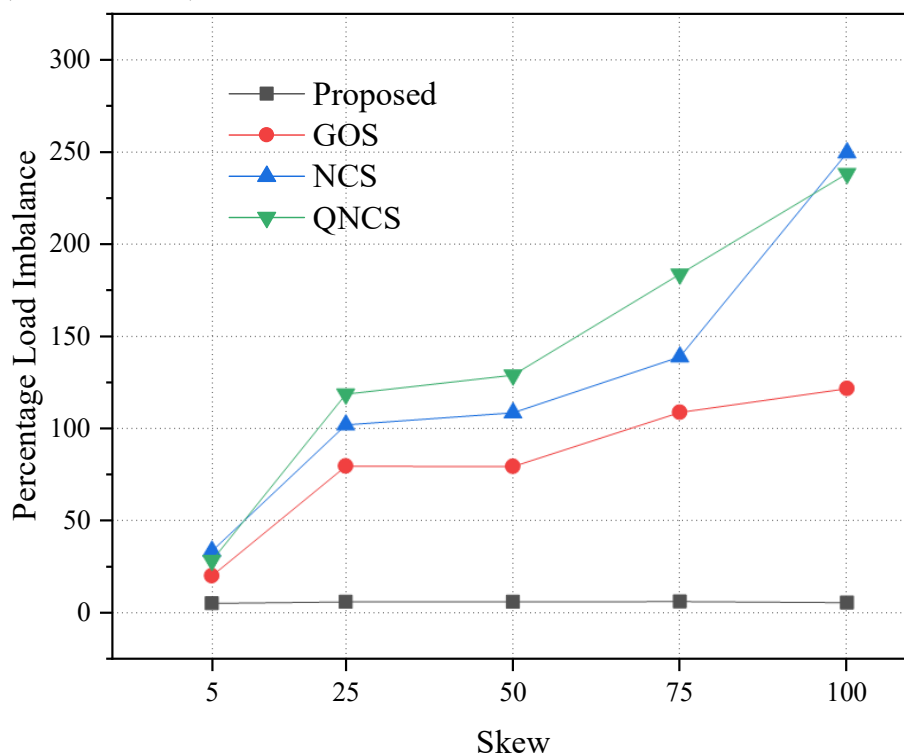


图 5-8 算法在系统异质性变化下对百分比负载不均衡度的表现

如图 5-8 所示，EALB 在系统异质性较低时就显示出较低的负载不均衡度，并且在系统异质性增加的情况下，依然能够保持稳定且较低的 PLI。随着系统异质性的增加，GOS、NCS 和 QNCS 的 PLI 显著上升，在系统异质性较高时 NCS 和 QNCS 的负载不均衡度较高，在面对智能体的异构性时未能有效调整任务分配。GOS 虽然在负载不均衡度上比 NCS 和 QNCS 略有改进，在高异质性下稳定性仍不如 EALB。

EALB 在面对系统异质性变化时展现了出色的负载均衡能力，能够有效应对不同能力智能体之间的性能差异，无论系统的异质性如何变化，EALB 始终保持相对均衡的任务分配，避免出现某些智能体负载过重其他智能体负载不足的情况。

5.4.6 系统规模的影响

在这组实验中研究了不同负载均衡方案在不同系统上的性能。本章改变智能体的数量，任务的数量保持不变。考虑 5 种不同类型的系统，所有系统的参与者数量均为 100。每个系统的智能体数量不同，系统利用率为 70%，权重组合 $(w_1, w_2)=(0.7, 0.3)$ 。在本实验中，系统数量从 60 到 960 不等。第一个系统中有 60 个智能体，在第二个系统中智能体数量是第一个系统的 2 倍。在本实验中，所有考虑的系统都具有相同的速

度偏度。智能体的数量代表了多智能体系统中智能体的规模。随着智能体数量的增加，系统复杂性也会增加，任务的分配和调度变得更加困难，系统响应时间的变化是评估性能的重要指标。

如图 5-9 所示，EALB 在智能体数量增加的情况下，均衡性指数始终保持在较高且稳定的水平，接近于 1.00。能够有效地适应系统规模的扩展，即使智能体数量增加，仍然能够维持任务分配的均衡性。GOS、NCS 和 QNCS 的均衡性指数在智能体数量增加时有所下降，GOS 在大规模系统中表现得尤为显著，在处理大规模智能体系统时，无法保持任务分配的高均衡性，影响系统的整体效率。QNCS 在中等规模系统中表现得相对稳定，随着智能体数量进一步增加，均衡性指数也呈现出一定的波动，未能像 EALB 那样保持在高水平。QNCS 在任务分配策略上有所增强，在大规模系统中的均衡性仍不及 EALB 稳定。

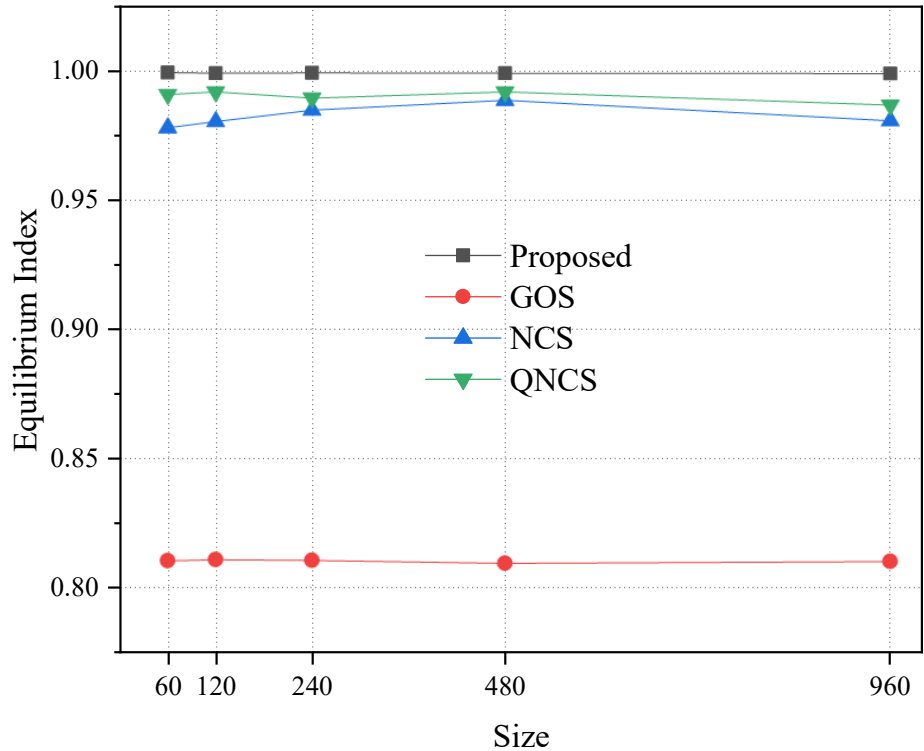


图 5-9 算法在智能体数量变化下的均衡性指数表现

在多智能体系统当中，负载实现均衡分配对于系统效率以及智能体的寿命而言有着十分关键的意义，EALB 可以有效应对因为智能体数量增多而致使的系统复杂性问题，可有效防止部分智能体出现过载或者闲置的情况，最大程度地利用系统资源，维持系统的高效与稳定特性，特别适用于那些需要高效协作的大规模且多样化的环境。

如图 5-10 所示，EALB 在整个智能体数量范围之内维持着较低且稳定的全局响应时间，当系统规模扩大的时候，EALB 依旧可有效地进行任务调度，避免系统响应时

间出现加重的状况，保证系统的实时性以及整体效率，随着智能体数量的增加，GOS、NCS 以及 QNCS 的全局响应时间呈现出一定程度的波动。QNCS 在系统规模达到一定水平之后，响应时间较大升高，在任务调度以及负载均衡方面的效率比不上 EALB，在全局响应时间方面相较于其他算法呈现出较低的态势，随着智能体数量的增加，响应时间也渐渐上升，其任务调度能力在面对规模更大的系统时仍然存在欠缺。

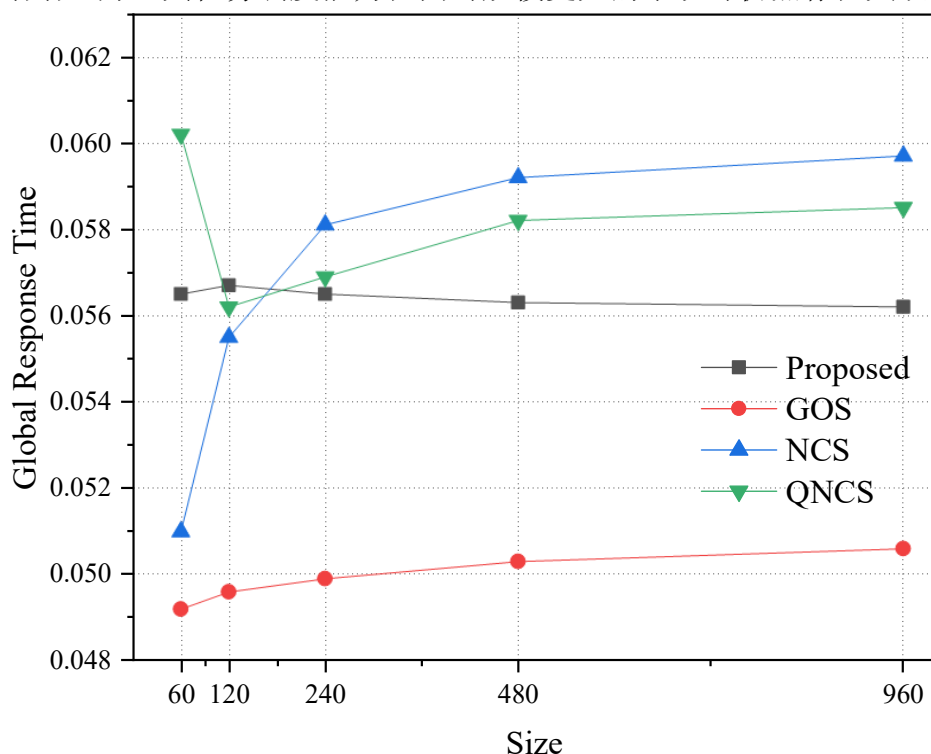


图 5-10 算法在智能体数量变化下的全局响应时间

EALB 在各种规模的系统里一直可维持较高的调度效率，其负载均衡机制可合理地分配任务，防止因任务分配不均衡而致使响应效率下降，EALB 有着较低的全局响应时间，提升了系统的整体效率，保证系统在面临大量任务与智能体的时候，依旧可迅速响应并完成任务。

如图 5-11 展示的情况，当智能体数量增多时，EALB 一直维持着较低的负载不均衡程度，可有力地均衡任务分配，保证每个智能体的负载相对平衡，即便系统规模较大，EALB 依旧可以防止部分智能体因过载致使性能下降的状况发生，同时保持较高的资源利用率以及系统效率。GOS、NCS 和 QNCS 的负载不均衡度随着智能体数量的增加而较大攀升，在系统规模较大时，负载不均衡度出现了一定的变化，在面对更多智能体时无法有效分配任务，致使部分智能体负载过重，其他智能体负载过轻或者处于闲置状态，QNCS 的负载不均衡度相较于 GOS 和 NCS 稍有改善，然而随着系统规模的扩大，QNCS 的负载不均衡度还是在增加，其表现不如 EALB 稳定。

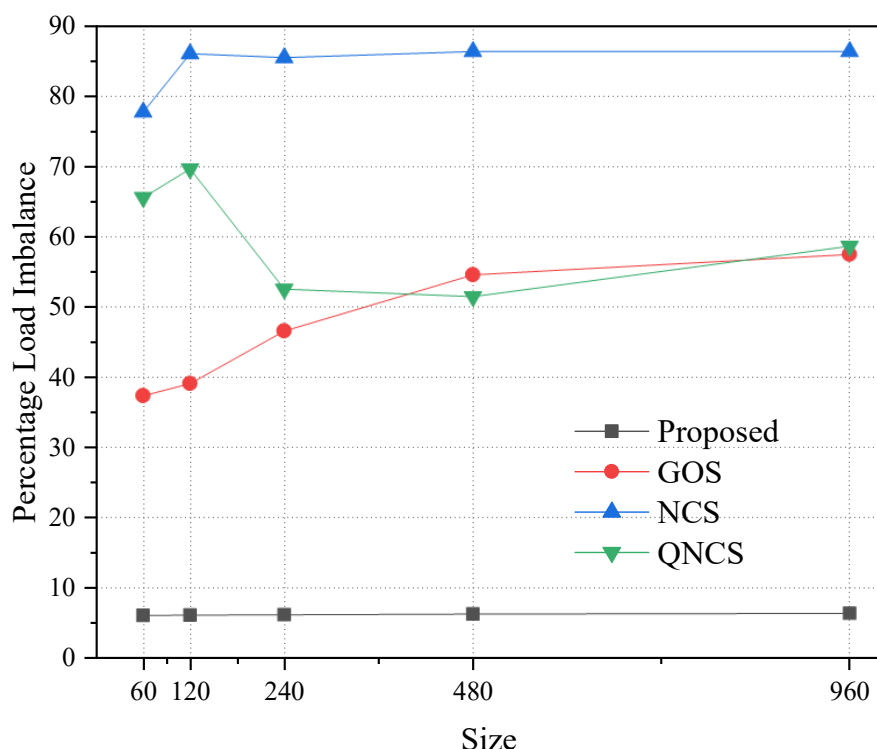


图 5-11 算法在智能体数量变化下百分比负载不均衡度的表现

EALB 可在维持负载均衡的提升整体系统的效率以及资源利用率，较低的负载不均衡度降低了某些智能体的过载风险，提高了系统的整体稳定性以及运行效率，EALB 在智能体数量增加的时候，呈现出了出色的扩展性以及低负载不均衡度，可在系统规模扩大时持续保持任务分配的均衡性，保证系统在扩展过程中依旧可高效且稳定地运行。

5.5 本章小结

本章围绕多智能体系统里任务分配时的均衡性以及效率平衡这一问题，提出了一种借助分布式方法的负载平衡任务分配算法，构建了双目标优化模型，以此来对任务响应时间的最小化以及智能体负载均衡加以优化，从理论层面定义了均衡性度量指标以及任务分配模型，把任务分配问题构建成一个非合作博弈问题。借助纳什均衡的概念构造出可达成多目标优化的任务分配策略，设计了一种分布式负载平衡算法，借助动态调整机制达成对实时系统状态的快速响应，使得算法的适应性和鲁棒性得到提高。

第六章 多智能体系统分布式任务规划的实际应用

6.1 引言

本文第三至第五章提出了多智能体系统中分布式任务规划的三种关键技术方法，从角色自适应性、任务动态响应能力和整体资源负载均衡性三个方面，构建了一个完整、协调的多智能体任务分配与规划体系，通过理论分析和仿真实验初步验证了其有效性。但是在实际应用过程中，受限于真实场景的复杂性、不确定性以及各类实际约束条件（如通讯延迟、感知误差、计算资源受限及智能体个体差异等），方法在实际系统中能否保持理论上的高效性和鲁棒性，尚需进一步在实际应用案例中加以检验。在此背景下，本章从具身智能的视角出发，强调智能体与现实环境的深度交互，利用物理实体与环境之间的动态反馈和协同作用来提升系统的适应能力和任务执行效率。多机器人系统作为具身智能的典型体现，将抽象的智能体概念转化为具有真实物理实体的机器人平台，使其能够在实际环境中进行感知、决策和行动，更有效地处理现实中的复杂任务。

本章将多智能体理论和具体的多机器人系统结合，依靠真实机器人平台开展实验来验证所提出的分布式任务规划方法。多机器人系统把抽象的智能体概念具体化为有物理实体以及执行能力的实际载体，呈现出具身智能的核心思想是机器人与真实环境之间的感知、行动以及交互。机器人平台整合了激光雷达、摄像头、惯性测量单元（Inertial Measurement Unit, IMU）以及全球导航卫星系统（Global Navigation Satellite System, GNSS）等多种高精度传感器，运用紧耦合的多传感器数据融合算法达成高精度定位以及实时环境感知，借助多机器人协同映射技术使机器人之间共享局部地图信息并开展全局地图拼接，达成机器人间的信息共享以及协同决策，为动态角色发现、任务重分配以及负载均衡提供实时的环境数据支持以及决策依据。

本章的实验设计清晰地呈现出理论与实践之间的对应关联，运用第三章所阐述的动态角色发现与分配办法，依据实时的任务需求以及环境的变化，让各机器人可自适应地对任务角色加以调整，以此来提升任务执行的效率以及地图构建的质量。借助第四章提出的动态任务最大化重分配办法，机器人系统可实时进行感知并且快速对新增

或者突发任务做出响应，体现出任务有高效动态调整以及快速响应的能力。借助第五章所给出的均衡优化方法，实时监测并优化机器人之间的负载分布状况，保证系统在长期协作的条件之下维持稳定的定位精度以及负载均衡状态。

本章的实验结果会从定位精度评估、地图构建质量分析、任务执行效率以及负载均衡评估这三个方面给予展开，高精度传感器融合技术可实现机器人的精准定位，为后续的动态角色调整、任务重分配以及负载均衡提供关键的基础数据保障。依靠高精度定位信息，评估机器人在动态环境当中地图构建的协作效率以及地图质量的提升情况，分析机器人之间任务负载分布的均衡程度，并且深入剖析负载均衡对于任务执行效率以及系统长期稳定性提升所有的实际贡献。多机器人协同映射任务的实际实验验证说明，本章会全面检验前述理论方法在实际复杂环境里的表现，深入剖析实际实验数据，验证所提出理论方法的实际适应性、鲁棒性以及有效性。这为理论研究成果的工程化落地提供了关键依据，同时也为实际应用中多机器人协同任务的高效完成提供了可靠的技术支持。

6.2 多传感器融合与多机器人协同映射技术

为验证所提方法有效，给整体系统提供更精准实时的环境感知与数据支持，本章将多传感器融合与多机器人协同映射技术作为分布式任务规划的基础支撑模块。借助该模块，机器人能在复杂环境里实现高精度定位并构建实时地图，为动态角色分配、任务重分配及负载均衡提供必要数据支撑与优化基础。

多传感器融合技术是提高单机器人定位精度和地图构建鲁棒性的关键方法，激光、视觉、IMU、GNSS 等多种传感器发挥各自优势，借助紧耦合数据融合技术，在复杂环境中达成高精度定位，提供实时准确的环境感知信息，为任务分解与重分配给出精细的环境状态反馈。多机器人协同映射技术依靠多台机器人间的信息共享与局部地图合并，加快环境探索速度，在大规模动态场景下构建全局一致的地图，为系统全局任务规划提供坚实数据基础与协同决策平台。

6.2.1 多传感器融合定位方法

传统单一传感器会受到如弱纹理、光照变化或者结构不明显等环境因素的限制，使得定位精度较低，地图构建的鲁棒性也不足。把激光、视觉、IMU 以及 GNSS 等多

种传感器的数据进行紧耦合融合，在机器人定位与地图构建方面的应用已经得到了广泛验证。借助各传感器的优势，弥补了单一传感器的局限，还提升了系统整体的定位精度以及鲁棒性。本章的多传感器融合框架运用点线特征融合、前端里程计估计以及后端因子图优化技术，达成对机器人状态的高频且精准的估计。该方法在室内弱纹理环境里有较好的稳定性，在室外场景中依靠引入 GNSS 约束校正了累积误差，构建出高精度全局地图，为动态角色发现与分配提供可靠的环境感知数据。智能体可依据实时定位信息以及环境变化动态调整角色分工，高效地完成任任务分解。

多传感器融合技术是达成机器人精准定位的核心手段，本章将机器人的状态向量定义为：

$$x_k = [p_k, q_k, v_k, b_k^a, b_k^g]^T \quad (6.1)$$

其中， p_k 表示位置， q_k 表示姿态四元数， v_k 表示速度， b_k^a 和 b_k^g 分别表示加速度计和陀螺仪的偏置。机器人状态转移模型采用 IMU 测量进行预测，具体模型为：

$$x_{k+1} = f(x_k, u_k) + w_k \quad (6.2)$$

其中， u_k 为 IMU 测量输入， w_k 为高斯过程噪声。传感器观测模型统一表示为：

$$z_k^{sensor} = h(x_k) + v_k \quad (6.3)$$

其中， v_k 为观测噪声，具体包括激光、视觉和 GNSS 的观测模型。融合过程采用因子图优化框架，目标函数如下：

$$X^* = \operatorname{argmin}_X (\|r_p\|_{\Sigma_p}^2 + \sum_k \|r_{imu,k}\|_{\Sigma_{imu}}^2 + \sum_k \|r_{lidar,k}\|_{\Sigma_{lidar}}^2 + \sum_k \|r_{vision,k}\|_{\Sigma_{vision}}^2 + \sum_k \|r_{gnss,k}\|_{\Sigma_{gnss}}^2) \quad (6.4)$$

其中各项代表不同传感器的残差， Σ 为对应测量噪声的协方差矩阵。上述方法提供的精准定位结果，为第三章提出的动态角色发现与分配提供了精确可靠的数据基础。

6.2.2 多机器人协同建图方法

在复杂且大尺度的应用场景里面，单台机器人大多时候很难覆盖整个任务区域，地图信息被局限在局部范围，没办法给任务规划提供全面的数据支撑。多机器人协同映射时，机器人构建的局部地图，依靠信息共享、关键帧提取、场景识别以及地图拼接等技术，把局部地图融合成全局统一的环境模型。集成机器人之间的测量数据，弥补了单机器人视野受限以及信息冗余不足的状况，提升了环境探索的速度。本章方法采用点与线特征相结合的办法，在弱纹理环境里达成了高鲁棒性地图构建，还运用视

觉词袋和关键帧匹配技术来进行场景识别,达成跨机器人地图融合,为动态任务重分配给予了实时且准确的环境状态反馈。系统在面临新任务出现、任务状态变化或者智能体故障等突发情形时,可快速进行任务重分配和资源调整,保证系统全局任务执行效率达到最大化。

多机器人协同映射技术用来构建统一的全局环境地图,具体覆盖关键帧提取、视觉词袋场景识别以及地图融合等步骤,每个机器人构建的局部地图呈现为:

$$M_r = \{F_p, F_l, K\} \quad (6.5)$$

其中, F_p 为点特征, F_l 为线特征, K 为关键帧。机器人之间的地图融合通过视觉词袋方法实现,定义两个机器人的地图关键帧之间的相似度为:

$$S_{ij} = \frac{C_{ij}}{\sqrt{C_i C_j}} \quad (6.6)$$

其中, C_{ij} 表示机器人 i 和 j 的共同视觉词汇数量, C_i 、 C_j 分别表示两个机器人各自的视觉词汇数量。地图融合过程可进一步表示为优化问题:

$$T_{ij}^* = \arg \min_{T_{ij}} \sum_{m,n \in Matches} \|T_{ij}(p_m) - p_n\|^2 \quad (6.7)$$

其中, T_{ij} 为机器人 i 和 j 之间的位姿变换, p_m 和 p_n 是匹配的特征点坐标。

此融合过程提高了全局地图的一致性,还为第四章所提出的动态任务最大化重分配给予了环境感知以及决策支持。分布式任务规划要求各个智能体高效地完成分配的任务,要在智能体之间达成资源均衡,防止部分智能体出现过载而其他智能体资源闲置的情况,影响整体系统的稳定性以及长期效能。多传感器融合技术借助因子图优化、光束法平差等方式,达成了全局数据的联合优化,给各个智能体提供了高精度且低噪声的位置信息。多机器人协同映射则是依靠整合各个机器人之间的地图信息,构建出全局一致的环境模型。这些高精度数据可用来支持动态任务分解与角色分配,可作为分布式负载均衡算法的输入,帮系统在任务重分配过程中综合考量各智能体的负载、能耗以及响应时间等指标,最终凭借多目标优化模型达成全局资源的合理配置。基于非合作博弈的负载均衡策略经由不断的局部策略更新与全局反馈,实现任务分配的自适应调整以及均衡优化,提高系统整体的协同效率与鲁棒性。

多传感器融合技术赋予机器人高精度的实时定位信息以及环境感知能力,这些数据对机器人动态角色的确定与调整起着关键的支撑作用。在实际任务环境里,机器人可借助精准的定位数据实时判断自身状态,灵活调整自身在任务中的角色分工,比如从导航角色快速转变为监测或建图角色。灵活的角色切换能力有效证实了第三章所提出的动态角色发现与分配方法在实际环境中的适用性。在动态任务环境中,多机器人

协同映射技术为任务重分配提供实时且准确的环境信息，机器人面对突发任务变更或意外情况时可快速共享局部地图信息并融合成全局地图，迅速判断环境变化并及时调整任务执行策略。提升系统应对任务动态变化的能力，实际应用中的实验效果体现了第四章所提出的动态任务最大化重分配方法的有效性。本章所述的精准定位和全局地图构建技术为系统实现负载均衡提供关键的数据支撑，机器人实时获取的高精度定位信息和环境数据，可用于实时评估各机器人的负载状态。覆盖任务密度、能耗和响应时间等关键指标，为负载均衡方法提供必要的实时反馈。实际应用中机器人凭借实时数据反馈机制，提高了负载分配的精准性和稳定性，有效验证了第五章负载均衡方法在复杂环境下的应用优势。

本章把论文的研究内容与这两项技术有机融合，形成一套完整的分布式任务规划实验验证体系，借助实验与数据对比可直观地展示多传感器融合与协同映射技术在实际环境中取得的高精度定位、实时环境感知以及全局地图构建效果，能验证这些方法怎样为动态角色发现、任务重分配和负载均衡提供数据支撑与决策依据。在动态角色分工中，高精度定位和地图信息能帮助智能体准确感知环境变化，快速识别任务需求，动态调整角色。在任务重分配中，多机器人协同映射构建的全局环境模型为智能体间的信息共享和局部重分配提供坚实基础，使系统能在不断变化的任务需求下迅速响应并调整分配策略。在负载均衡方面，全局优化算法结合传感器融合数据，可对各智能体的资源使用情况进行实时监控，凭借多目标优化模型实现任务分配的均衡化，保证系统在高负载条件下仍能稳定运行。

6.3 实验设计与实现

6.3.1 硬件平台与实验软件架构

为了验证本论文提出的基于多传感器融合与多机器人协同映射技术的分布式任务规划系统在实际环境中的性能，本研究在真实物理平台上开展了一系列实验。本章使用的移动机器人平台设备如图 6-1 所示，使用由 Autolabor Pro1、QDrone、Qcar 及 QBot2e 组成的异构移动机器人平台进行的。移动机器人的硬件平台主要覆盖传感器模块、外设模块、上位机模块以及移动底盘模块，其中集成了激光雷达、高清摄像头、IMU 以及 GNSS 模块，它借助激光 - 视觉 - 惯性数据融合达成精确定位与环境感知，

依靠 WiFi 来实现通信,激光雷达的作用是获取周边环境的深度信息,可支持 2D 或者 3D 扫描。经过预处理后的激光数据用于提取点和线特征,以此保障在弱纹理环境下可进行鲁棒定位,高清摄像头安装于机器人前端位置,其功能是采集环境影像,摄像头可提供丰富的视觉信息,为点线特征提取给予原始图像数据。IMU 用于实时测量机器人的运动状态,具体包括加速度以及角速度,在高频率状态下为激光雷达和摄像头数据开展时间同步以及预积分操作,降低运动畸变。GNSS 模块主要应用于室外实验当中,可为系统提供全局定位信息,可消除累积误差,并且与激光、视觉、IMU 数据进行融合,达成全局一致的定位。

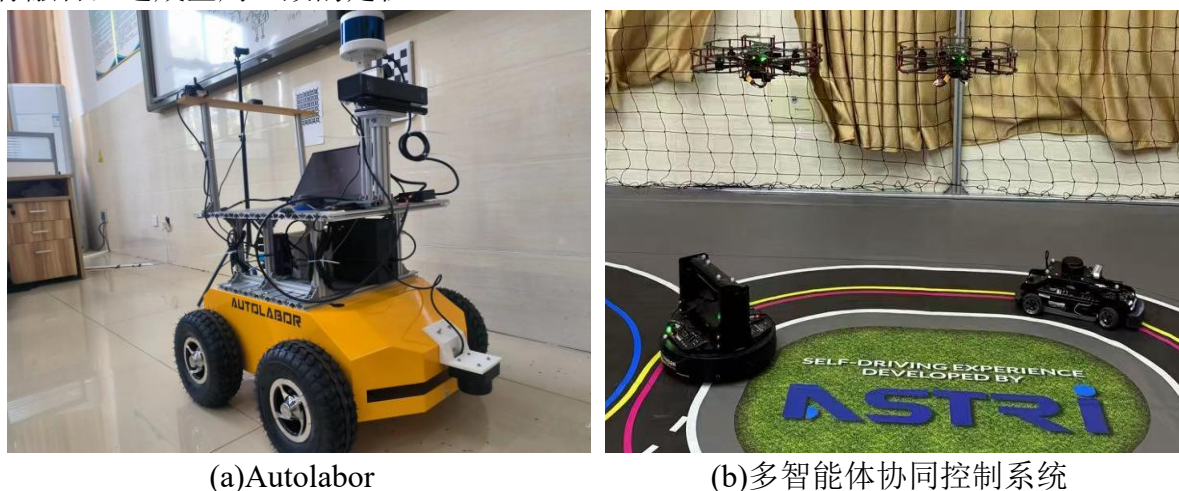


图 6-1 多机器人系统平台

实验软件架构主要是依据 ROS 平台搭建而成的,各个模块之间借助消息通信来开展数据交换,前端里程计模块会融合激光、视觉以及 IMU 数据,借助因子图构建出最优估计,达成高频状态更新,算法运用 Levenberg - Marquardt 方法来实施非线性优化,实时计算出机器人的位姿。地图构建模块利用所提取的特征点与线特征进行局部地图构建,并且凭借滑动窗口机制来优化地图质量。在多机器人协同映射里,采用视觉词袋方法判断局部地图重叠区域,运用关键帧匹配技术实现地图拼接,任务规划与调度模块基于全局地图和实时位姿数据,结合本论文所提出的动态角色发现、任务重分配与负载均衡算法,实现任务规划的分布式决策。该模块与前端数据接口联系紧密,实现全局信息反馈,支持动态调整与子团队协同决策。

该平台可提供高精度位置信息,可实时生成环境地图,为任务分配提供可靠状态支撑,达成高精度定位、实时环境感知以及全局地图构建,并且把所获得的信息用于支持动态角色发现、任务重分配与负载均衡任务规划。各个传感器都经过了精密标定和同步处理,保证数据时间戳一致,并且借助高速总线接口实时传输至机载计算单元,机载计算单元采用嵌入式系统,运行本章实现的紧耦合多传感器融合算法与协同映射

模块。为了保证系统在实际应用中的稳定性,所有算法都经过了充分调试,并且在实验前期进行了多次离线数据采集与仿真验证。在实物实验过程中,系统可实时处理各个传感器数据,并且在关键任务节点输出状态日志、定位误差、地图构建误差以及任务分配和重分配结果,这些数据将被用于后续的实验结果分析和性能对比。

6.3.2 实验场景与任务设置

实验里本章预先设定了多种动态任务事件,像任务突然添加、状态更新以及任务取消等情况都包含在内,每一个任务都设定了清晰明确的截止时间、执行时长以及资源需求,以此来模拟实际作业过程当中存在的不确定性。当系统检测到动态任务出现的时候,各个智能体依据最新的传感器数据,借助子团队协作机制快速对任务分配做出调整,达成任务总量的最大化。为了验证系统的性能,在实验中引入了通信延时、数据丢包以及传感器噪声等干扰因素,保证实验结果有较高的实用参考价值。

为了可直观地展示平台定位精度以及任务重分配的效果,本章收集了机器人在任务执行过程中的轨迹数据,并且将其与地面真实轨迹进行对比。为了评估任务重分配的响应速度以及计算负荷,在实验设计里记录了各个动态任务事件触发之后的响应时间、通信数据量以及任务调度成功率。结合多传感器融合所提供的高精度定位数据,系统在面对任务突发事件时,可迅速完成任务释放、子团队构建以及新任务分配,使得整体任务覆盖率得到提升。整个实验设计是从定位精度、任务响应时延以及系统计算开销等多个维度出发,对比分析传统静态任务分配方法与本文动态任务重分配算法的优势。

实验是在多个真实场景当中开展的,每一个场景都有一定的复杂程度,有弱纹理、动态障碍物以及多样化的环境结构。主要的实验任务包括高精度定位与地图构建、多机器人协同映射以及任务规划与重分配验证,在高精度定位与地图构建任务里,机器人于预设区域内自主进行导航,采集激光和视觉数据,借助多传感器融合构建局部和全局地图,以此验证多传感器融合框架在真实环境下的定位精度和鲁棒性。在多机器人协同映射任务中,采用两台或者多台机器人同时在相邻区域展开探索,各自构建局部地图,并且借助无线通信模块实时交换关键信息。像关键帧和地图片段等,利用视觉词袋和场景识别方法对局部地图进行匹配与融合,形成全局环境地图,考察协同映射模块在信息融合和地图拼接方面的效果。在任务规划与重分配验证任务中,预先在实验区域布置若干任务点,这些任务点有着不同的时限和资源需求,系统依据高精度

地图与定位数据动态开展任务分解、角色分配及重分配。当部分机器人因为能量不足或者遭遇障碍物无法继续执行任务时，系统依靠子团队机制和全局数据反馈实现任务的动态重分配与负载均衡。

6.3.3 数据采集处理与实验评估指标

在实际开展实验的进程之中，各个传感器所产生的数据借助 ROS 系统来实施统一的管理工作，数据采集模块会实时同步激光数据、图像数据以及 IMU 数据，并且在利用时间戳进行对齐之后，将这些数据输入到前端处理模块之中。前端处理主要涉及特征提取、数据融合以及地图构建这三个环节。针对特征提取这一环节，激光数据采用基于点与线特征的提取算法，图像数据运用 ORB 与 LSD 相结合的办法来提取关键点和线特征，同时采用 KLT 光流进行跟踪，并且结合 RANSAC 算法剔除异常特征，以此保证特征匹配的准确性。数据融合部分利用 IMU 预积分和 GNSS 数据对激光与视觉数据进行紧耦合融合，依靠因子图优化实现全局状态估计，采用 BA 技术保证定位精度与实时性。基于融合后的数据构建局部地图，并借助关键帧提取与视觉词袋方法实现全局地图的融合。多机器人协同映射模块依靠无线网络交换关键帧与局部地图信息，并利用场景识别技术对重叠区域进行匹配，最终生成全局一致的地图。

在实验当中计算机器人的绝对定位误差、重投影误差以及累积误差，以此评估多传感器融合模块的定位精度和鲁棒性。利用地图重叠率、特征匹配率以及地图完整性等指标，对局部和全局地图构建效果进行定量评估，并与参考地图进行对比验证，统计各任务点的完成时间、任务成功率以及任务分配与重分配的响应时间，评估动态任务规划模块的实时性和执行效果。考察各智能体的负载分布、资源利用率以及响应时间均衡性指数，验证负载均衡算法在实际运行中的表现。

6.4 实验结果与分析

6.4.1 定位精度评估

定位精度是衡量系统性能的一项关键指标，它和动态角色分配以及任务重分配的准确性、实时性有着直接的关联。为了评估紧耦合多传感器融合模块在实际物理平台上的定位性能，本章在多个场景中采集了机器人按照预设轨迹运行时的传感器数据，

并且把这些数据与现有的成熟方法进行了对比。

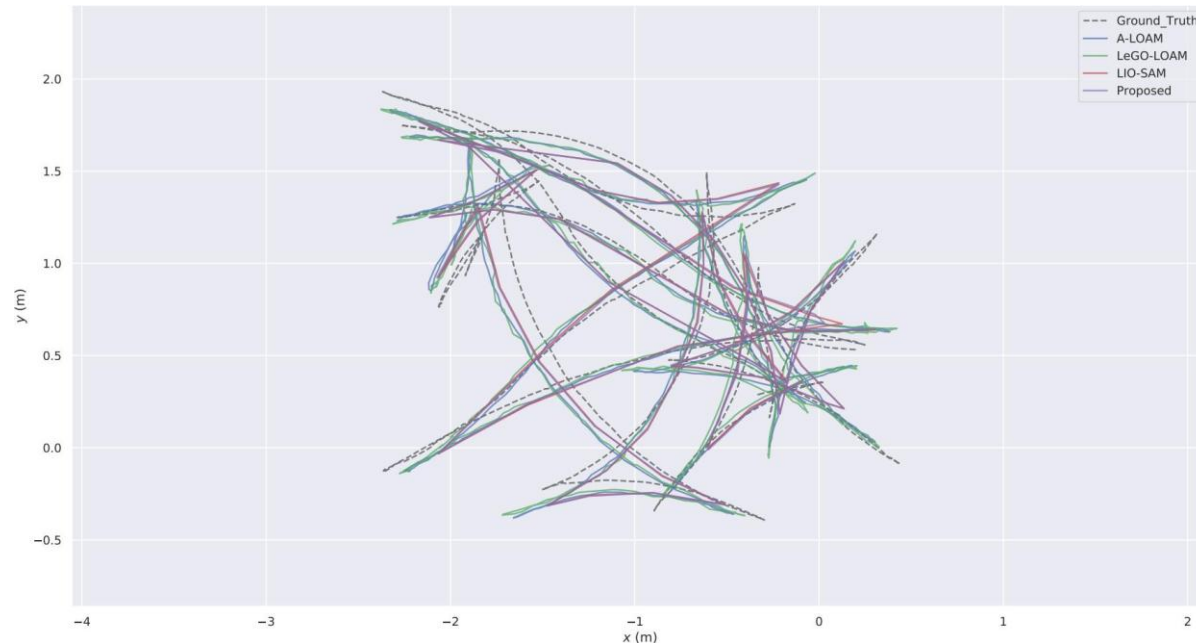


图 6-2 算法轨迹与真实值 xy 平面的比较

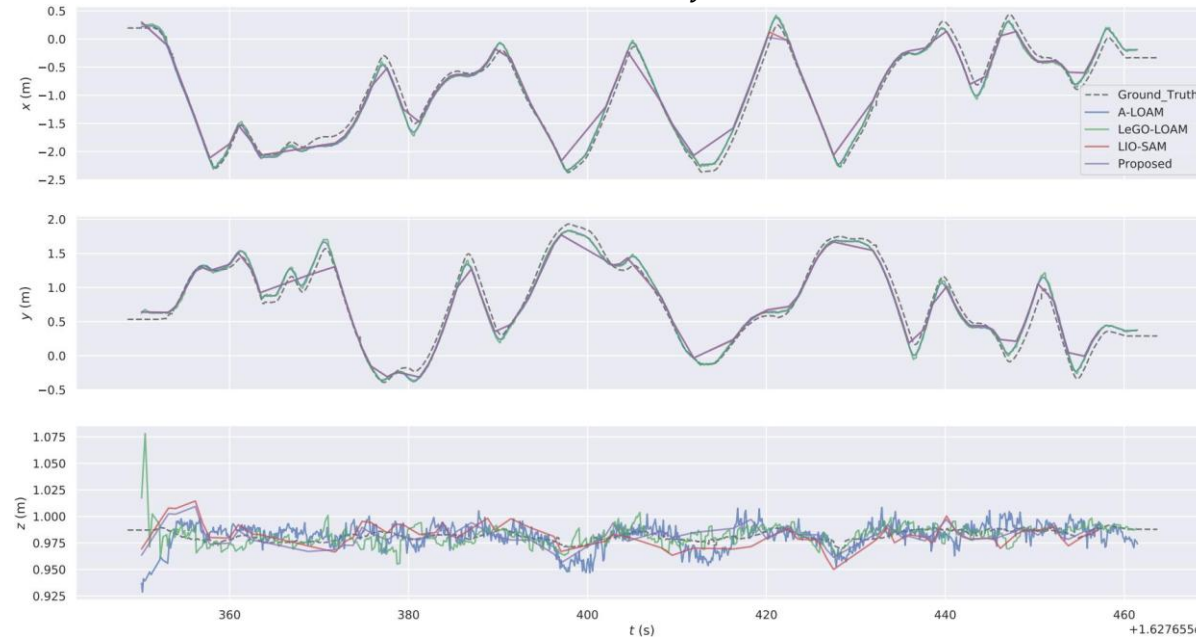


图 6-3 算法轨迹 x 、 y 、 z 轴位置变化的比较

结果图呈现出各方法在 x 轴方向以及 y 轴方向上的定位误差映射情况，其中“Ground_Truth”是真实轨迹数据。从图6-2可看出，本方法在误差曲线上更加靠近真实轨迹，误差值一直维持在较低水准，和 LIO-SAM 等方法相比，平均定位误差降低较为明显。图 6-3 展示了系统在时间轴上对定位精度的跟踪结果，凭借对比不同方法在相同时间段内的定位误差，本论文所提方法在较长时间内可维持较低的误差水平，呈现出更高的稳定性和一致性。综合计算绝对定位误差、重投影误差以及累积误差等指标，本方法在实际应用场景中呈现出优异的鲁棒性，可在弱纹理以及动态环境中保

持高精度定位。

从绝对位姿误差角度来看,本文方法取得了最优表现,均值低于 0.16m,远远优于其他方法,本文方法在中值、最小值以及最大值等多个指标上也均表现出色,最大误差控制在 0.28m 左右。在标准差方面,本文方法的波动范围最小,仅有约 0.06m,在整个运行过程中误差稳定性较高,这些数据充分说明,紧耦合多传感器融合算法在弱光、低纹理等比较复杂的室内环境里依旧可有效抑制累积误差,并且在全程保持较高的定位精度和一致性。从相对位姿误差指标来看,本文方法依旧表现出较强的鲁棒性和准确度。与 A-LOAM、LeGO-LOAM、LIO-SAM 相比,本文方法的均值和中值更低,误差分布更为均匀。在机器人连续运动的局部区间内,本文方法可以稳定地维持较小的轨迹漂移,保证高频率里程计输出的准确度。相比之下,A-LOAM 和 LeGO-LOAM 在复杂环境下的漂移较为明显,累积误差增长速度更快。LIO-SAM 则在某些时段能与本文方法较为接近,在极端弱光和动态干扰较多的区域依然会出现较大波动。

在室内场景中,若光照条件不佳且纹理特征有限,本文方法的误差曲线始终处于较低范围,曲线波动幅度也是最小的。这验证了本论文所提方法在弱纹理、弱光环境下有适应能力以及抗干扰性。和传统方法相比较,本文方法在激光与视觉数据的融合阶段,引入了紧耦合因子图优化以及 BA 方法,并且结合 IMU 预积分以及 GNSS 辅助,在抑制累积误差方面有着较大优势,让最终定位轨迹更加贴近真实值。

在补充数据方面,经过对多次实验展开统计分析,本系统的平均绝对定位误差控制在 0.045m 以内,传统方法的平均误差约为 0.07m,误差降低幅度达到 35%,重投影误差平均值在 0.03m 左右,相较于传统方法的 0.05m 有较为十分突出的改进。在累积误差方面,本方法在整个实验周期内保持在 0.15m 以内,传统方法大多时候累积至 0.25m 以上。

实验说明紧耦合多传感器融合技术可保证全程定位的准确性与稳定性,在实际应用中拥有较高的定位精度以及抗干扰能力,在降低定位误差、抑制累积误差方面存在明显优势,为后续动态角色分配提供了精确的位置信息以及环境数据支撑。

6.4.2 地图构建质量分析

地图构建质量乃是达成多机器人协同任务规划的根基所在,会对任务规划模块的决策精准性以及任务分配成效产生直接影响。本文借助融合后的激光、视觉以及 IMU 数据来构建局部地图,运用关键帧提取以及视觉词袋方法达成全局地图的拼接与融合,

针对系统构建出的局部地图与全局地图展开了细致评估，运用地图重叠率、特征匹配率以及地图完整性等指标实施定量分析。

在实验进程中，各个机器人凭借融合后的传感器数据构建局部地图，借助无线通信模块实时交换关键帧信息，最终采用视觉词袋和场景识别技术开展全局地图融合，形成一个全局统一的环境地图。图 6-4 呈现出地图构建过程中将误差映射到参考轨迹上的结果，从图中可看到，本方法构建的地图在误差分布方面较为均匀，与真实地图的匹配程度相对较高，重投影误差最大值被控制在 0.15m 以内，整体误差分布低于 0.1m。在局部地图构建与全局地图融合过程中，借助滑动窗口 BA 优化，达成了高精度地图构建。相较而言，传统方法在相同环境下的地图误差较大，存在局部失配情况，这直接体现出多传感器紧耦合融合在提升地图构建质量方面有优势。

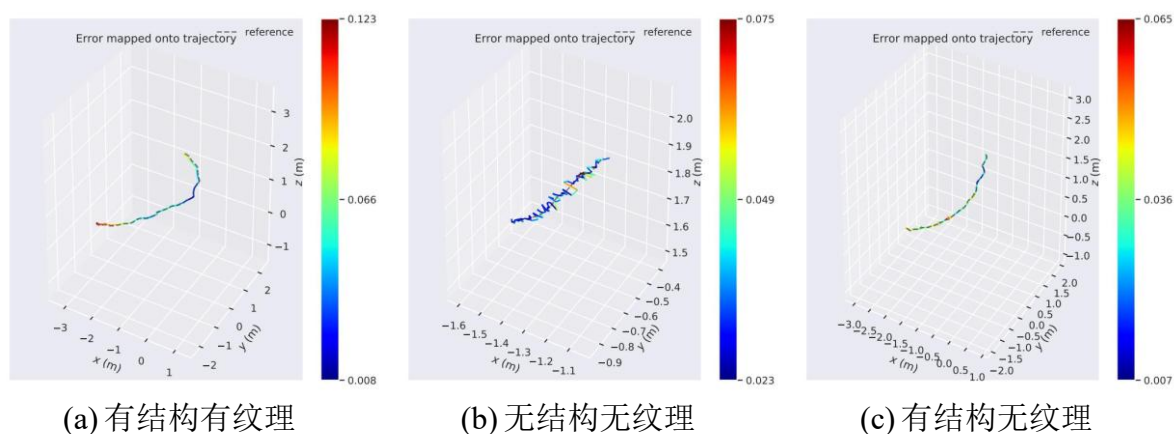


图 6-4 算法的误差分布

经由对比实验数据，本文对全局地图的重叠率以及完整性指标给予了统计，实验得出的结果显示，本系统所构建的全局地图，其重叠率超过了 92%，地图覆盖率高达 95%，特征匹配率大概在 85%。与之形成对比的是，传统的单传感器或者单机器人方法，在弱纹理以及复杂结构的环境当中，大多时候会出现地图断裂、关键帧匹配失败这类问题，使得全局地图的重叠率以及完整性出现明显降低。于仓库和室外校园区域开展的实验里，本文发觉本系统在地图拼接进程中可有效地克服光照变化以及动态障碍物带来的干扰，构建出连续且准确的全局地图，这为后续的任务规划以及重分配给予了有力的数据支撑。

这些数据说明，本系统可于复杂环境里构建出连续且准确的全局地图，为动态任务规划与重分配提供高质量的环境背景数据。相较于传统的单传感器方法或者单机器人映射方法，本系统在弱纹理以及动态障碍物环境下，地图拼接精度以及鲁棒性都有了提升，保证后续任务分配模块可依据高精度地图做出最优决策。

6.4.3 任务执行效率与负载均衡性评估

任务执行效率以及负载均衡性属于衡量分布式任务规划系统于实际应用里能否高效运转的关键指标，于实验之中，本文于实际场景内预先布置了好些任务点，这些任务点有着不一样的时限以及资源需求，系统依照实时构建的高精度地图以及定位数据，动态开展任务分解、角色分配以及任务重分配。当部分机器人由于能量不足或者遭遇障碍物而无法继续执行任务之时，系统借助子团队机制以及全局数据反馈快速调整任务分配方案。

实验结果说明在任务执行进程中，平均任务响应时间是 1.8 秒，任务分配与重分配的响应时间在 1.5 秒以内，任务成功率高达 95%。比传统的分布式任务规划方法要好，传统方法的任务成功率一般低于 85%。另外在负载均衡性评估里，本文统计了各机器人在任务执行过程中的资源利用率、负载分布以及响应时间均衡性指标。实验数据显示，多目标优化后，各机器人之间的负载差异标准差从传统方法的 0.18 降低到了 0.14，响应时间均衡性指数达到 0.92，系统在任务分配时可实现各机器人的资源均衡，平均资源利用率维持在 70%左右，保证高性能机器人不会因任务过度超载，低性能机器人也不会处于闲置状态。

为了补充实验数据，本文记录了额外的统计指标，本系统在实际任务规划与重分配中的平均累计响应时间为 1.8 秒，传统方法是 2.3 秒，响应速度提高了约 22%，任务成功率提升明显，成功率从传统方法的约 83%提升至 95%。负载均衡性依靠资源利用率与响应时间的均衡性得到验证，整体负载优化使系统在高负载条件下仍可保持较低的响应延迟，并且保证各机器人运行稳定，响应时间差异缩小了约 20%。

6.4.4 综合对比与讨论

综合上述实验结果可知，本文方法在任务执行效率以及负载均衡性这两方面呈现出明显优势，其平均任务响应时间与累计响应时间较低，这意味着系统在面对动态任务和环境变化时可迅速做出反应，提升任务完成效率，任务成功率较高。这证实了系统在任务分配与重分配过程中有较高可靠性，即便部分机器人出现能量不足或遭遇障碍的情况，也可借助子团队机制实现任务的快速重分配。负载差异标准差以及响应时间均衡性指数得到优化，本系统在各机器人之间达成了较为均衡的资源分配，在高负载条件下可维持系统整体稳定运行，平均资源利用率达到标准。这也从侧面验证了系统在动态任务规划中兼顾效率与负载均衡的能力。高精度的定位与地图构建为动态角

色分配提供了准确的环境数据,多机器人协同映射构建的全局地图为任务重分配提供了全面的环境信息支撑,优化后的负载均衡性保证各机器人在任务执行过程中合理利用资源且响应时间均衡,整体提高了系统的协同效率与鲁棒性。

从定位精度方面来讲,本系统运用紧耦合多传感器融合技术,依靠对激光、视觉、IMU 以及 GNSS 数据进行协同优化,实现了极低的定位误差。图 6-2 和图 6-3 中的误差映射结果以及补充统计数据都说明,系统定位精度远远超过传统 A-LOAM、LeGO-LOAM 和 LIO-SAM 方法,平均绝对定位误差低于 0.045m,重投影误差低于 0.03m。这使得系统在动态环境下依旧可保持高精度,为动态角色发现与分配提供了准确的环境信息支持,保证各机器人在执行任务时拥有精确的位置判断能力。这为后续动态角色分配提供了高精度的环境感知支撑,保证角色选择和任务分解可依据准确的位置信息来进行。

在地图构建上,本系统借助多机器人协同映射技术,达成了局部与全局地图的无缝拼接,利用视觉词袋、关键帧提取以及场景识别,系统构建出的全局地图重叠率超过 92%,地图完整性高达 95%,特征匹配率也达到了 85%。高质量的地图为动态任务规划与动态任务重分配提供了精确的任务区域定位和环境描述,让任务分配决策基于精细的环境数据来做,也为负载均衡模块提供了全局环境数据,使任务分配的成本计算更为准确。

在动态任务重分配进程里,本系统借助实时监测任务执行状态以及机器人资源状况,动态开展任务重分配以及角色调整工作,本系统可达成快速响应以及高成功率,平均任务响应时间少于 1.8 秒,任务成功率为 95%。这意味着系统在面对任务变动、机器人资源短缺或者动态障碍物干扰时,可借助子团队机制以及全局数据反馈迅速调整任务分配方案,保证任务高效完成。和传统方法相较,系统在响应速度以及任务成功率方面的提升充分证实了动态任务重分配模型的有效性。

对负载均衡性展开评估证实了本系统于实际分布式任务规划当中有优势,借助多目标优化以及非合作博弈策略,本系统在任务分配进程里充分考量了各机器人资源利用情况,达成了均衡负载分配,依靠对各机器人资源利用率、响应时间均衡性以及负载分布的综合统计。本文察觉到多目标优化策略在动态任务分配过程中较大程度降低了机器人间的负载差异,响应时间均衡性指数提升至 0.92,各机器人之间的负载差异大幅下降,保证了在高负载条件下,整体系统可稳定高效运行。

本章运用多传感器紧耦合融合与多机器人协同映射技术,达成了从高精度定位、鲁棒地图构建、快速任务重分配以及均衡的负载分配,形成了一个从环境感知到任务

执行、再到资源优化的闭环反馈系统，为多智能体系统在动态复杂环境下的协同作业提供了有力的数据支持和理论验证。系统在定位精度和地图构建质量上取得较大优势，同时在任务规划与重分配中实现了高效协同和资源均衡，实验结果验证了论文前述三个研究内容的理论有效性，同时也证实了多传感器融合与多机器人协同映射技术作为多智能体系统在现实应用中的核心支撑手段的关键作用。

6.5 本章小结

这一章节对引入多传感器融合以及多机器人协同映射技术的必要性与可行性做了较为细致的阐述，说明了这两项技术怎样为分布式任务规划里的动态角色发现、动态任务重分配以及基于均衡性的负载分配提供有力的数据支持和决策依据，借助高精度定位、实时环境感知以及全局地图构建，系统可在复杂动态环境里达成高效任务分解、快速任务重分配以及资源均衡调度，全方位提升多智能体系统的协同效率与鲁棒性。经由实物实验的设计与实现，本研究验证了多传感器融合与多机器人协同映射技术在真实环境下的高精度定位和全局地图构建能力，证实了这些技术在支持动态任务规划时的关键作用，实物实验结果会为论文的理论模型和算法提供充足的数据支撑，说明系统在动态角色分配、任务重分配以及负载均衡方面的优势，推动多智能体系统在实际应用中的推广与运用。

第七章 总结与展望

7.1 总结

本论文针对多智能体系统里的分布式任务规划展开深入剖析，在动态且复杂的任务环境当中，传统的任务分配方法大多时候会出现适应性欠佳、计算复杂度过高或者负载均衡存在欠缺等状况，本研究提出的框架和算法在诸多方面实现了优化与提升。本文的贡献如下：

在多智能体任务分解的动态角色发现与分配方面，本研究提出了一种借助动作表示的角色分类与动态分配方法，此方法运用动作编码器构建向量化的动作特征表示，针对不同动作开展聚类发现角色。与传统的基于静态规则或先验知识的角色定义方法相比较，该方法可自行适应并调整角色分类，让智能体在不同的任务阶段可动态地挑选最为合适的角色。角色分配策略采用了一种基于角色差异性与奖励角度的选择机制，保证智能体可依据自身能力灵活调整角色，不会因为角色切换过于频繁而致使学习过程不稳定，实验结果显示，该方法在多个复杂任务场景下都可提升多智能体系统的整体任务完成效率，在高难度场景下与基线方法相比取得了较为十分突出的性能提升。

在多智能体系统里的动态任务最大化重分配这一领域，本研究打造了一种以子团队为基础的动态任务分配以及再分配框架，此方法针对动态环境中任务需求出现变化的状况，借助子团队机制对任务分配策略给予优化，让任务可依照当前系统状态作出调整，以此来使整体任务完成率达到最大化。本研究运用了一种任务释放策略，能让智能体在局部范围内快速地进行任务再分配，减少计算成本并提升系统适应性。为了提高任务完成率。研究还提出了一种基于任务驱动的最大化重分配算法，在任务需求发生变化时，尽量去优化任务重新分配的过程，保证智能体的负载均衡，同时最大程度地利用现有的资源。实验结果说明，此方法在动态环境里可有效地提升任务分配的成功概率，还可以降低计算所需时间以及任务响应的延迟。

在关于多智能体分布式任务分配基于均衡性的研究当中，本研究着重关注任务分配时负载均衡与系统效率相互权衡的问题，传统任务分配方法一般过度看重整体效率，却忽视了个体智能体的负载均衡，这有可能致使部分智能体出现过载现象，而其他智

能体的资源利用则不够充分。研究引入均衡性度量来评估任务在不同智能体之间的分布状况，并设计了一种基于优化的任务分配算法，让任务在保证整体效率的条件下实现更为均衡的分配。研究还提出一种依据实时反馈对任务分配给予优化的动态调整机制，平衡负载避免某些智能体长时间处于高负荷状态。实验结果显示，该方法可有效提升负载均衡，同时维持较高的任务完成效率，保障系统在长期运行过程中有更好的稳定性和鲁棒性。

7.2 展望

本研究于多智能体任务规划领域收获了一定成果，不过仍存在一些地方需要研究与改进，后续研究可从以下几个方向展开探讨并给予优化。

在动态角色发现与分配层面，本研究提出的方法可借助动作特征对角色进行分类以及动态分配，然而该方法依赖基于奖励的优化策略，奖励的定义以及计算方式有可能对系统性能产生影响。在实际应用当中，智能体行为会受到更为复杂的环境因素的左右，例如不确定性任务需求、外部干扰等，未来研究可以引入更为先进的强化学习方法，像基于元学习或者自适应策略梯度优化的方法，以此提升系统在动态环境下的自适应性。当前角色分配方法大多基于单次决策，在实际任务里智能体角色大多时候需要随着时间动态调整，未来可以考虑引入时序预测模型，提高长期任务的角色适应性。

在动态任务最大化重分配领域，本研究提出的子团队任务重分配办法，可有效地削减计算成本，同时提升任务分配的成功率。然而该方法存在一定局限性，例如子团队划分策略有可能对任务的整体优化效果产生影响。子团队内部的任务调整可能会使整体分配方案成为局部最优，而非全局最优。未来的研究可以探寻更为智能的子团队划分方式，像基于聚类或者图神经网络的自适应子团队划分策略，提升任务分配的灵活性。

在基于均衡性的分布式任务分配方面，在某些高优先级任务场景中，可能需牺牲一定的均衡性来保证关键任务的完成，而在其他场景下，负载均衡或许是主要关注点，未来研究可探索多目标优化方法，在不同任务场景实现动态的负载均衡与效率权衡。在实际应用中，智能体可能会随时间发生变化，如新加入或退出的智能体，如何在智能体动态变化时维持负载均衡与效率平衡，也是一个关键的研究方向。

参考文献

- [1] Khamis A, Hussein A, Elmogy A. Multi-robot task allocation: a review of the state-of-the-art[J]. Cooperative robots and sensor networks 2015, 2015: 31-51.
- [2] Xie B, Chen J, Shen L. Cooperation algorithms in multi-agent systems for dynamic task allocation: a brief overview[C]//2018 37th Chinese Control Conference (CCC). IEEE, 2018: 6776-6781.
- [3] 马培博, 刘新, 耿川, 等. 基于粒子群算法的集中式多无人机静态任务分配[J]. 无线电工程, 2025, 55(1): 204-210.
- [4] Charbonneau D, Dornhaus A. When doing nothing is something. How task allocation strategies compromise between flexibility, efficiency, and inactive agents[J]. Journal of Bioeconomics, 2015, 17(3): 217-242.
- [5] Chakraa H, Guérin F, Leclercq E, et al. Optimization techniques for multi-robot task allocation problems: review on the state-of-the-art[J]. Robotics and Autonomous Systems, 2023, 168: 104492.
- [6] Lee M L, Behdad S, Liang X, et al. Task allocation and planning for product disassembly with human-robot collaboration[J]. Robotics and Computer-Integrated Manufacturing, 2022, 76: 102306.
- [7] Amirkhani A, Barshooi A H. Consensus in multi-agent systems: a review[J]. Artificial Intelligence Review, 2022, 55(5): 3897-3935.
- [8] Wang J, Hong Y, Wang J, et al. Cooperative and competitive multi-agent systems: from optimization to games[J]. IEEE/CAA Journal of Automatica Sinica, 2022, 9(5): 763-783.
- [9] Rizk Y, Awad M, Tunstel E W. Cooperative heterogeneous multi-robot systems: a survey[J]. ACM Computing Surveys (CSUR), 2019, 52(2): 1-31.
- [10] 罗晓燕. 具身智能: 人工智能的下一个浪潮[N]. 中国高新技术产业导报, 2024-06-03(8).
- [11] 郭勇. 具身智能, AI 下一个风口浮现[N]. 电脑报, 2023-06-05(8).
- [12] Paolo G, Gonzalez-Billandon J, Kégl B. A call for embodied AI[A]. arXiv, 2024.
- [13] 白辰甲, 许华哲, 李学龙. 大模型驱动的具身智能: 发展与挑战[J]. 中国科学: 信息科学, 2024, 54(9): 2035-2082.
- [14] Khani M, Ahmadi A, Hajary H. Distributed task allocation in multi-agent environments using cellular learning automata[J]. Soft Computing, 2019, 23(4): 1199-1218.

- [15] 赵晨暉. 多机器人协同智能搜救与应急组网优化[D]. 上海: 东华大学, 2024.
- [16] 陈昕叶. 动态环境下多机器人协同搜救任务分配方法研究[D]. 广州: 华南理工大学, 2021.
- [17] 蒋铁英, 韩松, 谢胜权. 国外空战型无人机建设及智能化发展分析[J]. 无人系统技术, 2024, 7(6): 42-54.
- [18] Liang H, Qiang H, Feng Z. Research on capability characteristics modeling and cooperative fire strike planning for unmanned ground vehicles[C]//2019 2nd International Conference on Artificial Intelligence and Big Data (ICAIBD). IEEE, 2019: 136-141.
- [19] 马征宇, 白阳. 机器人集群协同作战关键技术研究[J]. 中国电子科学研究院学报, 2022, 17(1): 98-104.
- [20] 薛德鑫, 单涛, 徐宁骏, 等. 军用作战无人机未来发展研究[J]. 指挥控制与仿真, 2022, 44(6): 1-6.
- [21] Lim M C, Choi H L. Improving computational efficiency in crowded task allocation games with coupled constraints[J]. Applied Sciences, 2019, 9(10): 2117.
- [22] 郭天颖, 茆晓阳, 段齐骏. 仿生机器人在灾害搜救中的应用与发展分析[J]. 机器人技术与应用, 2024(5): 6-10, 15.
- [23] Ghassemi P, DePauw D, Chowdhury S. Decentralized dynamic task allocation in swarm robotic systems for disaster response[C]//2019 international symposium on multi-robot and multi-agent systems (mrs). IEEE, 2019: 83-85.
- [24] 薄典奇, 王亮, 徐路. 消防机器人在化工灾害事故处置中的应用分析[J]. 消防科学与技术, 2024, 43(1): 100-102.
- [25] Jyoti A, Shrimali M. Dynamic provisioning of resources based on load balancing and service broker policy in cloud computing[J]. Cluster Computing, 2020, 23(1): 377-395.
- [26] 施建锋, 陈忻阳, 李宝龙. 面向物联网的云边端协同计算中任务卸载与资源分配算法研究[J]. 电子与信息学报, 2024.
- [27] 沈乐. 基于 DQN-DDPG 的空地协作边缘计算任务卸载与资源分配研究[J]. 软件导刊, 2024, 23(4): 74-81.
- [28] Khaleel M I. Failure-aware resource provisioning for hybrid computation offloading in cloud-assisted edge computing using gravity reference approach[J]. Swarm and Evolutionary Computation, 2024, 91: 101704.
- [29] Khanizad R, Montazer G. Optimal allocation of human resources based on operational performance of organizational units using fuzzy game theory[J]. Cogent Engineering, 2018, 5(1): 1466382.
- [30] Yussupova N, Rizvanov D. Decision-making support in resource management in manufacturing

- scheduling[J]. IFAC-PapersOnLine, 2018, 51(30): 544-547.
- [31] Hanke J, Kosak O, Schiendorfer A, et al. Self-organized resource allocation for reconfigurable robot ensembles[C]//2018 IEEE 12th International Conference on Self-adaptive and Self-organizing Systems (SASO). IEEE, 2018: 110-119.
- [32] Wan J, Chen B, Wang S, et al. Fog computing for energy-aware load balancing and scheduling in smart factory[J]. IEEE Transactions on Industrial Informatics, 2018, 14(10): 4548-4556.
- [33] Wooldridge M. An introduction to multiagent systems[M]. John wiley & sons, 2009.
- [34] Shoham Y, Leyton-Brown K. Multiagent systems: algorithmic, game-theoretic, and logical foundations[M]. Cambridge University Press, 2008.
- [35] Chen S, An Q, Zhou H, et al. Observer-based consensus for fractional-order multi-agent systems with positive constraint[J]. Neurocomputing, 2022, 501: 489-498.
- [36] Zhang Y, Zhang S, Zhang Y, et al. A study of TMA aircraft conflict-free routing and operation: with mixed integer linear programming, multi-agent path finding, and metaheuristic-based neighborhood search[J]. IEEE Transactions on Intelligent Transportation Systems, 2024, 25(10): 13976-13990.
- [37] Smith R G. The contract net protocol: high-level communication and control in a distributed problem solver[J]. IEEE Transactions on Computers, 1980, 29(12): 1104-1113.
- [38] Choi H L, Brunet L, How J P. Consensus-based decentralized auctions for robust task allocation[J]. IEEE Transactions on Robotics, 2009, 25(4): 912-926.
- [39] Li K, Hu Y, Wang J, et al. Task prioritization in multiagent environments: a novel approach using nash Q-learning[J]. IEEE Transactions on Consumer Electronics, 2025.
- [40] Li M, Sun H, Huang Y, et al. Shapley value: from cooperative game to explainable artificial intelligence[J]. Autonomous Intelligent Systems, 2024, 4(1): 2.
- [41] Watkins C J C H, Dayan P. Q-learning[J]. Machine Learning, 1992, 8(3): 279-292.
- [42] Iqbal S, de Witt C A S, Peng B, et al. Ai-qmix: attention and imagination for dynamic multi-agent reinforcement learning[J]. Arxiv Preprint Arxiv:2006.04222, 2020.
- [43] Jia Z, Yu J, Ai X, et al. Cooperative multiple task assignment problem with stochastic velocities and time windows for heterogeneous unmanned aerial vehicles using a genetic algorithm[J]. Aerospace Science and Technology, 2018, 76: 112-125.
- [44] Jain M, Saihjpai V, Singh N, et al. An overview of variants and advancements of PSO algorithm[J]. Applied Sciences, 2022, 12(17): 8392.
- [45] Wu H, Li H, Xiao R, et al. Modeling and simulation of dynamic ant colony's labor division for task

- allocation of UAV swarm[J]. *Physica A: Statistical Mechanics and its Applications*, 2018, 491: 127-141.
- [46] O'Hare G M, Jennings N R. *Foundations of distributed artificial intelligence: Vol. 9*[M]. John Wiley & Sons, 1996.
- [47] Hayes B, Scassellati B. Autonomously constructing hierarchical task networks for planning and human-robot collaboration[C]//2016 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2016: 5469-5476.
- [48] Luo X, Xu S, Liu R, et al. Decomposition-based hierarchical task allocation and planning for multi-robots under hierarchical temporal logic specifications[J]. *IEEE Robotics and Automation Letters*, 2024, 9(8): 7182-7189.
- [49] Parker L E, Rus D, Sukhatme G S. *Multiple mobile robot systems*[M]//Springer Handbook of Robotics. Springer, 2016: 1335-1384.
- [50] Dias M B, Zlot R, Kalra N, et al. Market-based multirobot coordination: a survey and analysis[J]. *Proceedings of the IEEE*, 2006, 94(7): 1257-1270.
- [51] Mosteo A R, Montano L. A survey of multi-robot task allocation[J]. *Instituto de Investigacin en Ingenier\la de Aragn (I3A), Tech. Rep*, 2010.
- [52] Liekna A, Lavendelis E, Grabovskis A. Experimental analysis of contract net protocol in multi-robot task allocation[J]. *Applied Computer Systems*, 2012, 13(1): 6-14.
- [53] 王松, 陈功. 基于合同网协议的分布式反导资源分配算法[J]. *计算机科学*, 2024, 51(S2): 723-729.
- [54] Yeung W L. Efficiency of task allocation based on contract net protocol with audience restriction in a manufacturing control application[J]. *International Journal of Computer Integrated Manufacturing*, 2018, 31(10): 1005-1017.
- [55] Du B, Li S. A new multi-satellite autonomous mission allocation and planning method[J]. *Acta Astronautica*, 2019, 163: 287-298.
- [56] Keykhaei M, Samany N N, Jelokhani-Niaraki M, et al. Multi-agent-based human cognition simulation of situation-aware earthquake emergency evacuation[J]. *International Journal of Disaster Risk Reduction*, 2024, 100: 104183.
- [57] Li Y, Huang Y, Zou Z, et al. Multi-AUV underwater static target search method based on consensus-based bundle algorithm and improved gladius bio-inspired neural network[J]. *Information Sciences*, 2024, 673: 120684.
- [58] Bi W, Shen J, Zhou J, et al. Heterogeneous multi-UAV mission reallocation based on improved

- consensus-based bundle algorithm[J]. Drones (2504-446X), 2024, 8(8).
- [59] Xiao J, Gao Q, Yang Z, et al. Multi-round auction-based resource allocation for edge computing: maximizing social welfare[J]. Future Generation Computer Systems, 2023, 140: 365-375.
- [60] K A A, J D U, Subramaniam U. A systematic literature review on multi-robot task allocation[J]. ACM Computing Surveys, 2025, 57(3): 1-28.
- [61] Whitbrook A, Meng Q, Chung P W. Reliable, distributed scheduling and rescheduling for time-critical, multiagent systems[J]. IEEE Transactions on Automation Science and Engineering, 2017, 15(2): 732-747.
- [62] Zhao W, Meng Q, Chung P W. A heuristic distributed task allocation method for multivehicle multitask problems and its application to search and rescue scenario[J]. IEEE Transactions on Cybernetics, 2015, 46(4): 902-915.
- [63] Gallud X, Selva D. Agent-based simulation framework and consensus algorithm for observing systems with adaptive modularity[J]. Systems Engineering, 2018, 21(5): 432-454.
- [64] Kim K S, Kim H Y, Choi H L. A bid-based grouping method for communication-efficient decentralized multi-UAV task allocation[J]. International Journal of Aeronautical and Space Sciences, 2020, 21(1): 290-302.
- [65] Chen X, Zhang P, Li F, et al. A cluster first strategy for distributed multi-robot task allocation problem with time constraints[C]//2018 WRC Symposium on Advanced Robotics and Automation (WRC SARA). IEEE, 2018: 102-107.
- [66] Nelke S A, Okamoto S, Zivan R. Market clearing-based dynamic multi-agent task allocation[J]. ACM Transactions on Intelligent Systems and Technology (TIST), 2020, 11(1): 1-25.
- [67] Su H H, Wein N. Lower bounds for dynamic distributed task allocation[A]. arXiv, 2020.
- [68] Farinelli A, Rogers A, Jennings N R. Agent-based decentralised coordination for sensor networks using the max-sum algorithm[J]. Autonomous Agents and Multi-Agent Systems, 2014, 28(3): 337-380.
- [69] Clearwater S H. Market-based control: a paradigm for distributed resource allocation[M]. World Scientific, 1996.
- [70] Wellman M P. A market-oriented programming environment and its application to distributed multicommodity flow problems[J]. Journal of Artificial Intelligence Research, 1993, 1: 1-23.
- [71] Quinton F, Grand C, Lesire C. Market approaches to the multi-robot task allocation problem: a survey[J]. Journal of Intelligent and Robotic Systems, 2023, 107(2): 29.
- [72] Arslan G, Marden J R, Shamma J S. Autonomous vehicle-target assignment: a game-theoretical

- p>formulation[J]. 2007.
- [73] Kreps D M. Nash equilibrium[M]//Eatwell J, Milgate M, Newman P. Game Theory. London: Palgrave Macmillan UK, 1989: 167-177.
- [74] Ye M, Han Q L, Ding L, et al. Distributed nash equilibrium seeking in games with partial decision information: a survey[J]. Proceedings of the IEEE, 2023, 111(2): 140-157.
- [75] Chen H, Covert I C, Lundberg S M, et al. Algorithms to estimate shapley value feature attributions[J]. Nature Machine Intelligence, 2023, 5(6): 590-601.
- [76] Jang I, Shin H S, Tsourdos A. Anonymous hedonic game for task allocation in a large-scale multiple agent system[J]. IEEE Transactions on Robotics, 2018, 34(6): 1534-1548.
- [77] Xie B, Gu X, Chen J, et al. A multi-responsibility-oriented coalition formation framework for dynamic task allocation in mobile-distributed multi-agent systems[J]. International Journal of Advanced Robotic Systems, 2018, 15(6): 1729881418813037.
- [78] Makkapati V R, Tsiotras P. Optimal evading strategies and task allocation in multi-player pursuit-evasion problems[J]. Dynamic Games and Applications, 2019, 9(4): 1168-1187.
- [79] Fu X, Zhang J, Zhang L, et al. Coalition formation among unmanned aerial vehicles for uncertain task allocation[J]. Wireless Networks, 2019, 25(1): 367-377.
- [80] Roldán J J, Del Cerro J, Barrientos A. Should we compete or should we cooperate? Applying game theory to task allocation in drone swarms[C]//2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2018: 5366-5371.
- [81] Dai W, Lu H, Xiao J, et al. Task allocation without communication based on incomplete information game theory for multi-robot systems[J]. Journal of Intelligent and Robotic Systems, 2019, 94(3): 841-856.
- [82] Kraicz S, Robinson I, Elkind E. Streamlining equal shares[A]. arXiv, 2025.
- [83] Sebbane Y B, Sebbane Y B. Planning and decision making for aerial robots[M]. Springer, 2014.
- [84] Zhang Z, Jiang J, Haiyan X U, et al. Distributed dynamic task allocation for unmanned aerial vehicle swarm systems: a networked evolutionary game-theoretic approach[J]. Chinese Journal of Aeronautics, 2024, 37(6): 182-204.
- [85] Nouredine D B, Gharbi A, Ahmed S B. Multi-agent deep reinforcement learning for task allocation in dynamic environment[C]//12th International Conference on Software Technologies. 2017.
- [86] Guo H, Miao Z, Ji J C, et al. An effective collaboration evolutionary algorithm for multi-robot task allocation and scheduling in a smart farm[J]. Knowledge-Based Systems, 2024, 289: 111474.

- [87] Li Z, Shi N, Zhao L, et al. Deep reinforcement learning path planning and task allocation for multi-robot collaboration[J]. Alexandria Engineering Journal, 2024, 109: 408-423.
- [88] Kaelbling L P, Littman M L, Moore A W. Reinforcement learning: a survey[J]. Journal of Artificial Intelligence Research, 1996, 4: 237-285.
- [89] Clifton J, Laber E. Q-learning: theory and applications[J]. Annual Review of Statistics and Its Application, 2020, 7(1): 279-301.
- [90] Wang X, Wang S, Liang X, et al. Deep reinforcement learning: a survey[J]. IEEE Transactions on Neural Networks and Learning Systems, 2024, 35(4): 5064-5078.
- [91] Canese L, Cardarilli G C, Di Nunzio L, et al. Multi-agent reinforcement learning: a review of challenges and applications[J]. Applied Sciences, 2021, 11(11): 4948.
- [92] Albrecht S V, Christianos F, Schäfer L. Multi-agent reinforcement learning: foundations and modern approaches[M]. MIT Press, 2024.
- [93] Marinescu A, Dusparic I, Clarke S. Prediction-based multi-agent reinforcement learning in inherently non-stationary environments[J]. ACM Trans. Auton. Adapt. Syst., 2017, 12(2): 9:1-9:23.
- [94] Risukhin A, Rao K, Caffee B, et al. ColorGrid: a multi-agent non-stationary environment for goal inference and assistance[A]. arXiv, 2025.
- [95] Majkowska A, Zydek D, Koszalka L. Task allocation in distributed mesh-connected machine learning system: simplified busy list algorithm with Q-learning based queuing[M]//Advances in Intelligent Systems and Computing: Vol. 226. 2013: 763-772.
- [96] Yu T, Huang J, Chang Q. Optimizing task scheduling in human-robot collaboration with deep multi-agent reinforcement learning[J]. Journal of Manufacturing Systems, 2021, 60: 487-499.
- [97] Xu Y H, Yang C C, Hua M, et al. Deep deterministic policy gradient (DDPG)-based resource allocation scheme for NOMA vehicular communications[J]. IEEE Access, 2020, 8: 18797-18807.
- [98] Wei X, Yang L, Cao G, et al. Recurrent MADDPG for object detection and assignment in combat tasks[J]. Ieee Access, 2020, 8: 163334-163343.
- [99] Zhang M, Tong W, Zhu G, et al. SQIX: QMIX algorithm activated by general softmax operator for cooperative multiagent reinforcement learning[J]. IEEE Transactions on Systems, Man, and Cybernetics: Systems, 2024, 54(11): 6550-6560.
- [100] 江泰民, 谭泰, 李辉, 等. 基于分层深度强化学习的六自由度固定翼无人机路径跟踪方法[J]. 计算机工程, 2024.
- [101] Vinyals O, Babuschkin I, Czarnecki W M, et al. Grandmaster level in StarCraft II using multi-agent

- reinforcement learning[J]. *Nature*, 2019, 575(7782): 350-354.
- [102]Ding C, Zheng Z. Multi-agent collaborative operation planning via cross-domain transfer learning[J]. *Knowledge-Based Systems*, 2025: 113172.
- [103]Hu Y, Chen M, Saad W, et al. Distributed multi-agent meta learning for trajectory design in wireless drone networks[J]. *IEEE Journal on Selected Areas in Communications*, 2021, 39(10): 3177-3192.
- [104]Badreldin M, Hussein A, Khamis A. A comparative study between optimization and market-based approaches to multi-robot task allocation[J]. *Advances in Artificial Intelligence*, 2013, 2013: 12-12.
- [105]Yoon S, Kim J. Efficient multi-agent task allocation for collaborative route planning with multiple unmanned vehicles[J]. *IFAC-PapersOnLine*, 2017, 50(1): 3580-3585.
- [106]Yoon S, Kim J. Fleet size optimization and collaborative route planning for multi-vehicle task allocation[C]//*AIAA Scitech 2019 Forum*. 2019: 1572.
- [107]Flushing E F, Gambardella L M, Di Caro G A. Simultaneous task allocation, data routing, and transmission scheduling in mobile multi-robot teams[C]//*2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017: 1861-1868.
- [108]Zhang S, Xue Y, Zhang H, et al. Improved hungarian algorithm-based task scheduling optimization strategy for remote sensing big data processing[J]. *Geo-spatial Information Science*, 2024.
- [109]Kuhn H W. The hungarian method for the assignment problem[J]. *Naval Research Logistics Quarterly*, 1955, 2(1-2): 83-97.
- [110]Odili J, Kahar M N M, Noraziah A, et al. A comparative evaluation of swarm intelligence techniques for solving combinatorial optimization problems[J]. *International Journal of Advanced Robotic Systems*, 2017, 14(3): 1729881417705969.
- [111]Tang J, Liu G, Pan Q. A review on representative swarm intelligence algorithms for solving optimization problems: applications and trends[J]. *IEEE/CAA Journal of Automatica Sinica*, 2021, 8(10): 1627-1643.
- [112]Tang J, Duan H, Lao S. Swarm intelligence algorithms for multiple unmanned aerial vehicles collaboration: a comprehensive review[J]. *Artificial Intelligence Review*, 2023, 56(5): 4295-4327.
- [113]Alshawhi M A, Shalan M B. Minimal time dynamic task allocation for a swarm of robots[J]. *International Journal of Mechanical Engineering and Robotics Research*, 2017, 6(6): 481-487.
- [114]Huang H, Zhuo T. Multi-model cooperative task assignment and path planning of multiple UCAV formation[J]. *Multimedia Tools and Applications*, 2019, 78(1): 415-436.
- [115]Sun B, Zeng Y, Zhu D. Dynamic task allocation in multi autonomous underwater vehicle

- confrontational games with multi-objective evaluation model and particle swarm optimization algorithm[J]. *Applied Soft Computing*, 2024, 153: 111295.
- [116]Wang C, Mu D, Zhao F, et al. A parallel simulated annealing method for the vehicle routing problem with simultaneous pickup–delivery and time windows[J]. *Computers & Industrial Engineering*, 2015, 83: 111-122.
- [117]Alarabi S, Lei T, Santora M, et al. Multi-robot path planning using potential field-based simulated annealing approach[C]//*Unmanned Systems Technology XXVI: Vol. 13055*. SPIE, 2024: 102-117.
- [118]Zhou X, Wang H, Ding B, et al. Balanced connected task allocations for multi-robot systems: an exact flow-based integer program and an approximate tree-based genetic algorithm[J]. *Expert Systems with Applications*, 2019, 116: 10-20.
- [119]Gao S, Wu J, Ai J. Multi-UAV reconnaissance task allocation for heterogeneous targets using grouping ant colony optimization algorithm[J]. *Soft Computing*, 2021, 25(10): 7155-7167.
- [120]Soleimanpour-Moghadam M, Nezamabadi-Pour H. Discrete genetic algorithm for solving task allocation of multi-robot systems[C]//*2020 4th Conference on Swarm Intelligence and Evolutionary Computation (CSIEC)*. IEEE, 2020: 6-9.
- [121]Wang S, Liu Y, Qiu Y, et al. Cooperative task allocation for multi-robot systems based on multi-objective ant colony system[J]. *IEEE Access*, 2022, 10: 56375-56387.
- [122]李华峰, 王富臣, 江广冀, 等. 蚁群优化算法驱动的 DQN 多任务协同调度方法研究[J]. *电气自动化*, 2025, 47(1): 5-7.
- [123]Awadallah M A, Makhadmeh S N, Al-Betar M A, et al. Multi-objective ant colony optimization: review[J]. *Archives of Computational Methods in Engineering*, 2024.
- [124]Tkach I, Jevtić A, Nof S Y, et al. A modified distributed bees algorithm for multi-sensor task allocation[J]. *Sensors*, 2018, 18(3): 759.
- [125]Xu W, Yang H, Ji Z, et al. The bees algorithm[J]. *Intelligent Engineering Optimisation with the Bees Algorithm*, 2024: 209.
- [126]Schwarzrock J, Zacarias I, Bazzan A L, et al. Solving task allocation problem in multi unmanned aerial vehicles systems using swarm intelligence[J]. *Engineering Applications of Artificial Intelligence*, 2018, 72: 10-20.
- [127]Bonabeau E, Theraulaz G, Deneubourg J L. Fixed response thresholds and the regulation of division of labor in insect societies[J]. *Bulletin of Mathematical Biology*, 1998, 60(4): 753-807.
- [128]Beshers S N. Regulation of division of labor in insects: a colony-level perspective[J]. *Current Opinion*

- in *Insect Science*, 2024, 61: 101155.
- [129]Tan Y, Zheng Z yang. Research advance in swarm robotics[J]. *Defence Technology*, 2013, 9(1): 18-39.
- [130]Santos L A, Maia S M, Batista J G, et al. Recent advances in swarm mobile robotics[J]. *Journal of Mechatronics Engineering*, 2024, 7(1): 1-24.
- [131]Claus C, Boutilier C. The dynamics of reinforcement learning in cooperative multiagent systems[J]. *Aaai/iaai*, 1998, 1998(746-752): 2.
- [132]Tan M. Multi-agent reinforcement learning: independent vs. cooperative agents[C]//*Proceedings of the tenth international conference on machine learning*. 1993: 330-337.
- [133]Foerster J, Assael I A, De Freitas N, et al. Learning to communicate with deep multi-agent reinforcement learning[J]. *Advances in neural information processing systems*, 2016, 29.
- [134]Gupta J K, Egorov M, Kochenderfer M. Cooperative multi-agent control using deep reinforcement learning[M]//Sukthankar G, Rodriguez-Aguilar J A. *Autonomous Agents and Multi-Agent Systems: Vol. 10642*. Cham: Springer International Publishing, 2017: 66-83.
- [135]Mahajan A, Rashid T, Samvelyan M, et al. Maven: multi-agent variational exploration[J]. *Advances in neural information processing systems*, 2019, 32.
- [136]Das A, Gervet T, Romoff J, et al. TarMAC: targeted multi-agent communication[C]//*Proceedings of the 36th International Conference on Machine Learning*. PMLR, 2019: 1538-1546.
- [137]Wang J, Ren Z, Han B, et al. Towards understanding linear value decomposition in cooperative multi-agent q-learning[J]. 2020.
- [138]Kraemer L, Banerjee B. Multi-agent reinforcement learning as a rehearsal for decentralized planning[J]. *Neurocomputing*, 2016, 190: 82-94.
- [139]Bezerra L C D, Santos A M G dos, Park S. Learning policies for dynamic coalition formation in multi-robot task allocation[A]. *arXiv*, 2024.
- [140]Rashid T, Samvelyan M, De Witt C S, et al. Monotonic value function factorisation for deep multi-agent reinforcement learning[J]. *Journal of Machine Learning Research*, 2020, 21(178): 1-51.
- [141]Son K, Kim D, Kang W J, et al. Qtran: learning to factorize with transformation for cooperative multi-agent reinforcement learning[C]//*International conference on machine learning*. PMLR, 2019: 5887-5896.
- [142]Yang Y, Hao J, Liao B, et al. Qatten: a general framework for cooperative multiagent reinforcement learning[A]. *arXiv*, 2020.

- [143]Wang J, Ren Z, Liu T, et al. QPLEX: duplex dueling multi-agent Q-learning[A]. arXiv, 2021.
- [144]Liu J, Wang Y, Pan D, et al. QoS-aware task offloading and resource allocation optimization in vehicular edge computing networks via MADDPG[J]. Computer Networks, 2024, 242: 110282.
- [145]Yang Y, Luo R, Li M, et al. Mean field multi-agent reinforcement learning[C]//Proceedings of the 35th International Conference on Machine Learning. PMLR, 2018: 5571-5580.
- [146]Guo F, Wu Z. Learning maximum entropy policies with qmix in cooperative marl[C]//2022 IEEE 2nd International Conference on Electronic Technology, Communication and Information (ICETCI). IEEE, 2022: 357-361.
- [147]Liu J, Li F, Jin X, et al. Distributed task allocation for multi-agent systems: a submodular optimization approach[A]. arXiv, 2024.
- [148]Soliman N. Characterizing and predicting tasks at risk in team task management[D]. Massachusetts Institute of Technology, 2022.
- [149]Wooldridge M, Jennings N R, Kinny D. The gaia methodology for agent-oriented analysis and design[J]. 2000, 3: 285-312.
- [150]Omicini A. SODA: societies and infrastructures in the analysis and design of agent-based systems[M]//Ciancarini P, Wooldridge M J. Agent-Oriented Software Engineering: Vol. 1957. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001: 185-193.
- [151]Padgham L, Winikoff M. Prometheus: a methodology for developing intelligent agents[C]//Proceedings of the first international joint conference on Autonomous agents and multiagent systems part 1 - AAMAS '02. Bologna, Italy: ACM Press, 2002: 37.
- [152]Cossentino M, Gaglio S, Sabatucci L, et al. The PASSI and agile PASSI MAS meta-models compared with a unifying proposal[M]//Pěchouček M, Petta P, Varga L Z. Multi-Agent Systems and Applications IV: Vol. 3690. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005: 183-192.
- [153]Zhu H, Zhou M. Role-based multi-agent systems[M]//Personalized information retrieval and access: concepts, methods and practices. IGI Global, 2008: 254-285.
- [154]Spanoudakis N, Moraitis P. Using ASEME methodology for model-driven agent systems development[M]//Weyns D, Gleizes M P. Agent-Oriented Software Engineering XI: Vol. 6788. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011: 106-127.
- [155]DeLoach S A, Ojeda J C G. O-MaSE: a customisable approach to designing and building complex, adaptive multi-agent systems[J]. International Journal of Agent-Oriented Software Engineering, 2010, 4(3): 244.

- [156]Sun C, Liu W, Dong L. Reinforcement learning with task decomposition for cooperative multiagent systems[J]. IEEE Transactions on Neural Networks and Learning Systems, 2020, 32(5): 2054-2065.
- [157]Lhaksmana K M, Murakami Y, Ishida T. Role-based modeling for designing agent behavior in self-organizing multi-agent systems[J]. International Journal of Software Engineering and Knowledge Engineering, 2018, 28(1): 79-96.
- [158]Wang D, Zhong F, Li M, et al. Romat: role-based multi-agent transformer for generalizable heterogeneous cooperation[J]. Neural Networks, 2024, 174: 106129.
- [159]Wilson A, Fern A, Tadepalli P. Bayesian policy search for multi-agent role discovery[C]//Proceedings of the AAAI Conference on Artificial Intelligence: Vol. 24. 2010: 624-629.
- [160]Wang T, Dong H, Lesser V, et al. ROMA: multi-agent reinforcement learning with emergent roles[A]. arXiv, 2020.
- [161]Le H M, Yue Y, Carr P, et al. Coordinated multi-agent imitation learning[C]//International Conference on Machine Learning. PMLR, 2017: 1995-2003.
- [162]Nguyen D, Nguyen P, Venkatesh S, et al. Learning to transfer role assignment across team sizes[A]. arXiv, 2022.
- [163]Hu S, Xie C, Liang X, et al. Policy diagnosis via measuring role diversity in cooperative multi-agent RL[C]//International Conference on Machine Learning. PMLR, 2022: 9041-9071.
- [164]Wang T, Gupta T, Mahajan A, et al. RODE: learning roles to decompose multi-agent tasks[A]. arXiv, 2020.
- [165]Christianos F, Papoudakis G, Rahman M A, et al. Scaling multi-agent reinforcement learning with selective parameter sharing[C]//International Conference on Machine Learning. PMLR, 2021: 1989-1998.
- [166]Samvelyan M, Rashid T, de Witt C S, et al. The StarCraft multi-agent challenge[A]. arXiv, 2019.
- [167]Oliehoek F A, Amato C. A concise introduction to decentralized POMDPs[M]. Cham: Springer International Publishing, 2016.
- [168]Ong S C, Png S W, Hsu D, et al. POMDPs for robotic tasks with mixed observability[C]//Robotics: Science and systems: Vol. 5. 2009.
- [169]Hausknecht M J, Stone P. Deep recurrent Q-learning for partially observable MDPs[C]//AAAI Fall Symposia. 2015.
- [170]Ha D, Dai A, Le Q V. HyperNetworks[A]. arXiv, 2016.
- [171]Sunehag P, Lever G, Gruslys A, et al. Value-decomposition networks for cooperative multi-agent

- learning[A]. arXiv, 2017.
- [172]Skaltsis G M, Shin H S, Tsourdos A. A survey of task allocation techniques in MAS[C]//2021 International Conference on Unmanned Aircraft Systems (ICUAS). IEEE, 2021: 488-497.
- [173]Seenu N, Rm K C, Ramya M M, et al. Review on state-of-the-art dynamic task allocation strategies for multiple-robot systems[J]. Industrial Robot: the international journal of robotics research and application, 2020, 47(6): 929-942.
- [174]Luo L, Chakraborty N, Sycara K. Distributed algorithms for multirobot task assignment with task deadline constraints[J]. IEEE Transactions on Automation Science and Engineering, 2015, 12(3): 876-888.
- [175]Otte M, Kuhlman M J, Sofge D. Auctions for multi-robot task allocation in communication limited environments[J]. Autonomous Robots, 2020, 44(3-4): 547-584.
- [176]Ye F, Chen J, Sun Q, et al. Decentralized task allocation for heterogeneous multi-UAV system with task coupling constraints[J]. Journal of Supercomputing, 2021, 77(1): 111-132.
- [177]Nayak S, Yeotikar S, Carrillo E, et al. Experimental comparison of decentralized task allocation algorithms under imperfect communication[J]. IEEE Robotics and Automation Letters, 2020, 5(2): 572-579.
- [178]Turner J, Meng Q, Schaefer G, et al. Distributed task rescheduling with time constraints for the optimization of total task allocations in a multirobot system[J]. IEEE Transactions on Cybernetics, 2017, 48(9): 2583-2597.
- [179]Whitbrook A, Meng Q, Chung P. Addressing robustness in time-critical, distributed, task allocation algorithms[J]. Applied Intelligence, 2019, 49.
- [180]Yang M, Bi W, Zhang A, et al. A distributed task reassignment method in dynamic environment for multi-UAV system[J]. Applied Intelligence, 2022, 52(2): 1582-1601.
- [181]Qamar R A, Sarfraz M, Ghauri S, et al. TRMaxAlloc: maximum task allocation using reassignment algorithm in multi-UAV system[J]. Comput. Commun., 2023.
- [182]Jiang Y. A survey of task allocation and load balancing in distributed systems[J]. IEEE Transactions on Parallel and Distributed Systems, 2015, 27(2): 585-599.
- [183]Tang X, Chanson S T. Optimizing static job scheduling in a network of heterogeneous computers[C]//Proceedings 2000 International Conference on Parallel Processing. 2000: 373-382.
- [184]Zomaya A Y, Teh Y H. Observations on using genetic algorithms for dynamic load-balancing[J]. IEEE Transactions on Parallel and Distributed Systems, 2001, 12(9): 899-911.

- [185]Zhang M, Li H, Liu L, et al. An adaptive multi-objective evolutionary algorithm for constrained workflow scheduling in clouds[J]. Distributed and Parallel Databases, 2018, 36(2): 339-368.
- [186]Kishor A, Niyogi R, Veeravalli B. Fairness-aware mechanism for load balancing in distributed systems[J]. IEEE Transactions on Services Computing, 2022, 15(4): 2275-2288.
- [187]Talukder A K M K A, Kirley M, Buyya R. Multiobjective differential evolution for scheduling workflow applications on global grids[J]. Concurrency and Computation: Practice and Experience, 2009, 21(13): 1742-1756.
- [188]Grosu D, Chronopoulos A T. Noncooperative load balancing in distributed systems[J]. Journal of Parallel and Distributed Computing, 2005, 65(9): 1022-1034.
- [189]Kameda H, Altman E. Inefficient noncooperation in networking games of common-pool resources[J]. IEEE Journal on Selected Areas in Communications, 2008, 26(7): 1260-1268.
- [190]Subrata R, Zomaya A Y, Landfeldt B. Game-theoretic approach for load balancing in computational grids[J]. IEEE Transactions on Parallel and Distributed Systems, 2008, 19(1): 66-76.
- [191]Tripathi R, Vignesh S, Tamarapalli V, et al. Non-cooperative power and latency aware load balancing in distributed data centers[J]. Journal of Parallel and Distributed Computing, 2017, 107: 76-86.
- [192]Li Z, Yu J, Liu X, et al. Load balancing for task scheduling based on multi-agent reinforcement learning in cloud-edge-end collaborative environments[C]//Proceedings of the 2024 8th International Conference on Machine Learning and Soft Computing. New York, NY, USA: Association for Computing Machinery, 2024: 94-100.
- [193]Baert Q, Caron A C, Morge M, et al. Fair multi-agent task allocation for large datasets analysis[J]. Knowledge and Information Systems, 2018, 54(3): 591-615.
- [194]Ebrahim M, Hafid A. Fully distributed fog load balancing with multi-agent reinforcement learning[A]. arXiv, 2024.
- [195]Omidi M. Fair and efficient resource scheduling in heterogeneous multi-agent systems[D]. University of Waterloo, 2024.
- [196]Wang Q, Li W, Mohajer A. Load-aware continuous-time optimization for multi-agent systems: toward dynamic resource allocation and real-time adaptability[J]. Computer Networks, 2024, 250: 110526.
- [197]Wu H, Ghadami A, Bayrak A E, et al. Task allocation with load management in multi-agent teams[C]//2022 International Conference on Robotics and Automation (ICRA). IEEE, 2022: 8823-8830.